한국성서대학교
KOREAN BIBLE UNIVERSITY

「2026년도 2학기 한국성서대학교 AI융합학부」

AIoT프로그래밍

중간 종합





목차

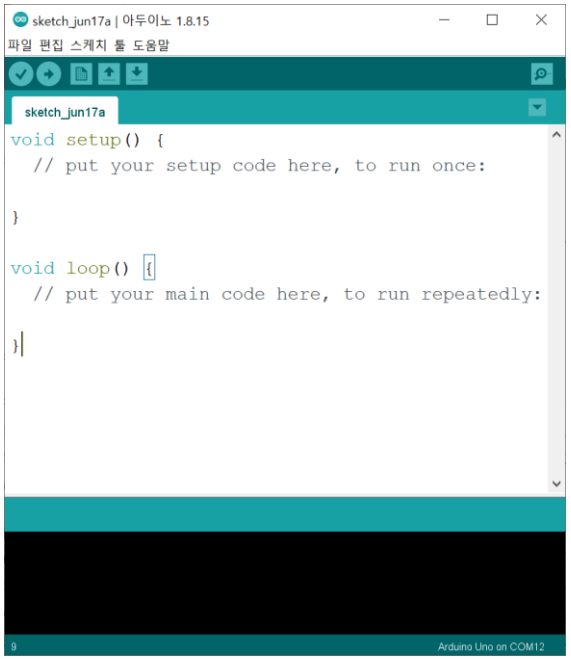
1. 아두이노 생태계
2. 아두이노 IDE 환경 설치 및 설정
3. 아두이노 기본 클래스
4. 아두이노 온라인 시뮬레이션

01 아두이노 생태계 탐험

아두이노 프로젝트 구성 요소

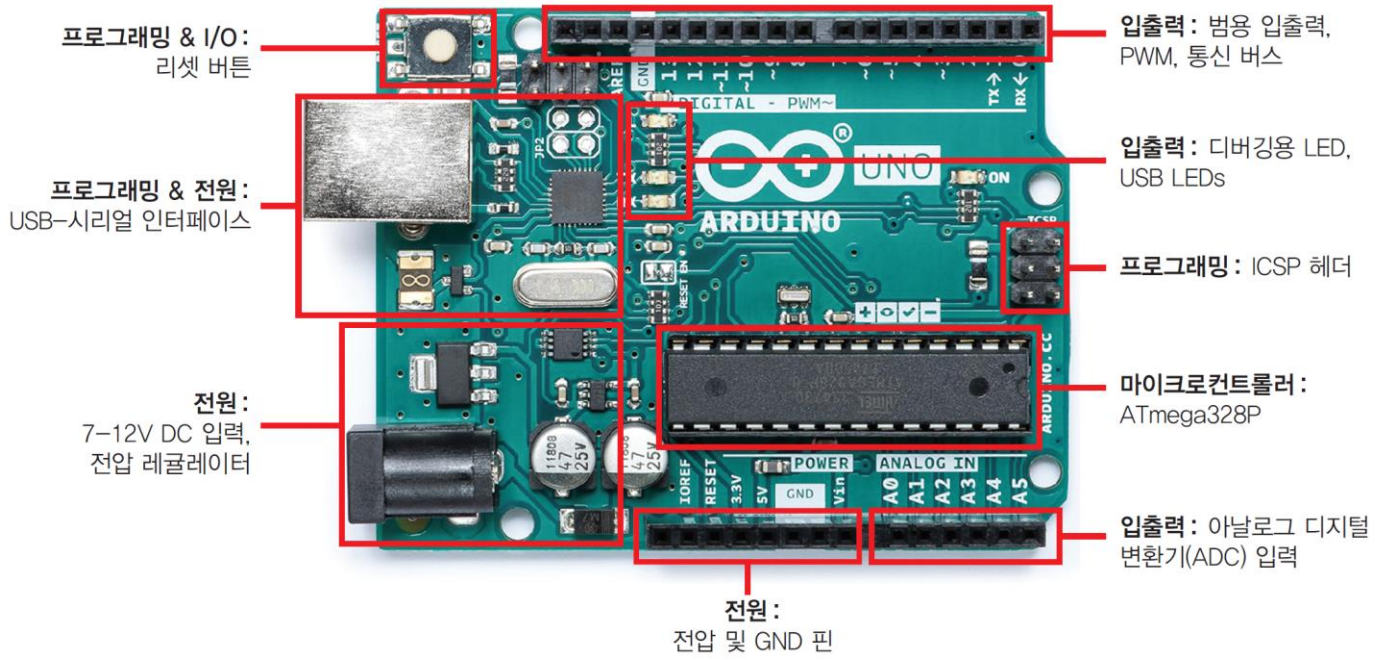
- 아두이노(Arduino)
  - 마이크로컨트롤러 플랫폼
  - 마이크로컨트롤러 보드+ 소프트웨어 개발 환경(IDE)
  - 오픈 소스 기반





아두이노 보드 : 구성요소

- 마이크로 컨트롤러 : 아두이노의 두뇌
- 프로그래밍 : 스케치 또는 펌웨어 업로드 지원
- 입출력 : 센서 등 입출력 장치 연결
- 전원 : 마이크로컨트롤러 및 보드의 동작 전압 공급



아두이노 보드 : 구성요소

- PWM은 Pulse Width Modulation (펄스 폭 변조)의 약자
- 전압을 연속적으로 바꿀 수 없는 디지털 핀에서 듀티 사이클(duty cycle, ON 되는 시간의 비율)을 조절하여 원하는 평균 전압을 생성
- 예를 들어 아두이노 우노 기준:
  - HIGH = 5V
  - LOW = 0V
  - 만약 50%의 듀티 사이클로 ON/OFF를 빠르게 반복하면 → 평균 약 2.5V처럼 동작
  - LED나 모터의 밝기나 세기를 변경 가능

아두이노 보드: 마이크로컨트롤러

- 초기 아두이노 보드는 아트멜(Atmel)에서 제작하는 Atmega 시리즈 마이크로컨트롤러 사용
- 스케치 또는 펌웨어의 컴파일 결과인 기계어 파일을 저장 및 실행
- 아두이노 우노 : 8비트 16MHz ATmega328P 사용

아두이노 보드: 프로그래밍 인터페이스

- 프로그램 업로드를 위해 사용
- 일반적인 방법
- 아두이노 우노의 경우
  - USB 연결만으로 USART (Universal Synchronous and Asynchronous Receiver/Transmitter) 통신을 통해 업로드 가능
    → 부트로더가 있어 가능
  - 아두이노 우노, 메가2560에는 USB 연결을 위한 ATmega16U2 마이크로컨트롤러가 포함되어 있음
  - 아두이노 레오나르도, Cortex-M0 기반 아두이노 보드는 마이크로컨트롤러 자체에서 USB 연결 지원

* USART(Universal Synchronous and Asynchronous Receiver/Transmitter) 통신: 별도의 클락으로 데이터를 타이머에 따라 송수신

아두이노 보드: 입력과 출력 / 전원

- 입력과 출력
  - 범용 입출력(GPIO) : 디지털 데이터 입출력을 위해 사용
  - ADC : 아날로그 데이터 입력을 위해 사용
  - 이외에 시리얼 통신, 펄스 폭 변조 신호 출력, 외부 인터럽트 등 지원
- 전원
  - 전원 공급 방법
    - USB 연결을 통한 5V 전원 공급
    - 배럴 잭이나 VIN 핀을 통한 6~20V 전원 공급 : 레귤레이터를 거쳐 5V 공급
  - 일부 아두이노 보드 및 확장 보드는 3.3V 전원을 사용하기도 함
    - 레귤레이터를 통해 3.3V 전원 역시 공급 가능

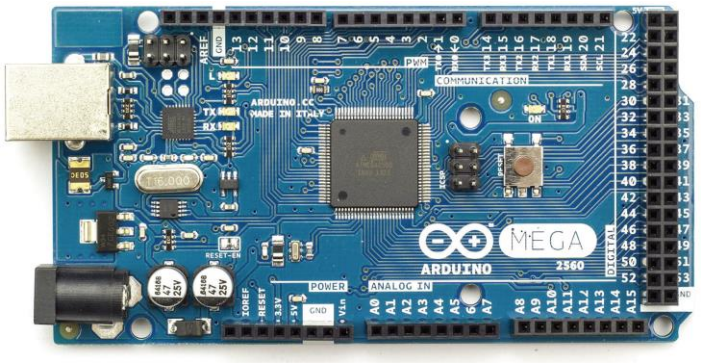
아두이노 보드 종류

- 아두이노 우노
  - 아두이노의 대표 보드이며 이 책에서 주로 사용
  - ATmega328P 마이크로컨트롤러 사용



아두이노 우노

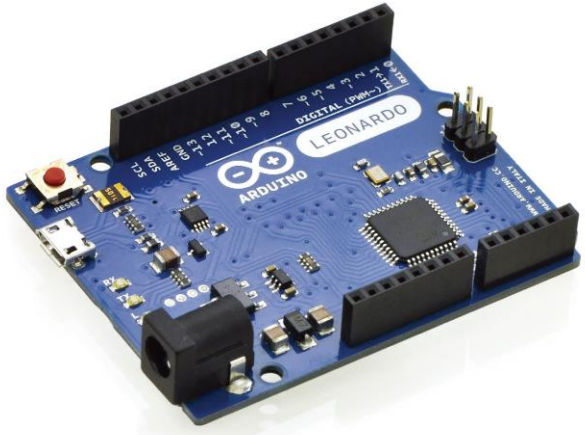
- 아두이노 메가2560
  - 아두이노 우노의 업그레이드 버전
  - 아두이노 우노보다 많은 수의 범용 입출력 핀, 큰 메모리, 많은 ADC 채널, 4개의 하드웨어 시리얼 인터페이스 등 지원



아두이노 메가2560

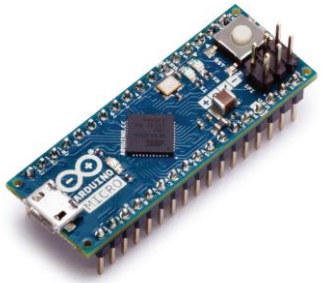
아두이노 보드 종류

- 아두이노 레오나르도
  - USB 인터페이스를 지원하는 ATmega32U4 마이크로컨트롤러 사용
  - 별도의 MCU 없이 USB 연결 가능
  - 조이스틱, 키보드 등 USB 장치를 만들기 위해 사용 가능



아두이노 레오나르도

- 아두이노 마이크로
  - 아두이노 레오나르도와 기능면에서 동일
  - 브레드보드에 연결하여 사용 가능한 소형 보드



아두이노 마이크로

아두이노 보드 종류

아두이노 듀에

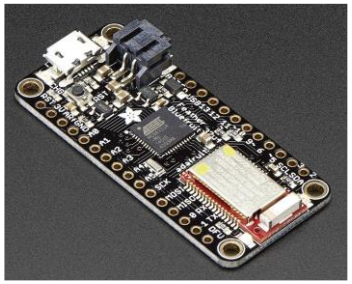
- Arm Cortex-M3 기반 SAM3X 마이크로컨트롤러 사용
- 높은 해상도의 ADC, DAC, USB 호스트, 84Mhz 클록, 32비트 CPU 등의 고성능 아두이노 보드



아두이노 듀에

Adafruit Feather 32U4 Bluefruit

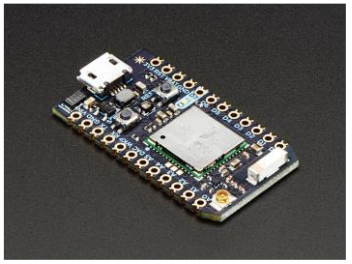
- 아두이노 레오나르도와 같은 ATmega32U4 마이크로컨트롤러 사용
- 블루투스 통신 기능 포함



Adafruit Feather 32U4 Bluefruit

Particle Photon

- Cortex-M3 기반 STM32F205 마이크로컨트롤러 사용
- 와이파이 통신 기능 포함



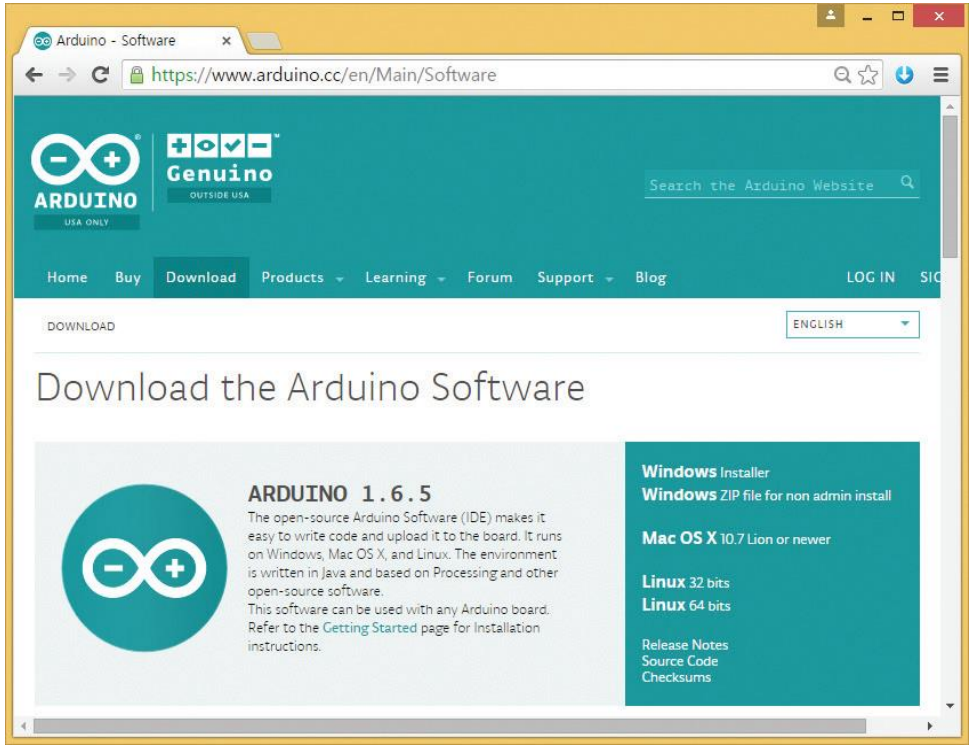
Particle Photon

아두이노 보드 vs 라즈베리파이

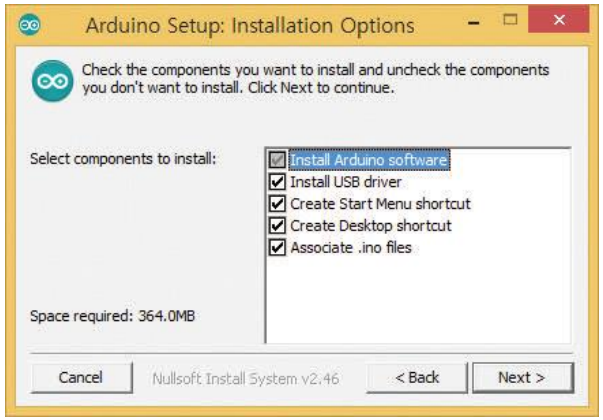
구분	아두이노	라즈베리파이
모양		
Spec	CPU : ATMega328 CPU Speed : 16Mhz RAM : 2K	CPU : ARM Cortex-A53 CPU Speed : 1.2Ghz RAM : 1G
OS	없음	라즈베리안(리눅스 계열), 리눅스, 윈도우
개발 언어	c언어	파이썬 ,웹프로그램,C언어등등
개발 방법	IDE(통합개발환경)에서 코딩 후 아두이노 보드로 업로드	os가 설치되어 있으므로 개발툴을 이용하여 개발 후 컴파일된 실행파일 수행

02 아두이노 IDE 환경 설치 및 설정

아두이노 IDE 설치 – 1

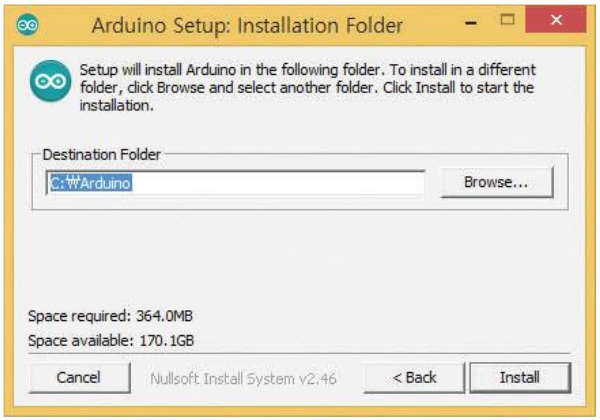


(1) IDE 다운로드 : www.arduino.cc

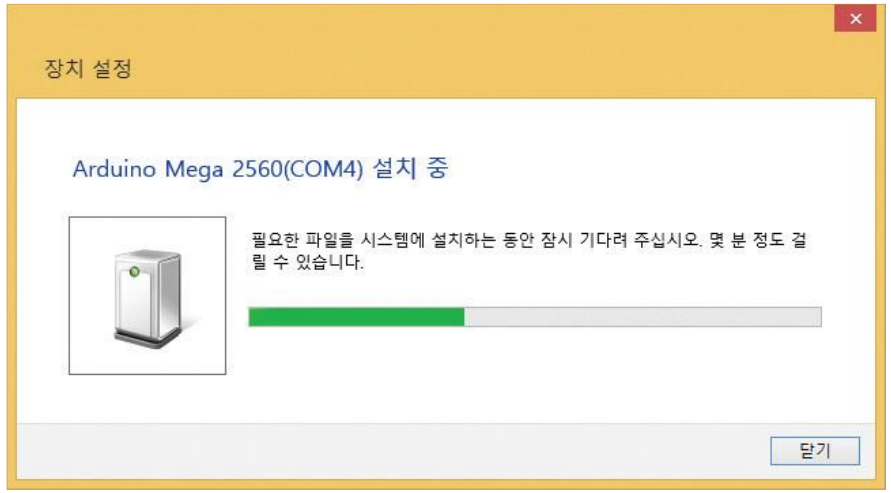


(2) 인스톨러 실행

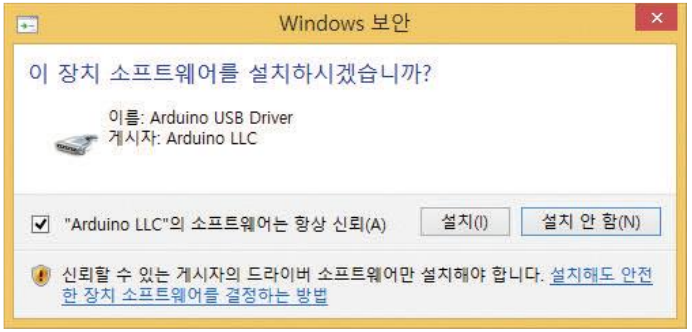
아두이노 IDE 설치 – 2



(3) 설치 디렉터리 지정



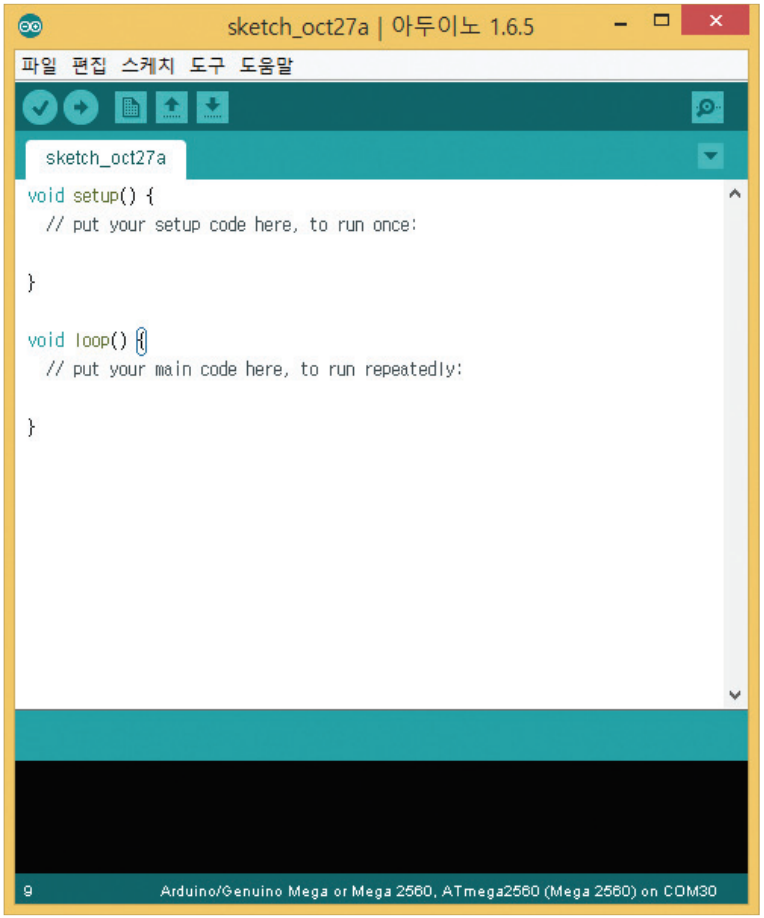
(5) 아두이노 보드 연결
: 가상 COM 포트 자동 설정



(4) 아두이노 보드 드라이버 자동 설치

아두이노 IDE

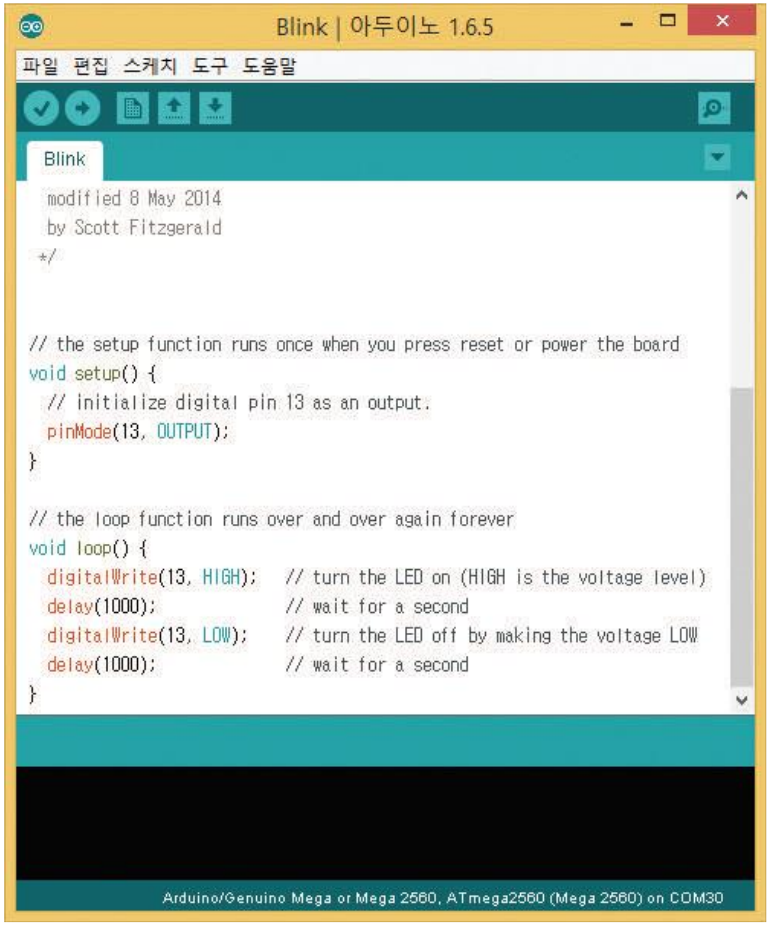
- 작고 간단한 통합개발환경
- 초보자를 위해 꼭 필요한 기능들만으로 구성
- 한 번의 클릭으로 컴파일에서 업로드까지 진행
- C로 구현되어 OS 간 이식성이 뛰어남
- 디버깅 기능은 제공하지 않음



첫 번째 아두이노 프로그램

- 아두이노에서 프로그램은 그림을 그리듯 쉽게 작성할 수 있다는 의미에서 '스케치(sketch)'라 불림

- 블링크 (Blink) 스케치
  - 13번 핀의 LED를 1초 간격으로 점멸하는 스케치
  - C 언어에서의 'Hello World'에 해당
  - '파일 → 예제 → 01.Basics → Blink' 선택



첫 번째 프로그램 자세히 살펴보기

- 줄 1~23 : 주석
  - '/*'로 시작하고 '*/'로 끝나는 여러 줄로 이루어진 주석
- 줄 25 : 한 줄 주석
  - 같은 줄에서 '//' 뒤의 내용은 주석으로 처리
- 줄 26~29 : 초기화 setup 함수
  - 프로그램 실행이 시작될 때 한 번만 실행
  - 핀의 입출력 설정, 통신 인터페이스 초기화 등의 작업 수행
- 줄 28 : 핀의 입출력 상태 설정 pinMode 함수
  - 아두이노의 디지털 핀은 입력 또는 출력으로 사용 가능
  - 입력과 출력으로 동시에 사용은 불가능하므로 pinMode 함수로 입력 또는 출력으로 사용하도록 지정
  - 디폴트 상태는 입력 상태

첫 번째 프로그램 자세히 살펴보기

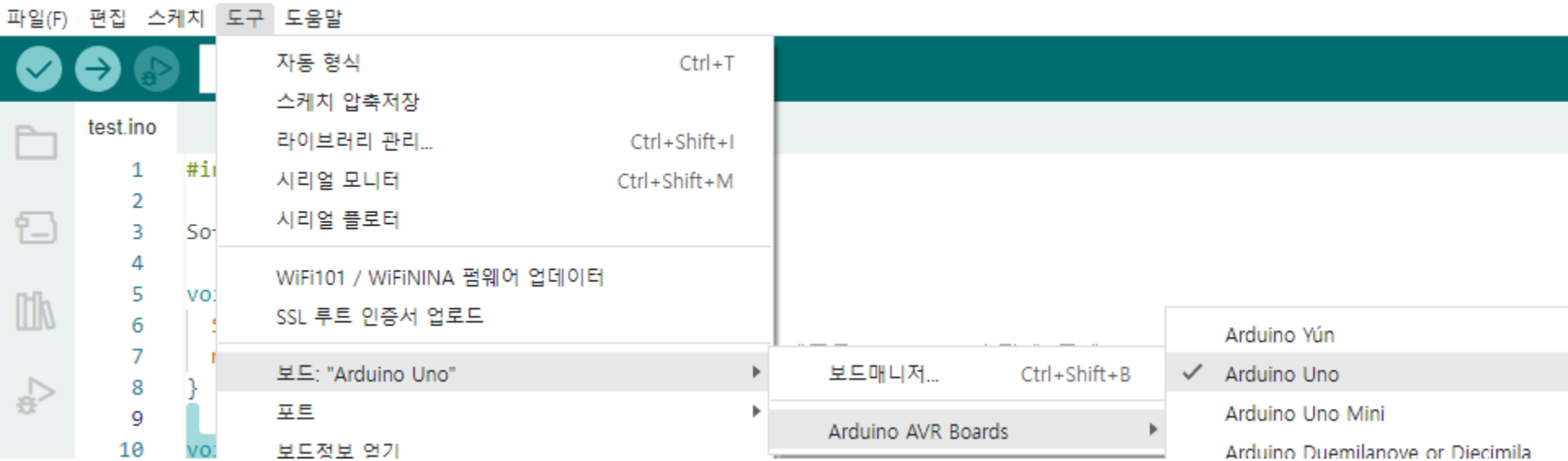
- 줄 28 : pinMode 함수
  - '/*'로 시작하고 '*/'로 끝나는 여러 줄로 이루어진 주석
  - 첫 번째는 핀 번호를 지정
    - 아두이노 우노에서 내장 LED는 13번 핀에 연결되어 있음
    - 아두이노 보드에 따라 내장 LED가 연결된 번호는 다를 수 있음
    - 13 대신 내장 LED가 연결된 핀 번호를 나타내는 LED_BUILTIN 사용 가능
  - 두 번째는 핀의 상태로 입력(INPUT) 또는 출력(OUTPUT)을 지정
- 줄 32~37 : 반복 수행 loop 함수
  - 초기화를 위한 setup 함수 수행 이후 반복해서 loop 함수가 호출됨
  - setup과 loop 함수는 아두이노의 기본 함수로 수행할 작업이 없어도 반드시 포함해야 함

첫 번째 프로그램 자세히 살펴보기

- 줄 33 : 디지털 데이터 출력 digitalWrite 함수
  - 2개의 매개변수를 가짐
  - 첫 번째는 핀 번호를 지정
  - 두 번째 매개변수는 핀으로 출력될 값 지정
    - HIGH(5V) 또는 LOW(0V) 지정
- 줄 34 : 시간 지연 delay 함수
  - 밀리초 단위의 지연 시간을 매개변수로 가짐
  - 블링크 예제에서는 1초, 즉, 1000밀리초 간격으로 점멸 상태가 바뀜

프로그램 업로드를 위한 설정 1

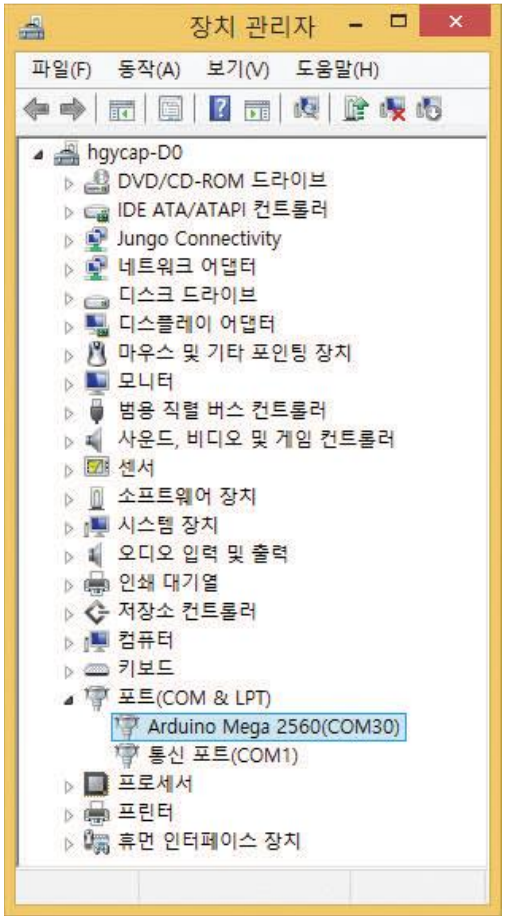
- 아두이노 보드 선택
  - 사용하고자 하는 보드를 선택
  - '도구 → 보드' 메뉴에서 'Arduino/Arduino Uno' 선택



프로그램 업로드를 위한 설정 3

- 포트 선택
  - 아두이노 보드에 할당된 가상 COM 포트 선택
  - '도구 → 포트' 메뉴에서 해당 포트 선택
  - 아두이노 보드에 할당된 포트는 장치관리자에서 확인





* 이 과목에서는 "Arduino Uno"선택

프로그램 업로드

- 툴바 사용
  - 확인 : 스케치 컴파일
  - 업로드 : 스케치 컴파일 및 생성된 기계어 파일을 아두이노로 업로드
    → 한 번의 클릭으로 업로드까지 진행
  - 시리얼 모니터 : 컴퓨터와의 데이터 송수신을 위한 프로그램 실행



스케치의 구조

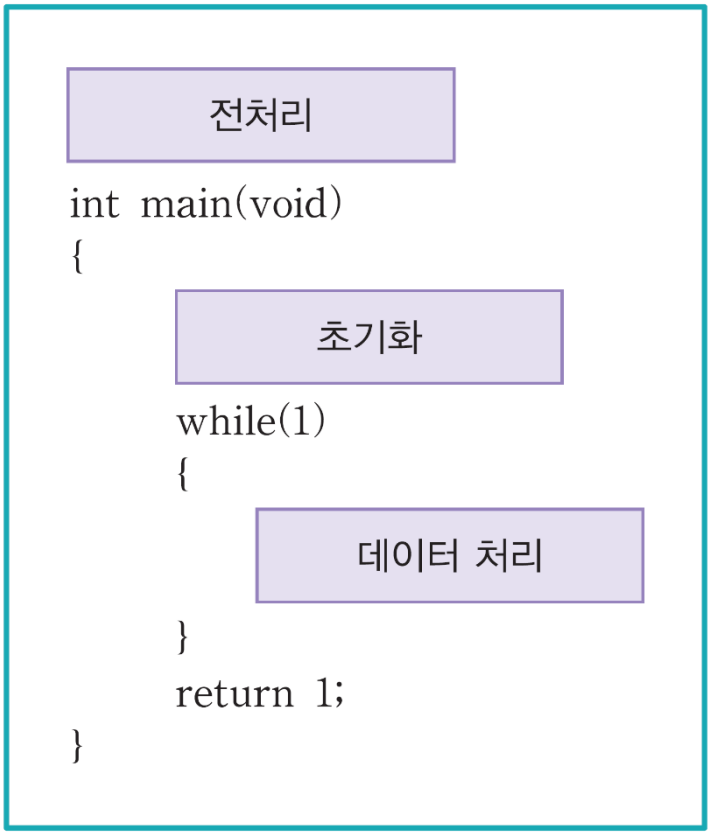
- 스케치는 C/C++을 기반으로 함
- main 함수는 존재하지 않음
  - main 함수는 숨겨져 있으므로 신경 쓰지 않아도 됨
- 2개의 기본 함수로 구성
  - setup 함수

    - 초기화 함수
    - 스케치 실행이 시작될 때 한 번만 실행

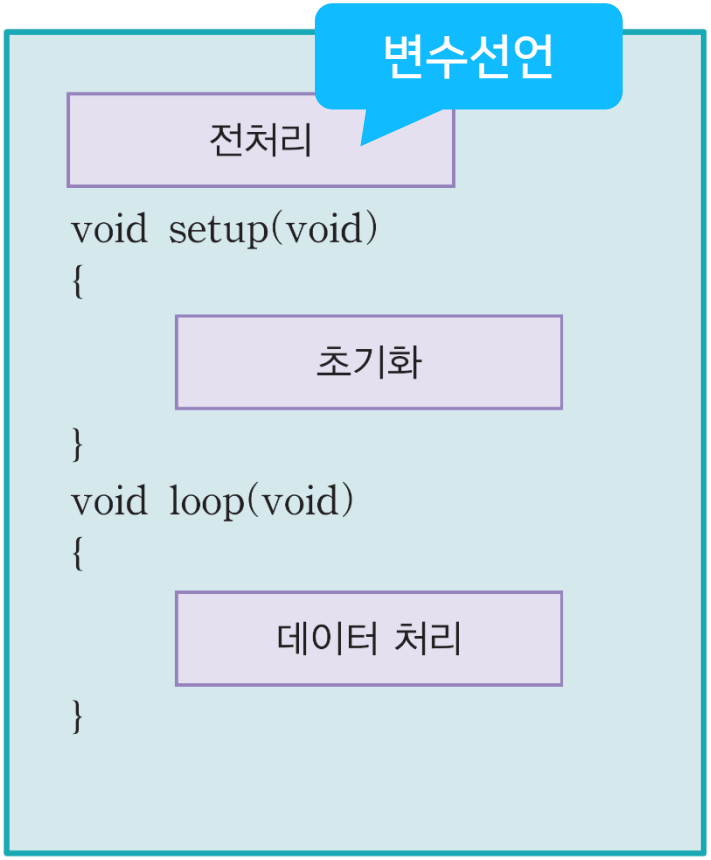
  - loop 함수

    - 반복 실행 함수
    - 아두이노를 위한 프로그램에서 메인/이벤트 루프에 해당

스케치의 구조 비교

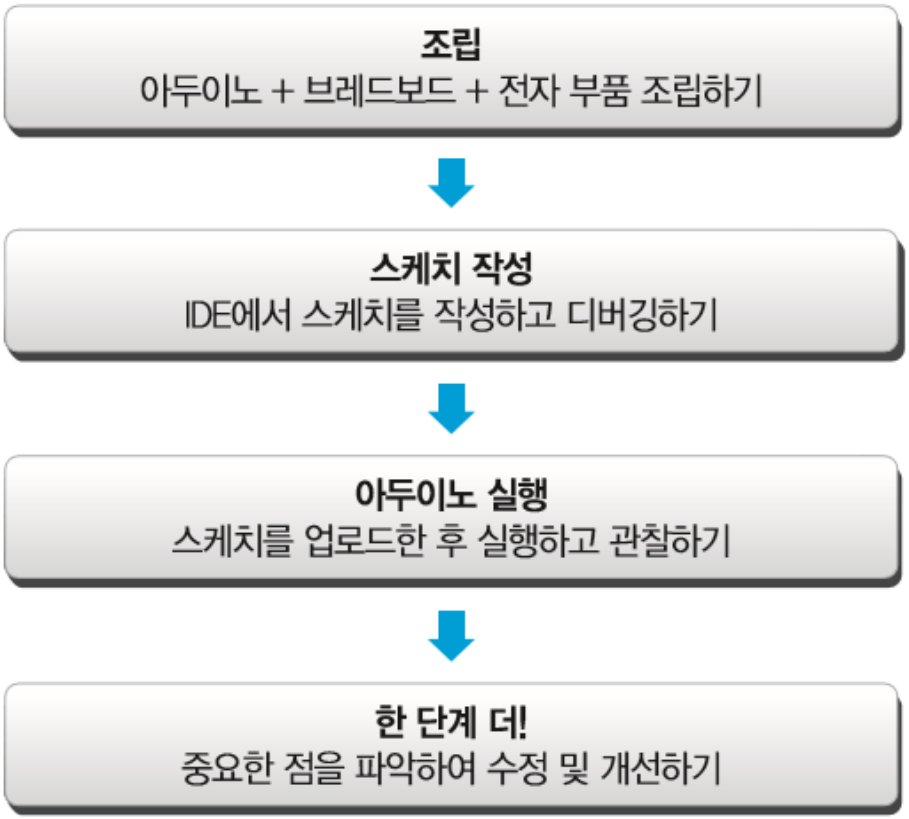


기존 C 프로그램의 구조



아두이노를 위한 스케치의 구조

아두이노 실행 단계


조립
아두이노 + 브레드보드 + 전자 부품 조립하기
스케치 작성
IDE에서 스케치를 작성하고 디버깅하기
아두이노 실행
스케치를 업로드한 후 실행하고 관찰하기
한 단계 더!
중요한 점을 파악하여 수정 및 개선하기

03 아두이노 기본 클래스

아두이노 라이브러리

- AVR 마이크로컨트롤러의 특정 기능이나, 특정 주변장치 제어를 위한 기능을 라이브러리로 구현
- C++ 기반의 객체지향 방식으로 작성
- 종류
  - 기본 라이브러리 : 아두이노에서 제공
    - 아두이노 설치 디렉터리 아래 'libraries' 디렉터리
  - 확장 라이브러리 : 써드 파티에서 제공
    - 스케치북 디렉터리 아래 'libraries' 디렉터리

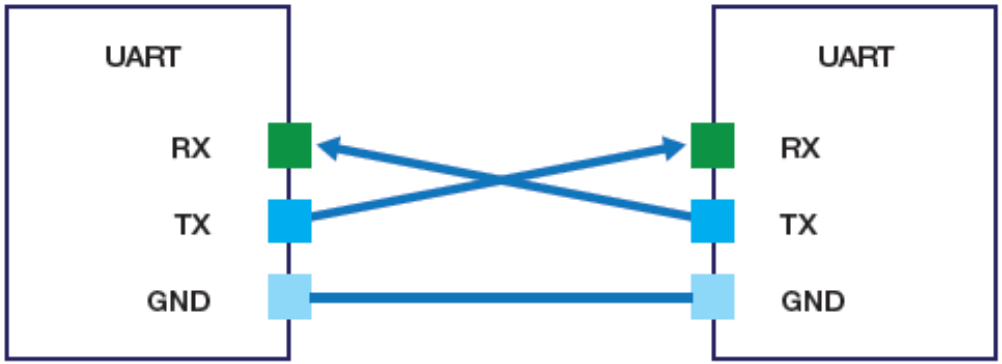
* AVR(Advanced Virtual RISC) 마이크로컨트롤러: ATMEL사에서 제작된 축소 명령어 집합 컴퓨터 (RISC, Reduced instruction set computer)구조의 MCU(Micro Controller Unit)

아두이노 라이브러리

- 라이브러리와 별도로 아두이노는 2개의 기본 클래스 제공
  - UART 시리얼 통신을 위한 Serial 클래스와 문자열 처리를 위한 String 클래스
    * UART (Universal Asynchronous serial Receiver and Transmitter)
  - 별도의 헤더 파일(*.h) 없이 사용 가능

- 시리얼 통신이란?

정보처리기기는 기본으로 시리얼 통신을 지원한다. 두 기기 간에 통신 속도를 약속하고 송수신 회선을 각각 지정하여 데이터를 주고받는다.



- UART에는 TX(transmission)와 RX(reception)이 존재한다.
- TX는 다른 기기의 RX와 연결하고 RX는 다른 기기의 TX와 연결한다.
- GND는 서로 연결한다.

RX: 수신단 / TX: 송신단/ GND: 그라운드 접지(전류)

시리얼 통신

클럭(Clock) 속도란?

- **클럭 신호**는 마이크로컨트롤러(CPU, MCU) 내부에서 **연산이나 신호 처리를 동기화**하는 기준 신호
- 흔히 **Hz(헤르츠)** 단위로 표현: 1Hz = 1초에 1번 신호 변화
- 예: 아두이노 우노(ATmega328P)
  - 기본 클럭 = 16MHz → 1초에 16,000,000번 동작 가능한 기준 신호가 있음

클럭 속도가 높으면?

- 1초 동안 더 많은 연산을 처리할 수 있음 → CPU가 빠름
- 단, 전력 소모 ↑, 발열 ↑ 가능

시리얼 통신

동기식(Synchronous) 통신/작동

- 동기식: 데이터 전송이나 동작이 클럭 신호와 정확히 맞물려서 수행됨
- 장점: 빠르고 타이밍이 정확
- 단점: 클럭 신호를 공유해야 해서 배선/설정 복잡

Ex) 마이크로컨트롤러 내부 연산 → 모든 레지스터와 연산이 클럭 펄스마다 동작

SPI, I2C 통신 → 마스터 클럭과 동기화된 데이터 전송

비동기식(Asynchronous) 통신/작동

- 비동기식: 클럭 신호 없이, 시작/종료 신호(Start/Stop)로 동작을 맞춤
- 장점: 배선 단순, 클럭 공유 필요 없음
- 단점: 동기식보다 속도가 느리고, 타이밍 오차 가능성 있음

Ex) UART/Serial 통신 → 시작 비트, 데이터, 정지 비트로 데이터를 주고받음

마이크로컨트롤러 내부에서도 인터럽트 기반 이벤트 처리는 비동기적

Serial 클래스

- UART 시리얼 통신을 위한 클래스 **UART(Universal Asynchronous Receiver/Transmitter)**
- 실제 클래스 이름은 Serial_이며, 그 객체가 Serial임
- 우노는 하나의 UART 시리얼 통신 포트만을 제공

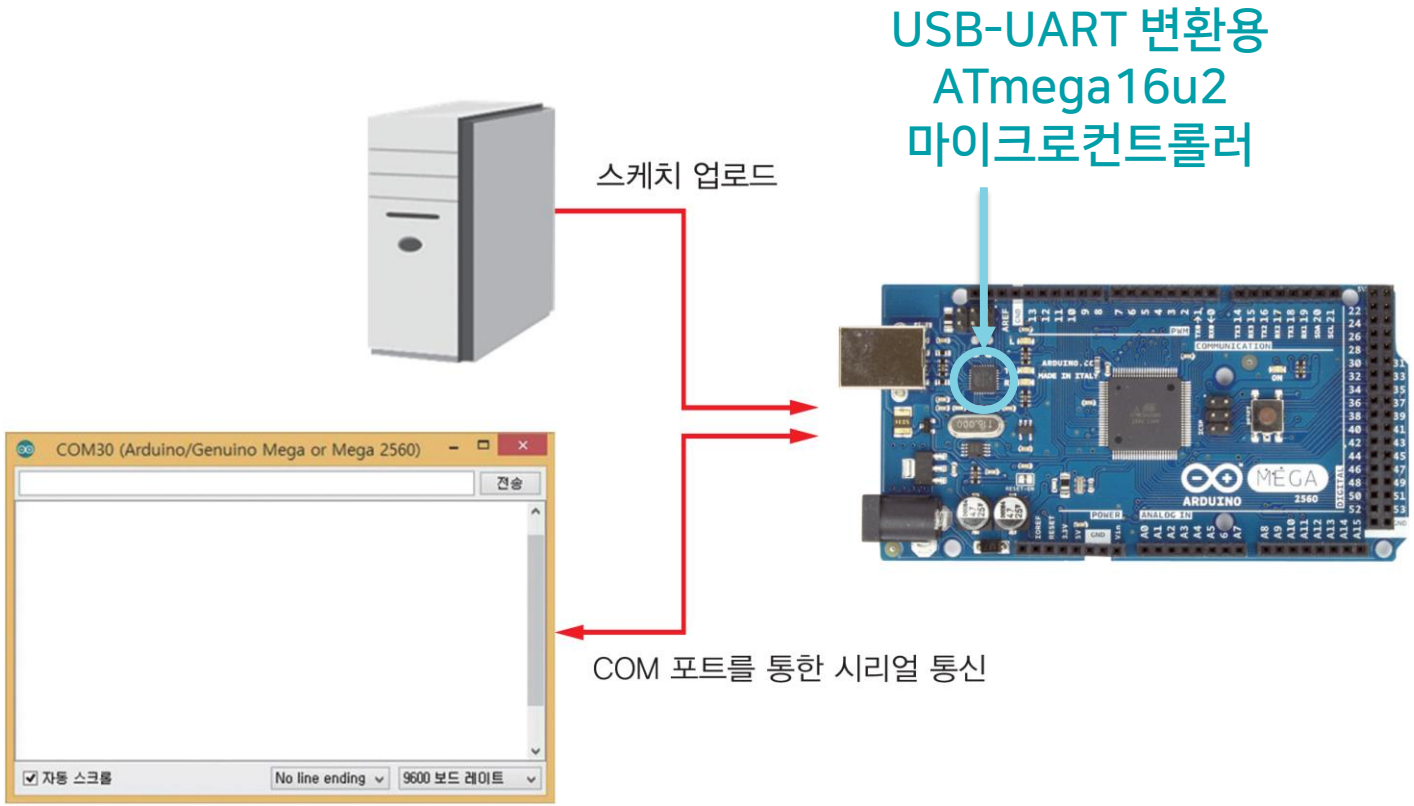
UART 시리얼 통신

- RS-232C 시리얼 케이블에서 정의된 RX, TX, GND 핀을 사용하여 통신
- UART 시리얼 통신은 시리얼 통신의 한 종류이지만, 흔히 시리얼 통신은 UART 시리얼 통신을 가리킴
- 아두이노 보드와 컴퓨터의 통신은 USB로 이루어짐
  - 아두이노 보드의 ATmega16u2 컨트롤러가 USB와 UART 사이의 변환을 담당
  - 0번 UART 채널에 해당하는 Serial 객체를 통해 USB를 통한 컴퓨터와의 시리얼 통신 수행

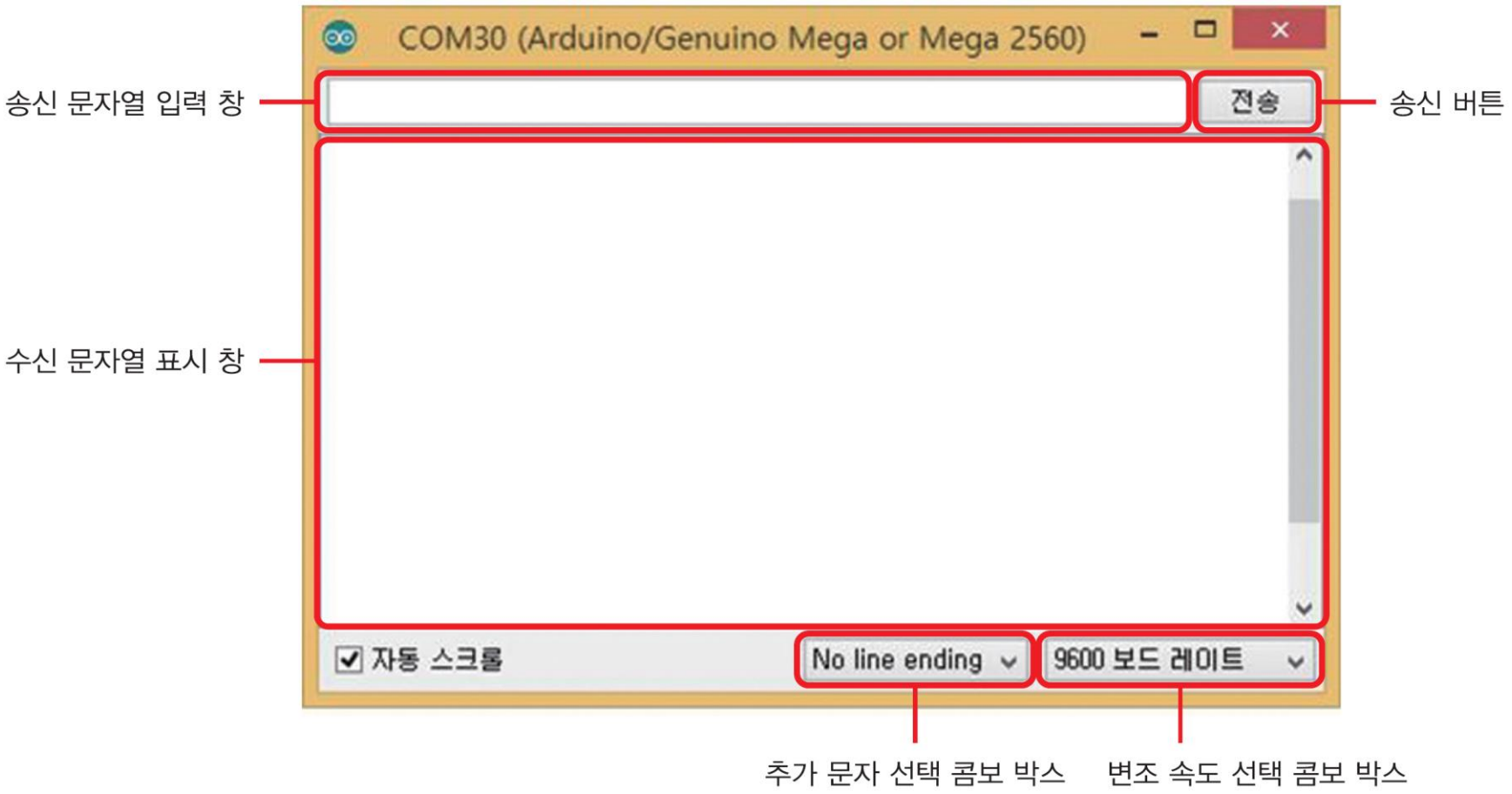
컴퓨터-아두이노 USB 연결

① 스케치 업로드를 위해 사용

② 시리얼 통신을 위해 사용



시리얼 모니터


COM30 (Arduino/Genuino Mega or Mega 2560)
송신 문자열 입력 창
전송
송신 버튼
수신 문자열 표시 창
자동 스크롤
No line ending
9600 보드 레이트
추가 문자 선택 콤보 박스
변조 속도 선택 콤보 박스

Serial 클래스 – 데이터 출력 함수

size_t print(value, format)

- **매개변수**
  - value : 출력값(char, char 배열, String, 정수, 실수 등)
  - format : 출력 형식
- **반환값** : 시리얼 포트로 출력된 바이트 수

size_t wirte(uint8_t ch)

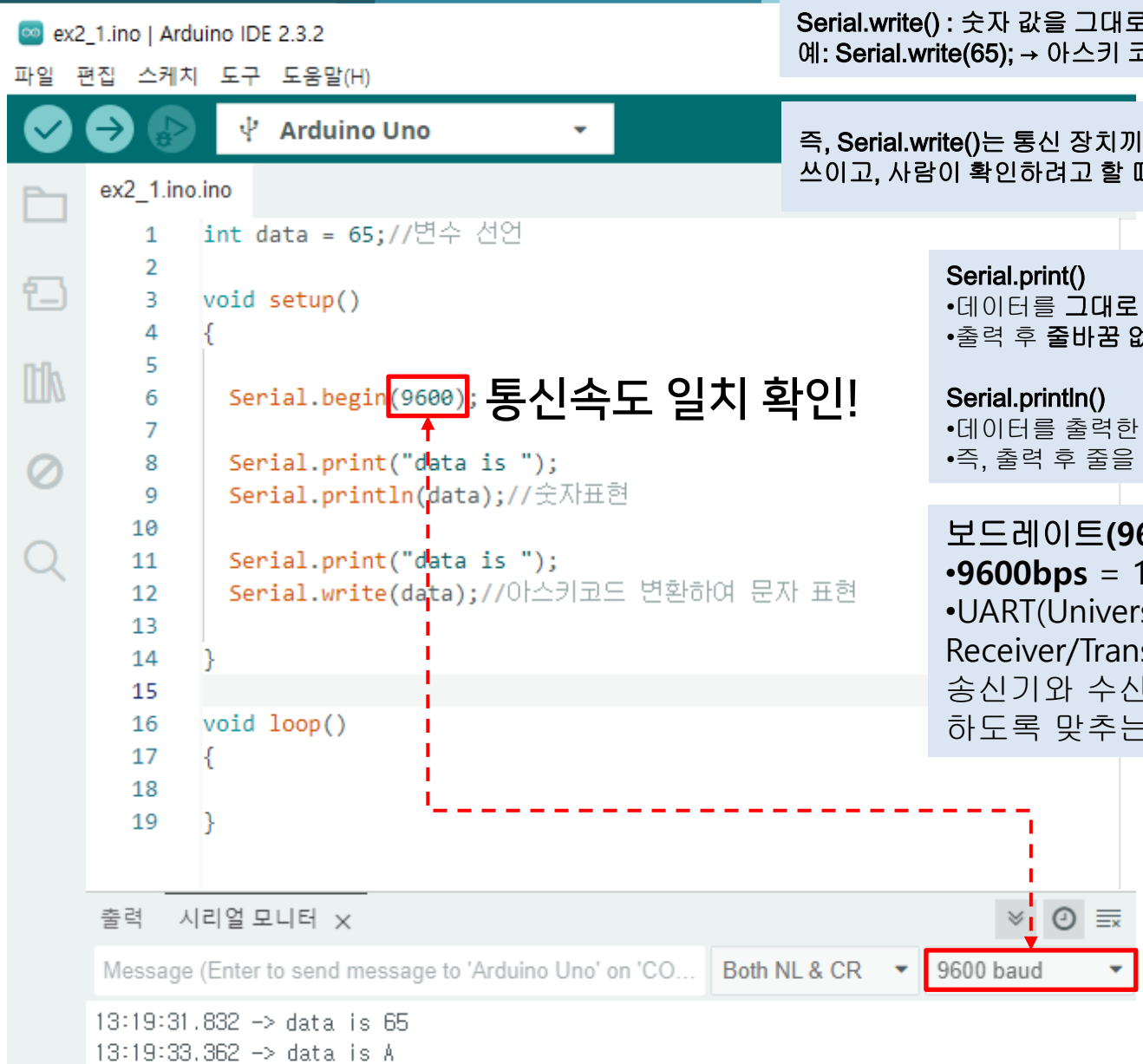
- **매개변수**
  - ch : 출력할 바이트 단위 데이터
- **반환값** : 시리얼 포트로 출력된 바이트 수

uno_ex2-1.ino: print와 write

Serial.print() : 숫자를 "사람이 읽을 수 있는 문자열"로 바꿔서 출력
예: Serial.print(65); → "65" (문자 '6'과 '5' 전송)

Serial.write() : 숫자 값을 그대로 1바이트(또는 지정한 크기)로 전송
예: Serial.write(65); → 아스키 코드 65 → 'A'

즉, Serial.write()는 통신 장치끼리 원시 데이터를 주고받을 때 주로 쓰이고, 사람이 확인하려고 할 때는 Serial.print()/println()을 더 많이 사용

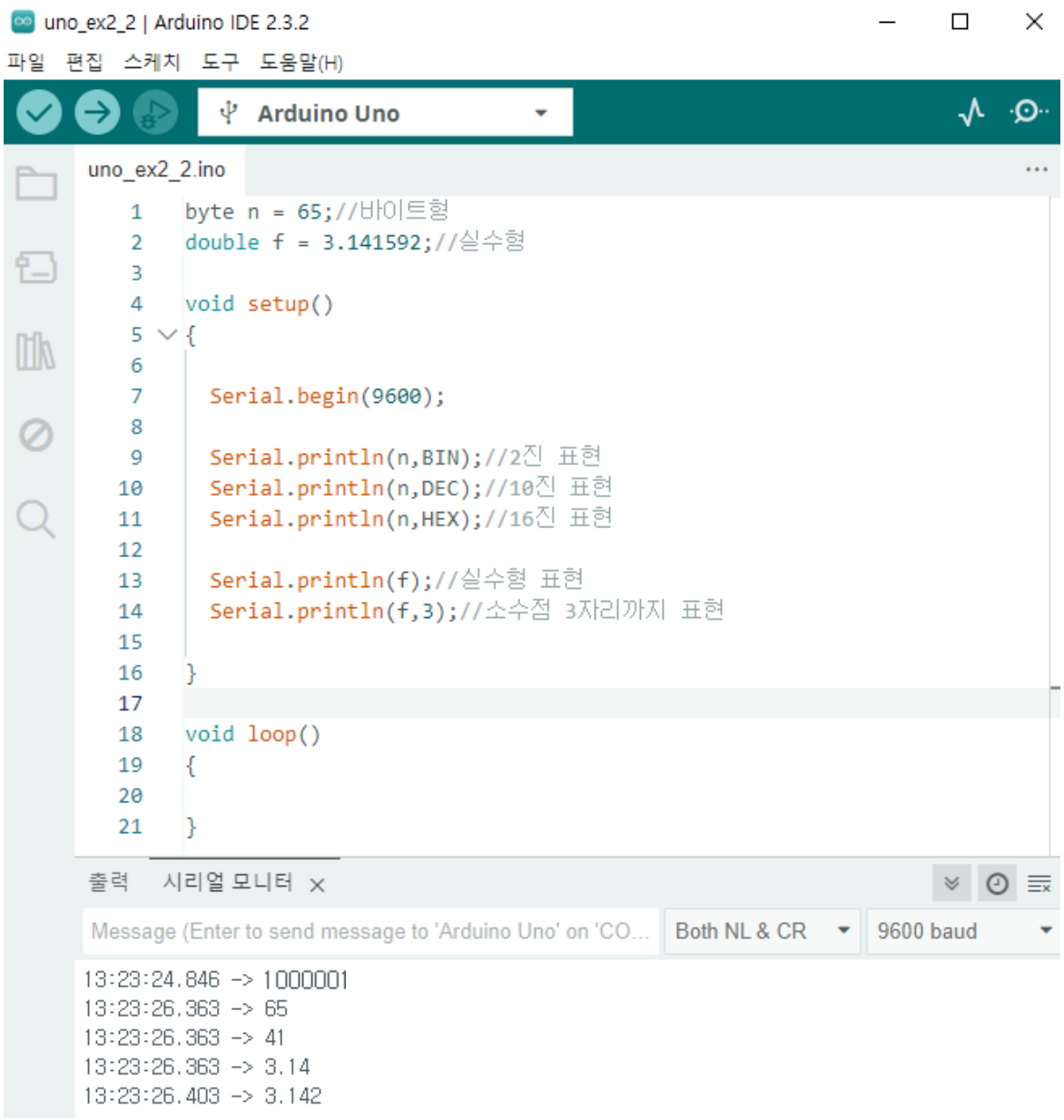


Serial.print()
- 데이터를 **그대로 출력**
- 출력 후 **줄바꿈 없음**

Serial.println()
- 데이터를 출력한 뒤 **자동으로 개행 문자(\r\n)** 추가
- 즉, 출력 후 줄을 바꿔 줌

보드레이트(9600bps)의 의미
- **9600bps** = 1초에 9600 비트(bit)를 전송
- UART(Universal Asynchronous Receiver/Transmitter) **비동기 통신 표준**에서, 송신기와 수신기가 **같은 속도로 비트를 처리**하도록 맞추는 값임.

uno_ex2-2.ino : 진법과 소수점 이하 자릿수



uno_ex2-3.ino : 에코 백

uno_ex2_3.ino

```
1   byte data = 0;
2
3   void setup()
4   {
5     Serial.begin(9600);
6   }
7
8   void loop()
9   {
10    if(Serial.available()>0){//수신된 존재 여부 확인
11
12      data = Serial.read();//실제 데이터 읽기
13
14      Serial.print("Echo back:");
15      Serial.println(data);
16    }
17  }
```

Serial.available() → 수신 버퍼에 남아 있는 바이트 수를 반환
> 0이면 읽을 데이터가 있다는 뜻

Serial.print("Echo back:");
Serial.write(data);
Serial.print(" ");
Serial.println(data);

출력 시리얼 모니터 ×

단어("apple") 입력후 엔터

Message (Enter to send message to 'Arduino

```
20:05:59.057 -> Echo back:a 97
20:05:59.057 -> Echo back:p 112
20:05:59.090 -> Echo back:p 112
20:05:59.090 -> Echo back:l 108
20:05:59.129 -> Echo back:e 101
20:05:59.129 -> Echo back:
20:05:59.129 -> 10
```

시리얼 통신으로 받은 데이터를 다시 보내주는(에코) 예제 : PC 시리얼 모니터에서 데이터 보내면 → 아두이노가 그대로 다시 출력(에코).

제어 문자		공백 문자		구두점		숫자		알파벳			
10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자
0	0x00	NUL	32	0x20	SP	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	₩	124	0x7C	|
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

String 클래스

- 문자열 처리를 위한 클래스
- C++에서의 문자열 처리를 위한 클래스와 유사
- 문자, 문자열, 숫자 등으로부터 String 객체 생성 가능
- String 객체와 다른 타입의 데이터 결합으로 새로운 String 객체 생성

```
String( " abc " ) + 123;          // String 객체 + 정수
```

- 큰따옴표로 표시되는 문자열은 String 클래스 객체가 아닌 문자 배열로 처리

```
" abc "  + 123;                   // 문자열 배열 + 정수 (×)
```

uno_ex2-4.ino : 문자열 생성 방법

uno_ex2_4.ino

```
1   String str1="Echo:";
2   String str2="";//입력받은 문자 조합
3   byte data = 0;//키보드 입력
4
5   void setup()
6   {
7     Serial.begin(9600);
8   }
9
10  void loop()
11  {
12    if(Serial.available()>0){//수신된 존재 여부 확인
13
14      data = Serial.read();//실제 데이터 읽기
15
16      if(data==10){//엔터 입력시
17
18        //실제 출력
19        Serial.println(str1+" "+str2);
20        str2="";//문자열 초기화
21
22      }else{//엔터 입력 아닌경우 모두 문자
23        str2+=(char)data;//기존 문자열 조합에 추가
24      }//if-else end
25    }//if end
26  }//loop end
```

출력 시리얼 모니터 ×

Message (Enter to send message to 'Arduino Uno' on 'COM4')

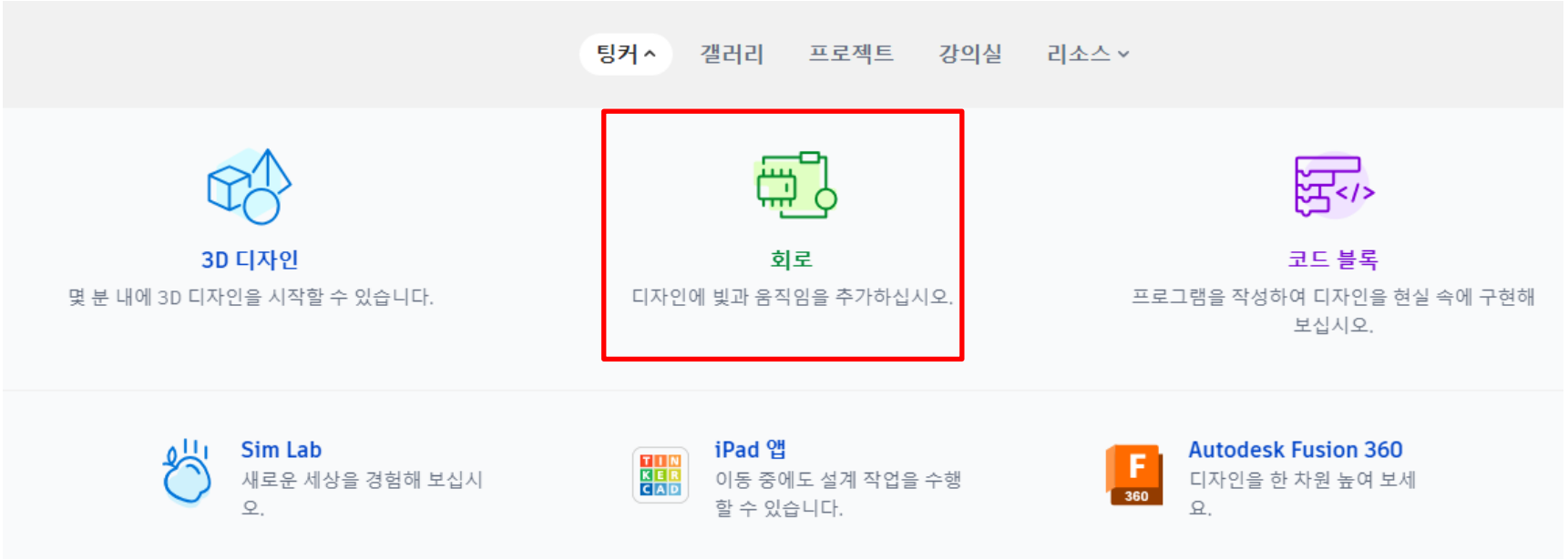
13:36:43.873 -> Echo: apple

* 앞으로 입력 데이터는 □ 로 표시

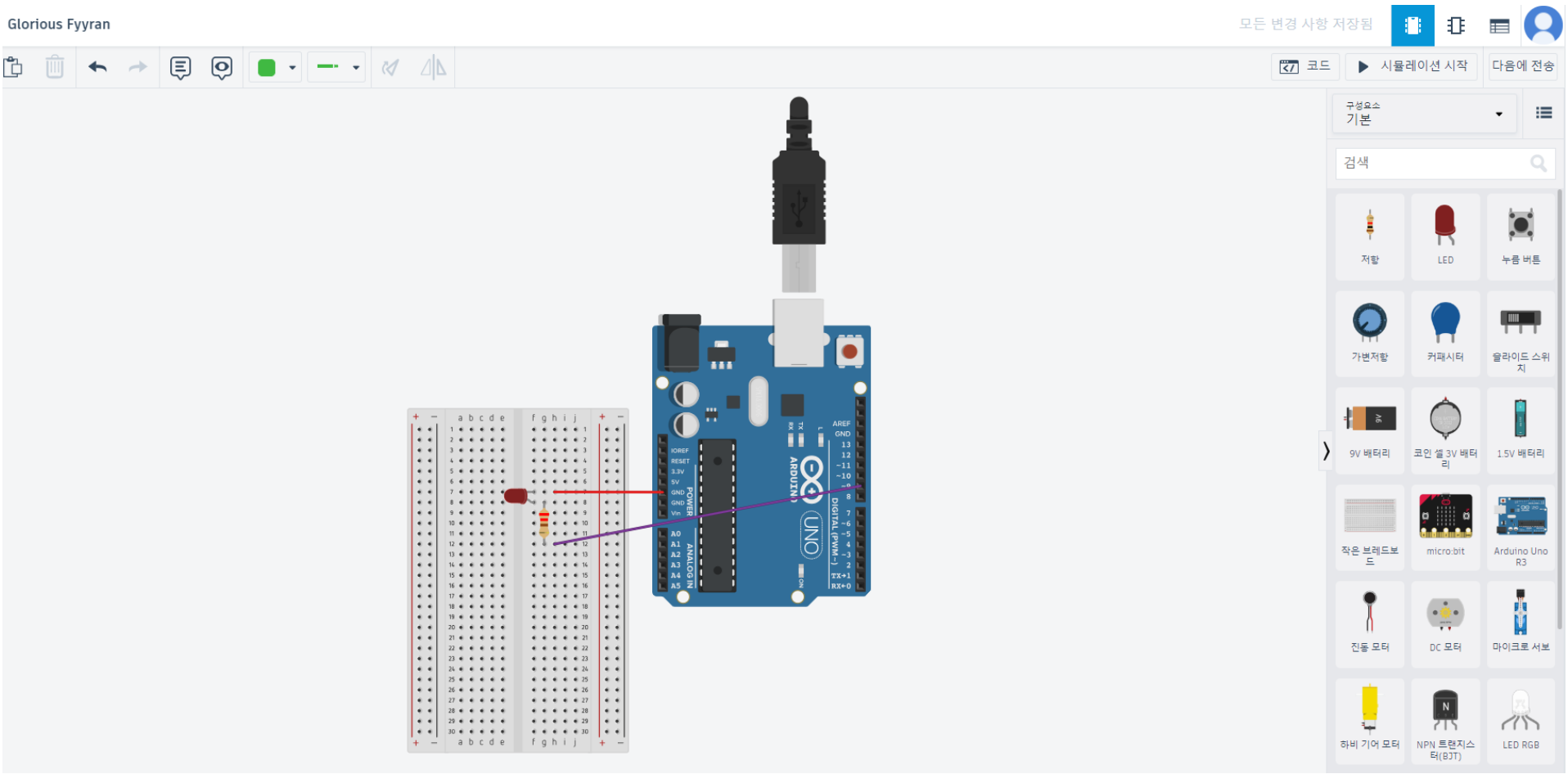
04 아두이노 온라인 시뮬레이션

팅커캐드

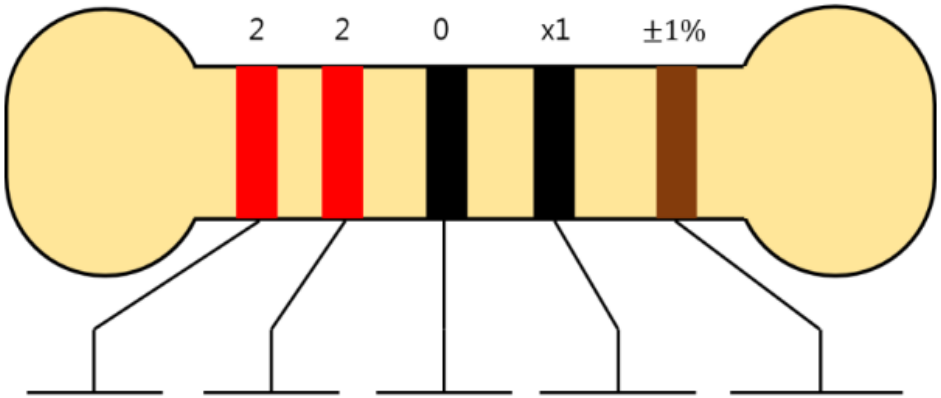
- https://www.tinkercad.com 회원가입



팅커캐드


Glorious Fyyran
모든 변경 사항 저장됨
코드
시뮬레이션 시작
다음에 전송
구성요소
기본
검색
저항
LED
누름 버튼
가변저항
커패시터
슬라이드 스위치
9V 배터리
코인 셀 3V 배터리
1.5V 배터리
작은 브레드보드
micro:bit
Arduino Uno R3
진동 모터
DC 모터
마이크로 서보
하비 기어 모터
NPN 트랜지스터(BJT)
LED RGB

4. 저항 읽는 법



첫 번째	두 번째	세 번째	단위	오차
0	0	0	x0.01(은)	±10% (은)
1	1	1	x0.1(금)	±5%(금)
2	2	2	x1	
3	3	3	x10	±1%
4	4	4	x100	±2%
5	5	5	x1K	
6	6	6	x10K	
7	7	7	x100K	±0.5%
8	8	8	x1M	±0.25%
9	9	9	x10M	±0.1%

< 220 Ω 저항 >

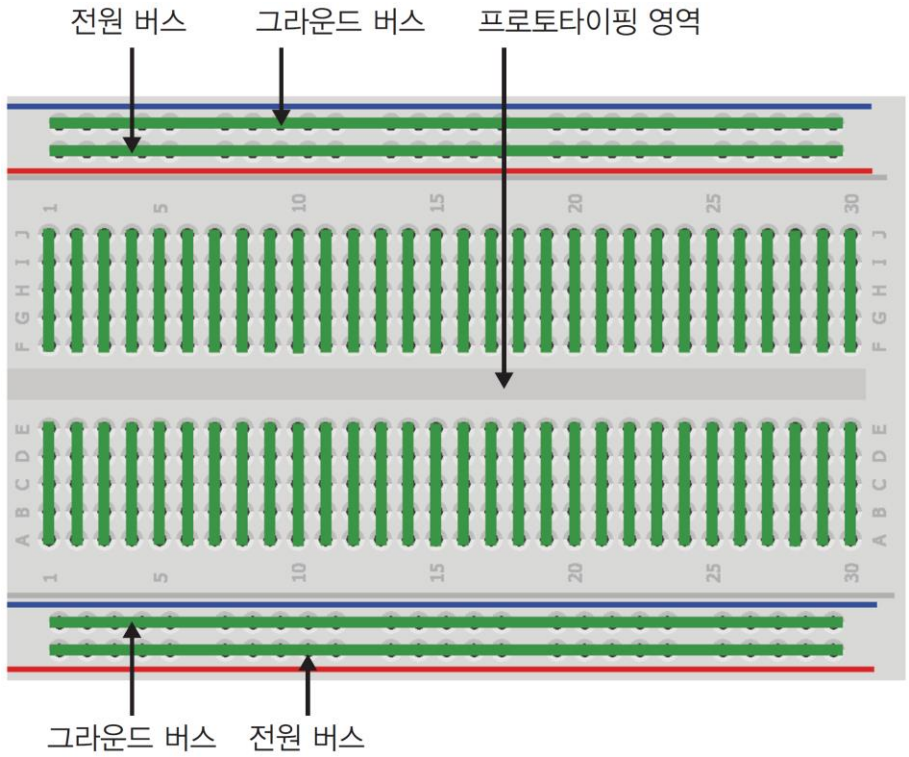
요약

- 아두이노 보드 구성 : 전원, 입출력핀
- 아두이노 보드 종류 : 아누이노 우노, 메가 등 (이외 라즈베리파이)
- 아두이노 IDE환경 설치 및 설정
  - 보드 선택 / 포트 반드시 확인
- 아두이노 프로그램 실행단계
- 기본 클래스
  - Serial
    - 통신 초기화: begin(속도)
    - 출력함수: print(), println(), write()
    - 입력함수: available(), read()
  - String

LED 모듈 연결 실습

필요한 도구 : 브레드 보드

- 부품을 납땜하지 않고도 회로 구성을 가능하게 하는 프로토타이핑 도구
- 양쪽에 파란색(그라운드)과 빨간색(전원 버스) 홀은 연결된 상태
- 수직 홀은 5개 그룹으로 연결된 상태
- 각 핀이 내부적으로 연결되어 있어 각 전기/전자 부품을 연결하기 편리함

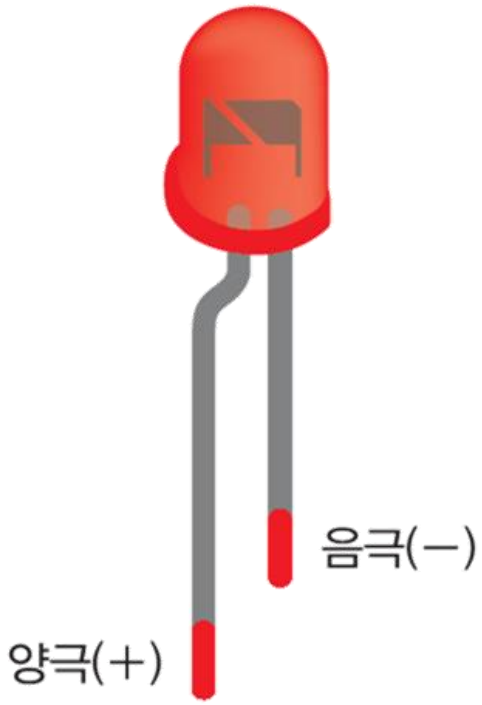


필요한 도구 : LED(Light Emitting Diode, 발광 다이오드)

- 양극(+, 애노드)에서 음극(-, 캐소드)방향으로만 전류가 흐름
  - 따라서 +, - 방향을 고려하여 연결해야 한다.

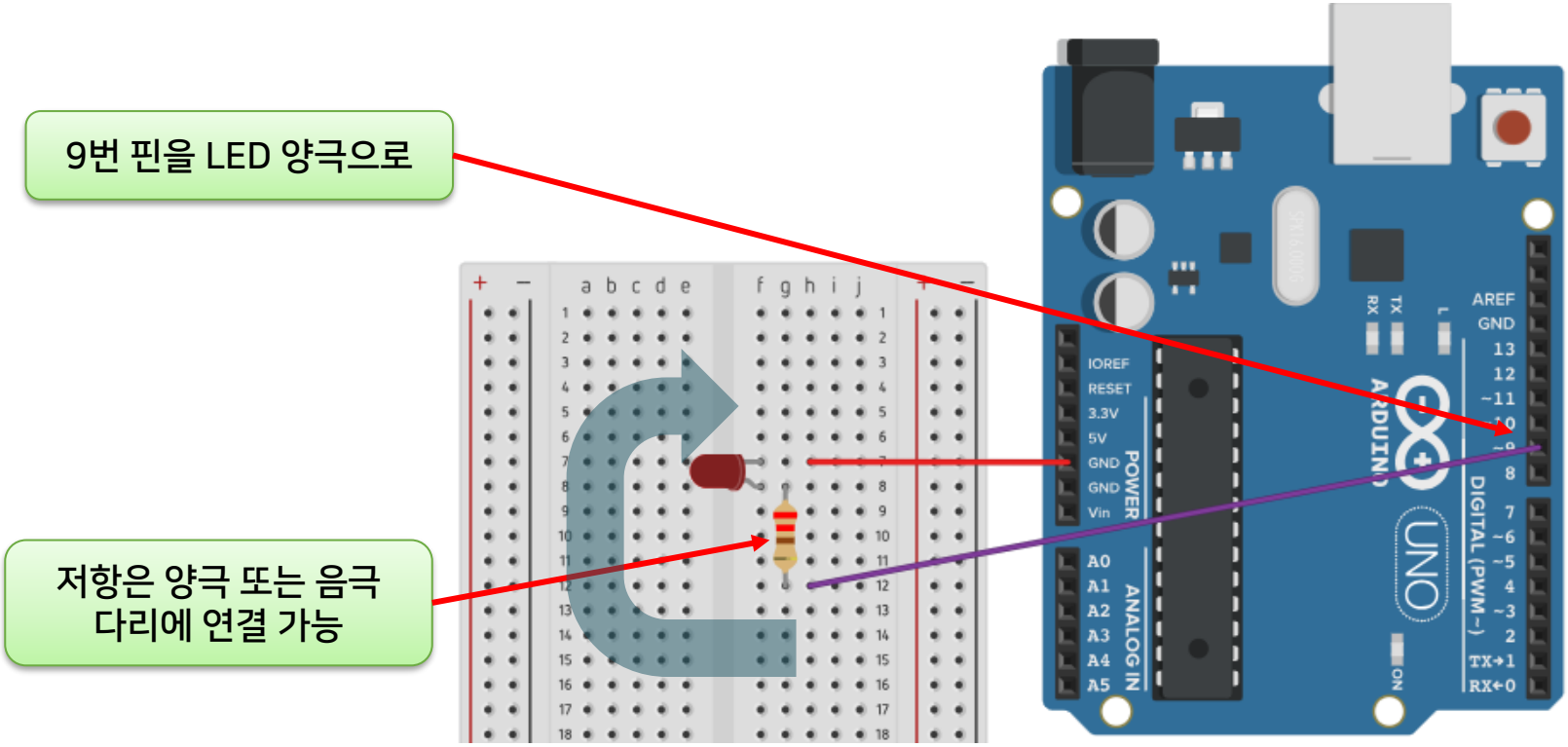
- 양극과 음극의 2개 다리를 가짐
  - 다리 길이가 긴 쪽이 양극

- 과도한 전류가 흐르지 않도록 전류 제한 필요
  - 200Ω 전후 저항이 일반적으로 사용됨



아두이노에 LED 연결하기

- 아래 그림과 같이 LED를 아두이노에 연결한다.
  - 일반적인 LED는 2V의 전압과 2mA의 전류를 소모하기 때문에 5V의 아두이노 전압을 낮추기 위해 저항을 사용해야함
  - 220Ω 저항을 LED에 직렬로 연결
- 저항을 사용하지 않고 LED를 직접 연결하면 LED가 타버리거나 손상될 수 있음



LED 예제 함수

void pinMode(uint8_t pin, uint8_t mode)

- **매개변수**
  - pin : 설정하고자 하는 핀 번호
  - mode : INPUT, OUTPUT, INPUT_PULLUP 중 하나
- **반환값** : 없음

void digitalWrite(uint8_t pin, uint8_t value)

- **매개변수**
  - pin : 핀 번호
  - value : HIGH(1) 또는 LOW(0)
- **반환값** : 없음

=>디지털 출력: 1또는 0으로만 표현

void delay(unsigned long ms)

- **매개변수**
  - ms : 밀리초 단위의 지연 시간
- **반환값** : 없음

uno_ex3-1: LED 모듈 예제

- 아두이노의 9번 핀에 연결된 LED를 켜기 위해, 9번 핀을 출력 모드로 설정

```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3   void setup()
4   {
5       pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
6       digitalWrite(LED, HIGH); //LED를 HIGH로 지정
7   }
8
9   void loop()
10  {
11      //이 실습에서는 loop문을 비움
12  }
```

➔ 9번 핀을 5V로 설정 → LED 켜짐

디지털 9번 핀 (5V) → 저항 (220Ω) → LED (+극 → -극) → GND → 아두이노 내부 회로 → 다시 전원 공급 회로로 반환

```
const int LED = 9;

void setup() {
  // put your setup code here, to run once:
  pinMode(LED, OUTPUT);
  digitalWrite(LED, LOW);
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

```
void setup() {
  // put your setup code here, to run once:
  pinMode(9, OUTPUT);
  digitalWrite(9, HIGH);
  delay(1000);
  digitalWrite(9, LOW);
  delay(1000);
}
void loop() {
  // put your main code here, to run repeatedly:
}
```

목차

0. 모듈 종류

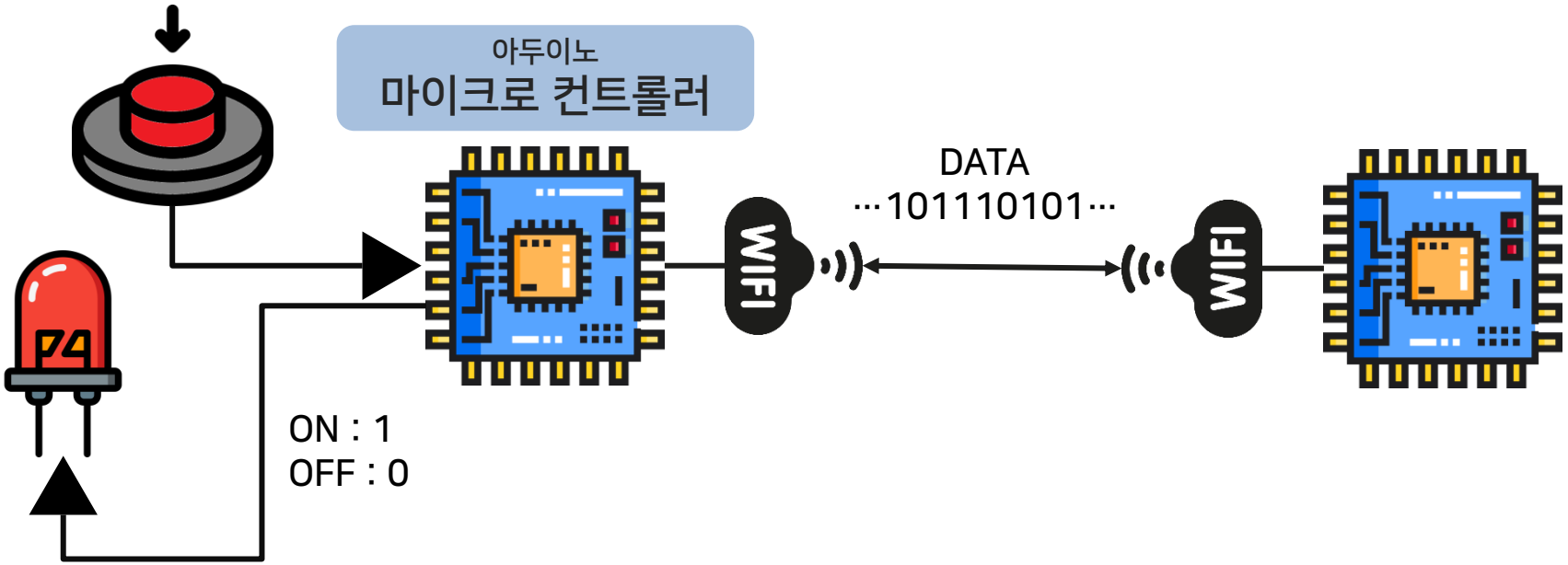
1. LED 모듈 연결

2. LED 다중 연결

3. RGB 모듈 연결

모듈 종류

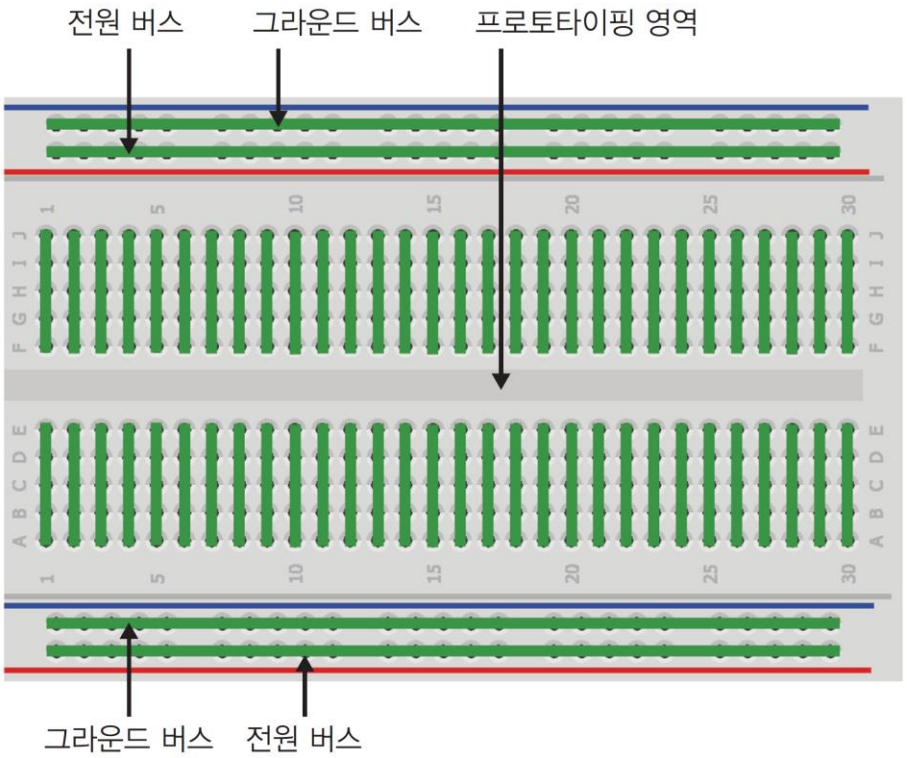
- 출력 모듈: 컴퓨터가 생성한 정보를 컴퓨터의 외부로 전송하는 데에 사용되는 장치
  - LED, 스피커 등
- 입력 모듈: 신호 등을 발생시켜 컴퓨터로 전송하는데 사용되는 설비나 장치
  - 스위치, 온도, 기울기 센서 등
- 통신모듈: 외부 컴퓨터와 데이터를 송수신하는데 사용되는 설비나 장치
  - Ethernet, wifi, 블루투스 등



01 LED 모듈 연결

필요한 도구 : 브레드 보드

- 부품을 납땜하지 않고도 회로 구성을 가능하게 하는 프로토타이핑 도구
- 양쪽에 파란색(그라운드)과 빨간색(전원 버스) 홀은 연결된 상태
- 수직 홀은 5개 그룹으로 연결된 상태
- 각 핀이 내부적으로 연결되어 있어 각 전기/전자 부품을 연결하기 편리함

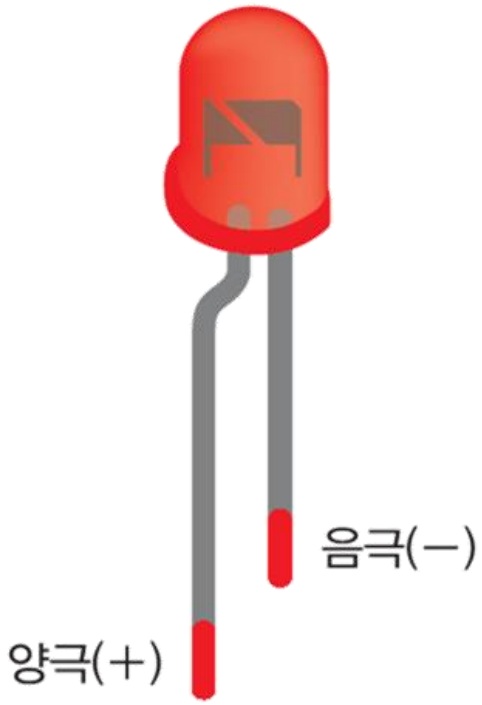


필요한 도구 : LED(Light Emitting Diode, 발광 다이오드)

- 양극(+, 애노드)에서 음극(-, 캐소드)방향으로만 전류가 흐름
  - 따라서 +, - 방향을 고려하여 연결해야 한다.

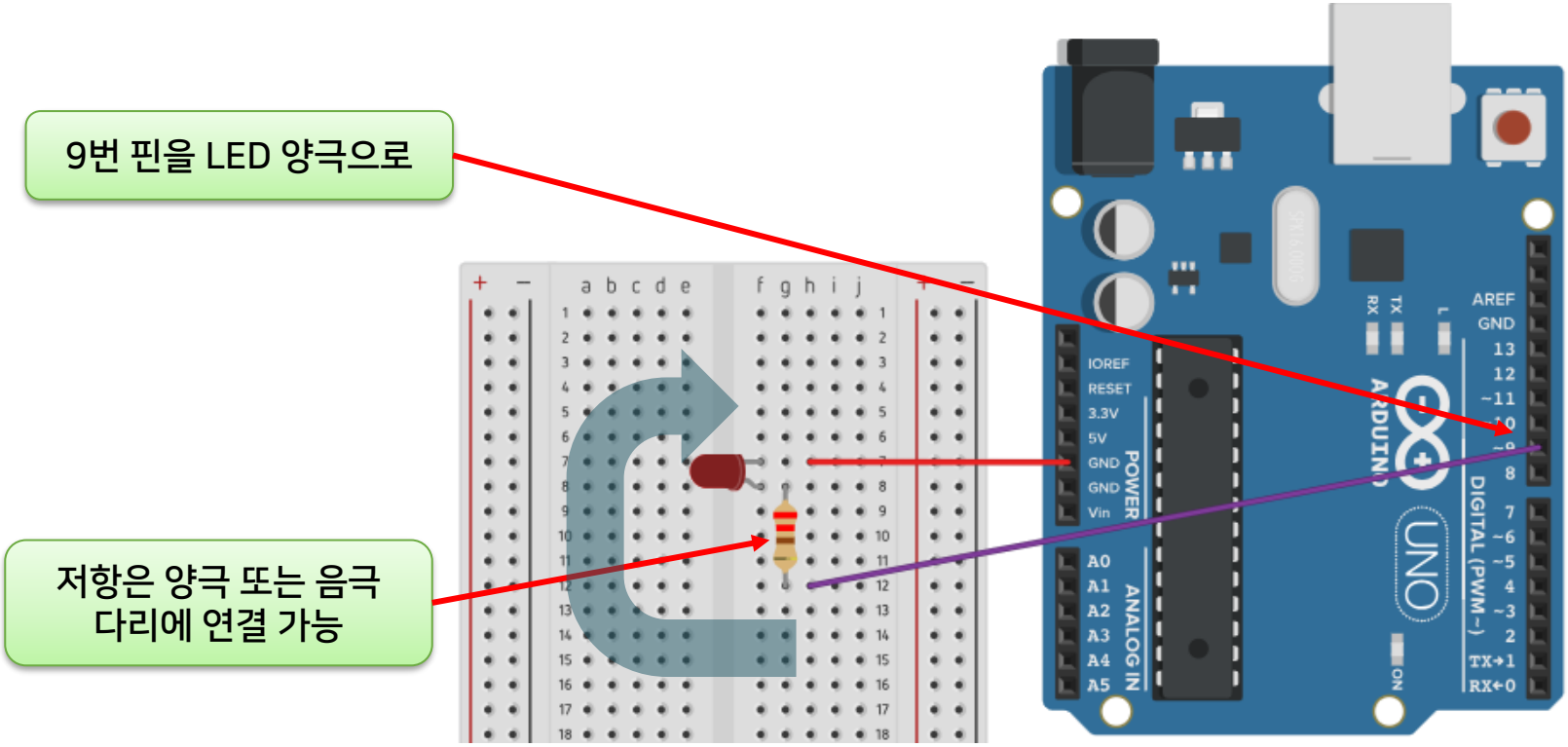
- 양극과 음극의 2개 다리를 가짐
  - 다리 길이가 긴 쪽이 양극

- 과도한 전류가 흐르지 않도록 전류 제한 필요
  - 200Ω 전후 저항이 일반적으로 사용됨



아두이노에 LED 연결하기

- 아래 그림과 같이 LED를 아두이노에 연결한다.
  - 일반적인 LED는 2V의 전압과 2mA의 전류를 소모하기 때문에 5V의 아두이노 전압을 낮추기 위해 저항을 사용해야함
  - 220Ω 저항을 LED에 직렬로 연결
- 저항을 사용하지 않고 LED를 직접 연결하면 LED가 타버리거나 손상될 수 있음



LED 예제 함수

void pinMode(uint8_t pin, uint8_t mode)

- **매개변수**
  - pin : 설정하고자 하는 핀 번호
  - mode : INPUT, OUTPUT, INPUT_PULLUP 중 하나
- **반환값** : 없음

void digitalWrite(uint8_t pin, uint8_t value)

- **매개변수**
  - pin : 핀 번호
  - value : HIGH(1) 또는 LOW(0)
- **반환값** : 없음

=>디지털 출력: 1또는 0으로만 표현

void delay(unsigned long ms)

- **매개변수**
  - ms : 밀리초 단위의 지연 시간
- **반환값** : 없음

uno_ex3-1: LED 모듈 예제

- 아두이노의 9번 핀에 연결된 LED를 켜기 위해, 9번 핀을 출력 모드로 설정

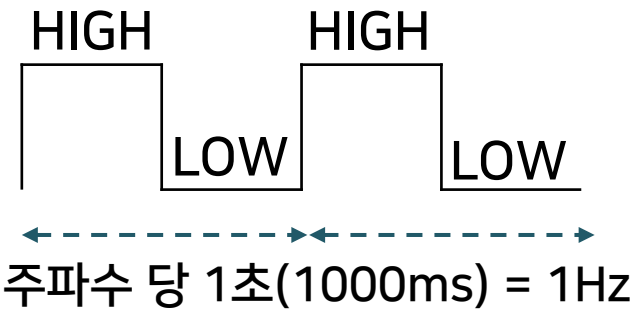
```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3   void setup()
4   {
5       pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
6       digitalWrite(LED, HIGH); //LED를 HIGH로 지정
7   }
8
9   void loop()
10  {
11      //이 실습에서는 loop문을 비움
12  }
```

➔ 9번 핀을 5V로 설정 → LED 켜짐

디지털 9번 핀 (5V) → 저항 (220Ω) → LED (+극 → -극) → GND → 아두이노 내부 회로 → 다시 전원 공급 회로로 반환

uno_ex3-2: LED 점멸

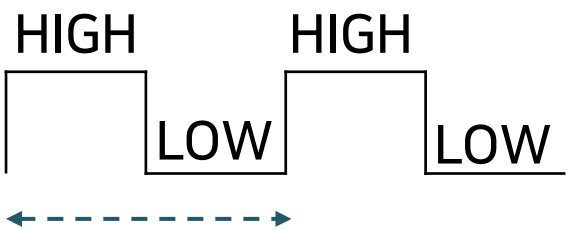
```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3   void setup()
4   {
5       pinMode(13, OUTPUT); //LED(13번 핀)를 출력으로 지정
6       digitalWrite(13, LOW);//13핀 OFF
7
8       pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
9   }
10
11  void loop()
12  {
13      digitalWrite(LED, HIGH);
14      delay(500);
15      digitalWrite(LED, LOW);
16      delay(500);
17
18  }
```



1주파수 당 1초(1000ms) = 1Hz

loop함수 변경

```
digitalWrite(LED, HIGH);
delay(5);
digitalWrite(LED, LOW);
delay(5);
```



1000/10= 100Hz

파형이 초당 50개 이상이 되면, 즉 50Hz이상의 주파수로
LED 점멸을 반복하면 인간의 눈으로는 깜빡거림을 감지할 수 없음

uno_ex3-3: 디지털 출력으로 밝기 조절

[어둡게 출력]

```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3   void setup()
4   {
5       pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
6   }
7
8   void loop()
9   {
10      digitalWrite(LED, HIGH); //LED를 HIGH로 지정
11      delay(1);
12      digitalWrite(LED, LOW); //LED를 LOW로 지정
13      delay(9);
14  }
```

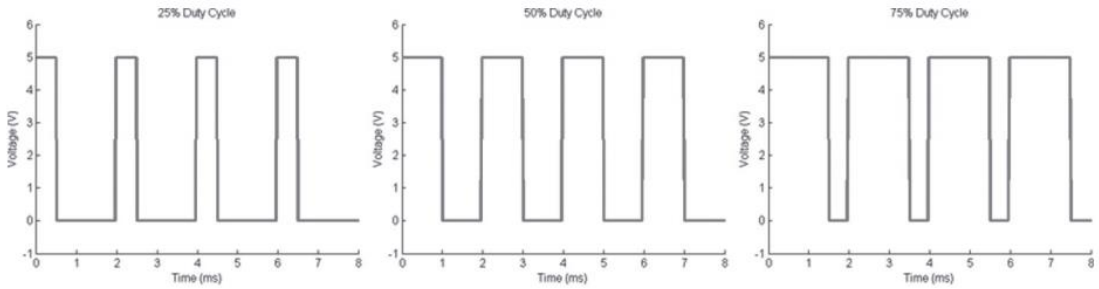
[밝게 출력]

```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3   void setup()
4   {
5       pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
6   }
7
8   void loop()
9   {
10      digitalWrite(LED, HIGH); //LED를 HIGH로 지정
11      delay(9);
12      digitalWrite(LED, LOW); //LED를 LOW로 지정
13      delay(1);
14  }
```

아날로그 출력

- 아날로그 출력 : ~~0~5V(3.3V 보드는 0~3.3V) 사이의 전압~~ 0~255의 값을 자유롭게 내보내는 기능

- PWM(Pulse Width Modulation, 펄스폭변조)
  - 순수 아날로그 제어를 모방한 것으로 다양한 환경에서 사용
  - PWM을 사용하면 디지털 신호를 아날로그 신호와 비슷하게 생성
  - PWM은 신호를 켜거나 끄는 방식으로 네모파의 듀티비(Duty Cycle)를 변조하여 사용
    - 듀티비 : 전체 주기에서 HIGH상태가 차지하는 비율
  - 아두이노 Uno는 3, 5, 6, 9, 10, 11 번 핀에서 PWM 출력이 가능(반드시 ~표기 확인)



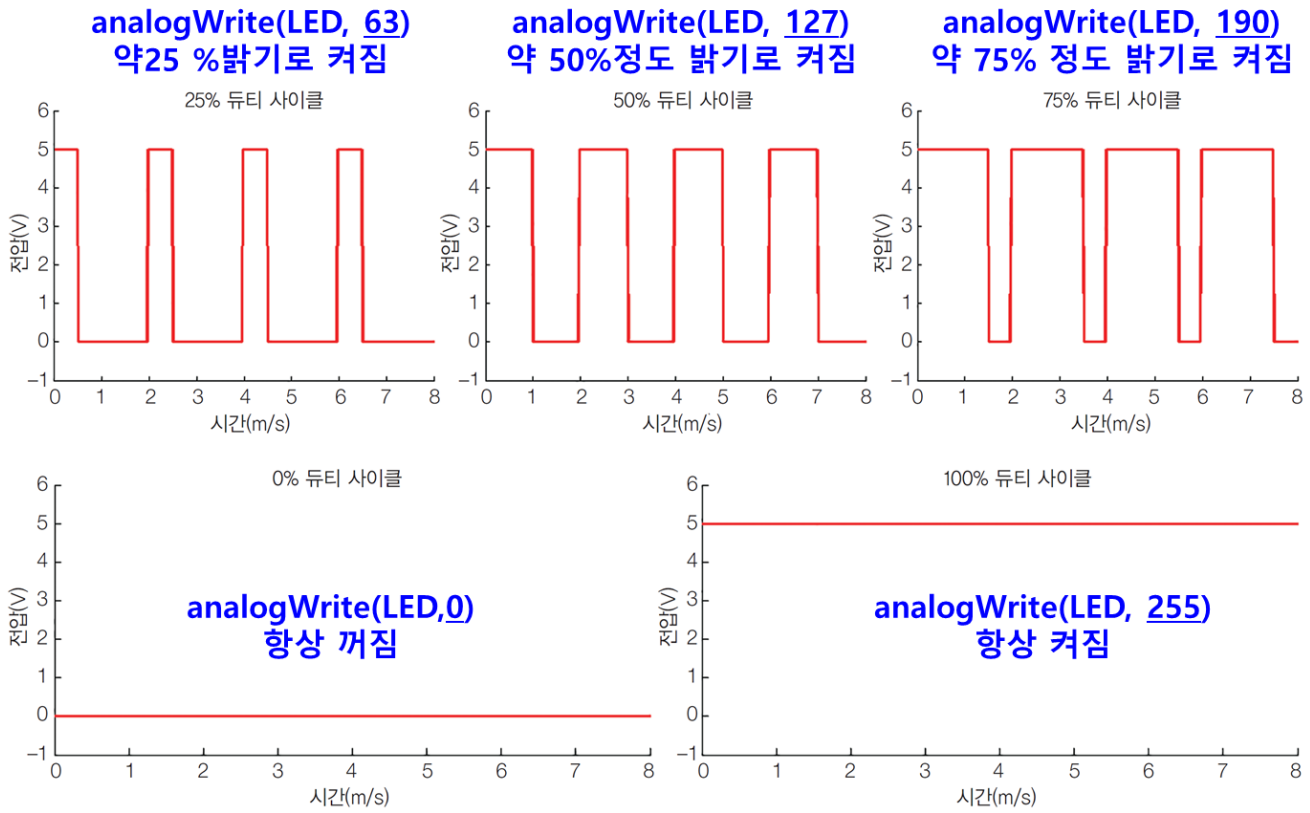
아날로그 출력

문제: LED를 통과하는 전류의 전압은 낮추지 않은 채 LED의 밝기를 어둡게 하려면 어떻게 해야 할까?

- 해결 방법
  - PWM으로 LED를 빠른 속도로 켜고 끌 수 있다면(듀티비 50%) LED는 중간 정도의 밝기로 켜져 있는 것처럼 보일 것이다. 우리 눈이 지각할 수 있는 것보다 LED가 더 빠르게 깜빡이기 때문이다.

analogWrite 함수

- analogWrite() 함수는 두 번째 인자 값에 따라 네모파의 듀티비를 지정
  - 0을 지정하면 네모파의 듀티비는 0%(항상 LOW)
  - 255를 지정하면 네모파의 듀티비는 100%(항상 HIGH)
  - 127을 지정하면 네모파의 듀티비는 50%(반은 HIGH, 반은 LOW)



uno_ex3-4: 아날로그 출력으로 밝기 조절

```
1  const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2
3  void setup()
4  {
5      pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
6  }
7
8  void loop()
9  {
10     for(int i=0; i<256; i++)
11     {
12         analogWrite(LED, i);
13         delay(10);
14     }
15     for(int i=255; i>=0; i--)
16     {
17         analogWrite(LED, i);
18         delay(10);
19     }
20 }
```

uno_ex3-5: 사용자 입력으로 아날로그 출력

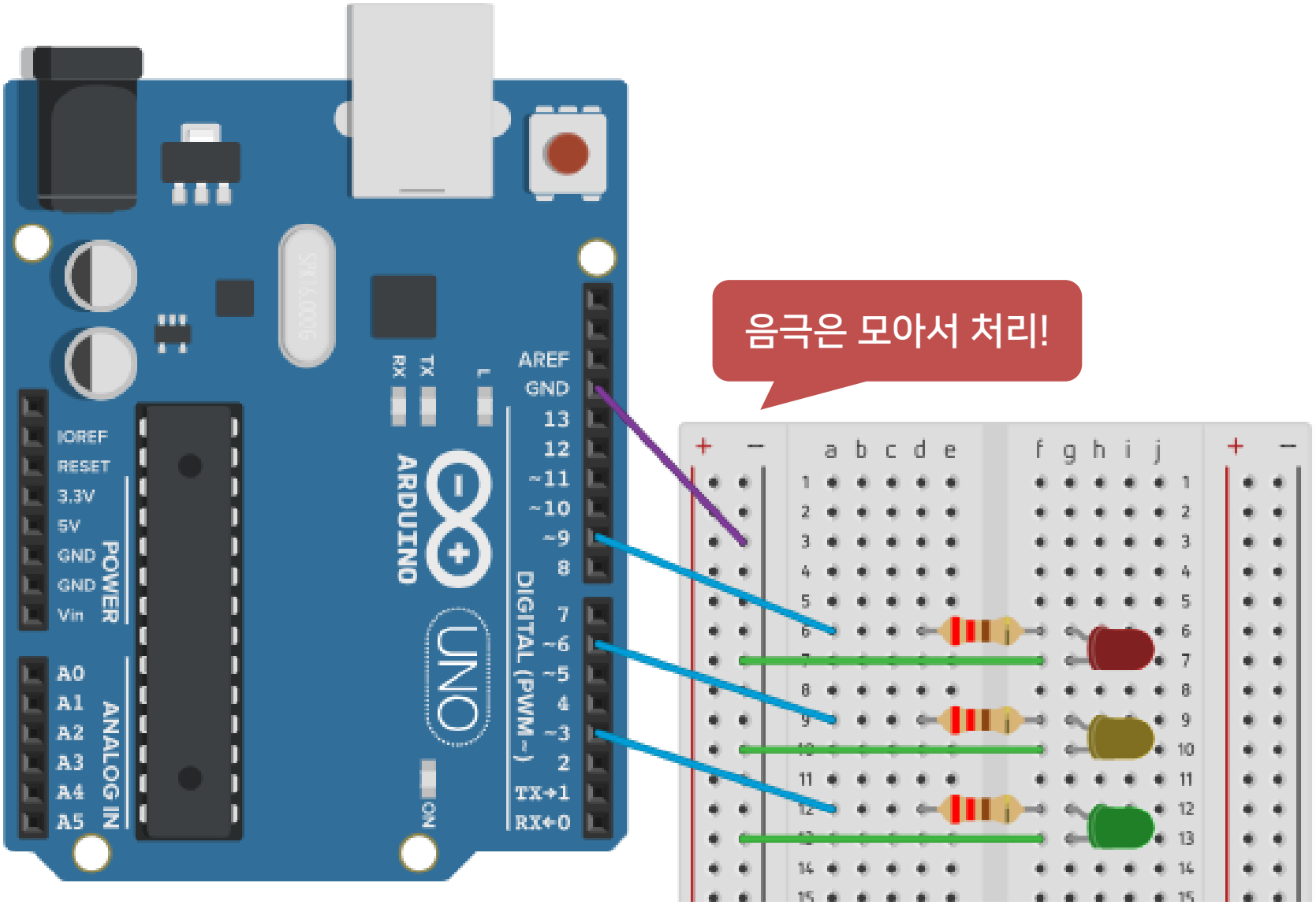
시리얼 모니터에서 숫자를 입력하면 LED 밝기를 PWM으로 조절하는 프로그램

```
1   const int LED=9;
2
3   byte data = 0;//시리얼 입력값
4   String input="";//최종 문자열
5   int adapt = -1;//최종 문자열 형변환
6
7   void setup()
8   {
9       Serial.begin(9600);
10      pinMode(LED, OUTPUT);
11  }
```

```
13  void loop()
14  {
15    if(Serial.available()>0){ //수신된 데이터 존재 여부 확인
16
17      data = Serial.read();//시리얼 입력값, 한 글자씩 data에 저장
18
19      if(data==10) {//엔터입력시 실행
20
21        adapt = input.toInt();//입력받은 숫자 형변환
22
23        if(0<=adapt && adapt<=255){//입력받은 숫자가 아날로그 값 범위 내일 경우 실행
24
25          Serial.println("Brightness:"+input);//input값 확인
26
27          while(Serial.available()<=0){//수신된 데이터 존재 여부 없으면 계속실행
28            analogWrite(LED, input.toInt());
29            delay(10);
30          }
31
32        }else Serial.println("wrong input");
33
34        input="";//초기화
35      }
36      else input+=(char)data;   /input data 저장
37    }
38
39  }
```

02 LED 다중 연결

LED 다중 연결


음극은 모아서 처리!
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM~)
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
ARDUINO
UNO
a b c d e
f g h i j

uno_ex3-6 : 신호등

```
1   int pin[3] = {9, 6, 3};
2   int i=0;
3
4   void setup() {
5
6     for(int i=0; i<sizeof(pin); i++) pinMode(pin[i], OUTPUT);
7   }
8
```

```
9   void loop() {
10
11    switch(i){
12      case 0: //빨간불일 경우
13        digitalWrite(pin[i], HIGH);
14        delay(3500);
15        digitalWrite(pin[i], LOW);
16        break;
17
18      case 1: //노랑불일 경우
19        digitalWrite(pin[i], HIGH);
20        delay(1000);
21        digitalWrite(pin[i], LOW);
22        break;
23
24      case 2://파란불일 경우
25        digitalWrite(pin[i], HIGH);
26        delay(2500);
27        digitalWrite(pin[i], LOW);
28        break;
29    }
30
31    //pin 번호 조정
32    if(i<2) i++;
33    else i=0;
34  }
```

uno_ex3-7: 사용자입력으로 LED 디지털 출력

```ino
int pin[3] = {9, 6, 3};
int input=0;
byte data=0;

void setup() {

  Serial.begin(9600);
  for(int i=0; i<sizeof(pin); i++) pinMode(pin[i], OUTPUT);
}

void loop() {

  if(Serial.available()>0){

    data = Serial.read();//사용자 입력

    if(data == 10){
      switch(input){
        case 0:
        case 1:
        case 2:
          Serial.println((String)input+" ON...");

          digitalWrite(pin[input], 1);
          delay(1000);//1초만 켜지고 off
          digitalWrite(pin[input], 0);

          break;

        default: Serial.println("wrong input");
      }
    }else input = data-48;//아스키코드->숫자변환
  }

}
```

결과

- 시리얼 모니터에서 0 + 엔터 → 핀 9 LED 1초 ON
- 시리얼 모니터에서 1 + 엔터 → 핀 6 LED 1초 ON
- 시리얼 모니터에서 2 + 엔터 → 핀 3 LED 1초 ON
- 다른 숫자 입력 → "wrong input"

uno_ex3-8: 사용자입력으로 LED 아날로그 출력

```
1   int pin[3] = {9, 6, 3};
2
3   String data=""; //사용자 입력
4   int no=0; //핀 번호 인덱스 0, 1, 2
5   int adapt=0; //밝기 0~255
6
7   void setup() {
8
9     Serial.begin(9600);//시리얼 통신을 위한 주파수
10
11    for(int i=0; i<3; i++) pinMode(pin[i], OUTPUT);
12
13  }
```

uno_ex3-8: 사용자입력으로 LED 아날로그 출력

```
15  void loop() {
16
17    if(Serial.available()>0){    //수신 데이터 존재 여부 확인
18
19      data = Serial.readString();//사용자 입력 시리얼 버퍼에 있는 문자열 전체를 읽어 String으로 반환
20
21      no = data.charAt(0)-48; 첫 번째 문자 숫자로 변환
22      adapt = data.substring(2,data.length()-1).toInt();//띄어쓰기 이후 문자 숫자로 변환
23                                  6 in case
24      Serial.println("no:"+(String)no+" / adapt:"+(String)adapt);
25
26      while(Serial.available()<=0){ //다음 입력을 받을 때까지 계속 출력
27        analogWrite(pin[no], adapt);
28        delay(10);        // Delay 변화에 따른 깜빡임 효과 설정
29        analogWrite(pin[no], 0);
30        delay(10);
31      }
32    }
33
34  }
```

출력 시리얼 모니터 ×

2 215

20:55:06.091 -> no:2 / adapt:215

substring(2, ...)는 인덱스 **2**부터 문자열을 잘라내는 것

char charAt(index)
String substring(from, to)

index	0	1	2	3	4	5
value	2		2	1	5	₩n

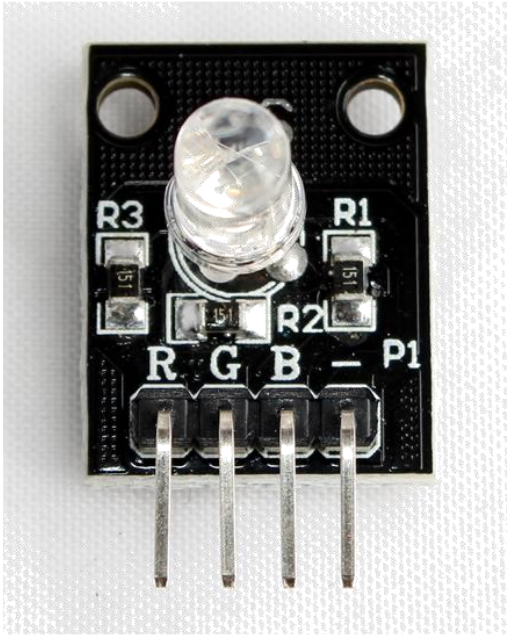
charAt(0)

substring(2,5)

03 RGB 모듈 연결

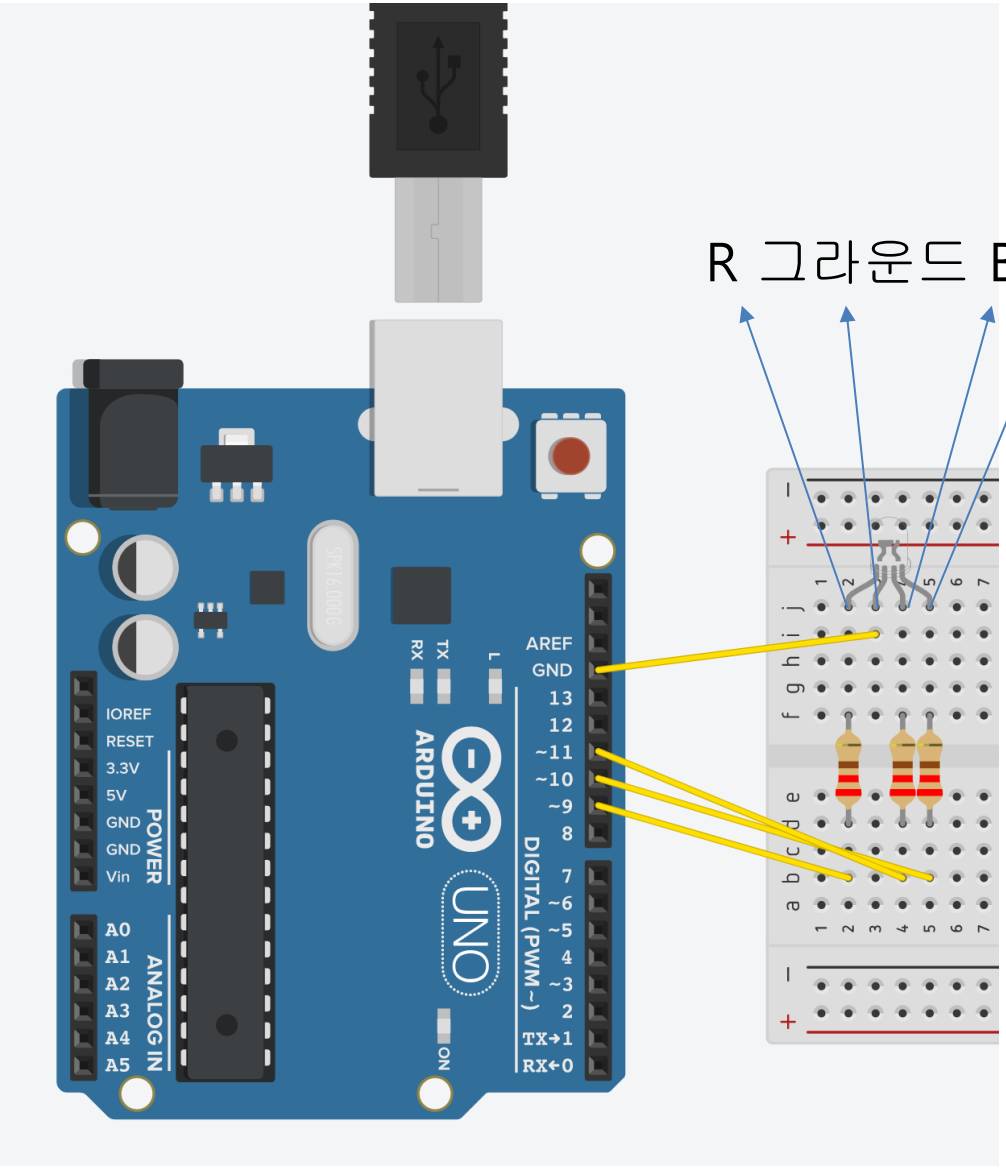
RGB LED

- RGB LED : 내부에 빨간색(R), 초록색(G), 파란색(B)의 광원이 내장되어 컬러를 표현할 수 있는 모듈
  - 3개 색상의 LED가 4개의 다리를 가지도록 구성
  - 각 색상별 양극(+)핀 3개와 음극(-)핀 1개



- RGB LED 제어
  - 디지털 혹은 PWM 신호로 각 LED 색상 밝기 제어
    - 각 LED의 밝기 정도를 조합하여 다양한 색 표현 가능

RGB LED 조명등 회로



RGB LED의 각 LED에
전류 제한을 위한 저항
연결이 필요함

R: 9번 핀
G: 10번 핀
B: 11번 핀

uno_ex3-09:RGB LED 조명등을 위한 디지털 출력

```
const int RLED=9;
const int GLED=10;
const int BLED=11;

int i=1;

void setup()
{
  pinMode(RLED, OUTPUT);
  pinMode(GLED, OUTPUT);
  pinMode(BLED, OUTPUT);

}

void allOff(){
  digitalWrite(RLED, LOW);
  digitalWrite(GLED, LOW);
  digitalWrite(BLED, LOW);

}
```

```
void setMode(int mode)
{
  switch(mode){
    case 1:

      digitalWrite(RLED, LOW);
      digitalWrite(GLED, HIGH);
      digitalWrite(BLED, HIGH);
      delay(500);

      allOff();
      delay(500);
      break;
    case 2:
      digitalWrite(RLED, HIGH);
      digitalWrite(GLED, LOW);
      digitalWrite(BLED, HIGH);
      delay(500);

      allOff();
      delay(500);
      break;

    case 3:
      digitalWrite(RLED, HIGH);
      digitalWrite(GLED, HIGH);
      digitalWrite(BLED, LOW);
      delay(500);

      allOff();
      delay(500);
      break;
  }
}
```

```
void loop() {

    setMode(i);

    if(i<3) i++;
    else i=1;
}
```

uno_ex3-10:RGB LED 조명등을 위한 아날로그 출력

랜덤 밝기를 이용하여 RGB LED를 출력하는 실습

```
1  const int RGB_PIN[]={9,10,11};
2  const int SIZE=3;
3  int randValue[SIZE]={0,0,0};
4
5  void setup()
6  {
7    Serial.begin(9600);//시리얼 통신을 위한 주파수
8
9    //핀 번호 설정
10   for(int i=0; i<SIZE; i++) pinMode(RGB_PIN[i], OUTPUT);
11
12   //랜덤시드값 설정
13   randomSeed(analogRead(0));
14 }
15
16 void allOff(){
17   for(int i=0; i<SIZE; i++) analogWrite(RGB_PIN[i], LOW);
18 }
19
20 void loop() {
21
22   for(int i=0; i<SIZE; i++) {
23     randValue[i] = (int)random(256);
24     analogWrite(RGB_PIN[i], randValue[i]);
25   }
26
27   Serial.println("R:"+(String)randValue[0]+"/G:"+(String)randValue[1]+"/B:"+(String)randValue[2]);
28
29   delay(500);
30
31   allOff();
32   delay(500);
33
34 }
```

analogRead(0): 아날로그 0번 핀의 신호를 받아서 난수 발생 시작값으로 사용
연결되지 않은 analog포트를 읽으면 0~1023중 하나의 값 반환

난수 생성 함수

void randomSeed(unsigned int seed)

- **매개변수**
  - seed : 의사 난수의 시작 위치 결정을 위한 값
- **반환값** : 없음

long random(long max)
long random(long min, long max)

- **매개변수**
  - min : 생성될 난수의 최솟값
  - max : max – 1이 생성될 난수의 최댓값
- **반환값** : [min, max – 1] 범위의 난수값

요약

- LED : 출력 신호 모듈
- LED 모듈을 아두이노와 연결시 반드시 저항필요!
- 디지털 출력 vs 아날로그 출력
  - 디지털 출력 : 0 or 1
  - 아날로그 출력: 0~255
  - 핀 번호 고려하여 연결
- LED 모듈 다중 연결 시 음극은 GND에 한 번에 처리
- RGB모듈 연결: 핀 번호와 R,G,B LED 연결 선 확인

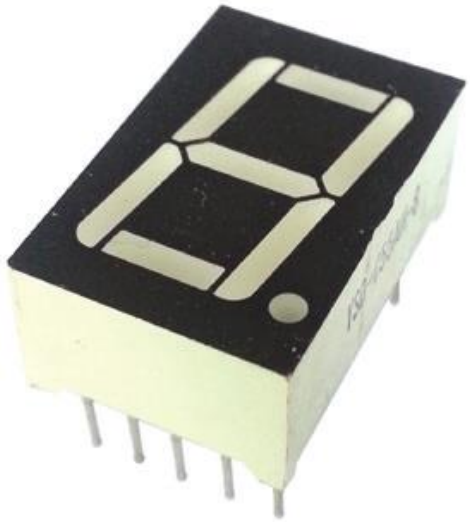
목차

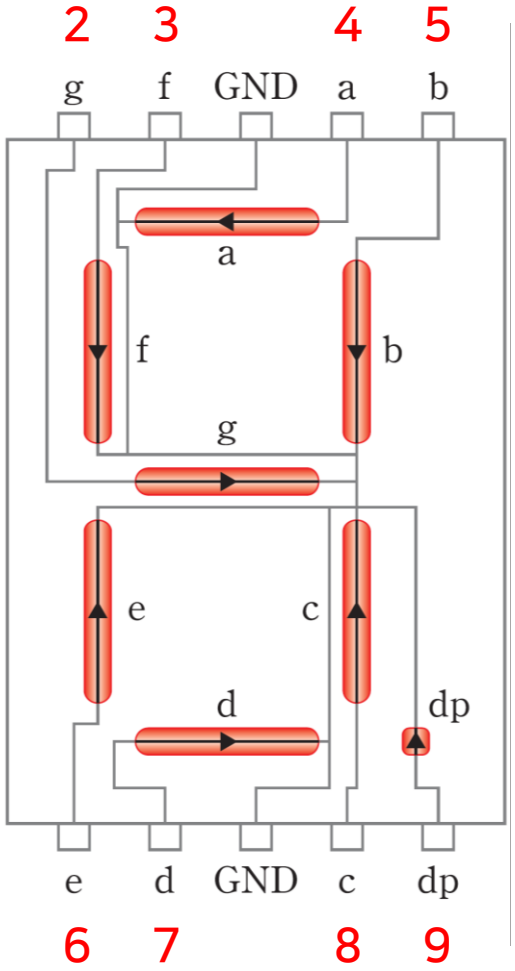
1. 세그먼트 LED
2. LCD
3. OLED

01 세그먼트 LED

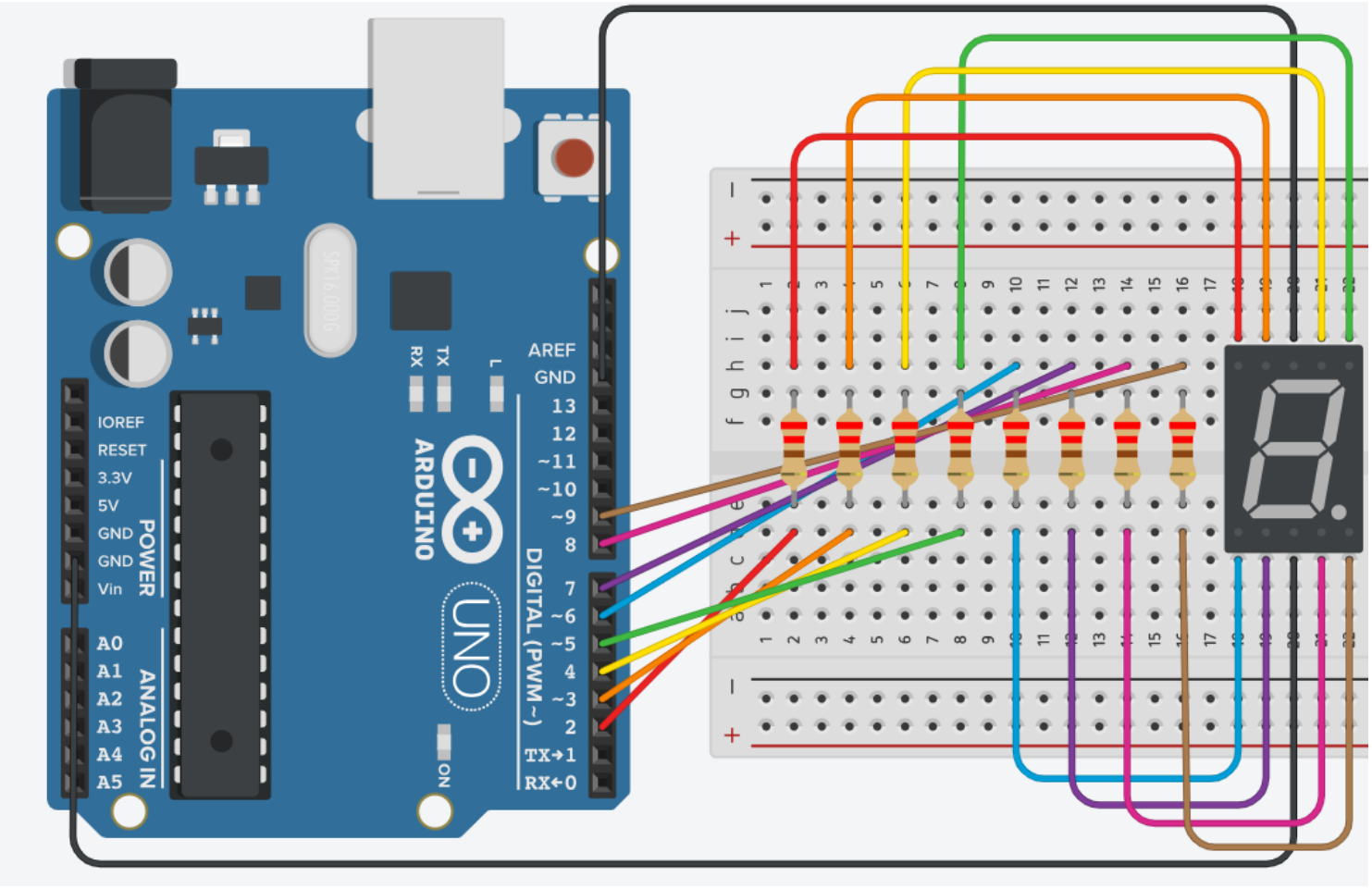
세그먼트 표시 장치

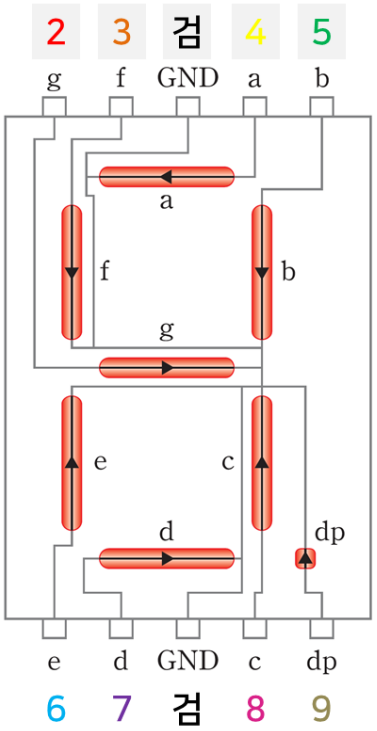
- 8개의 LED로 문자, 숫자 등 표시
  - 문자 표시를 위한 7개의 세그먼트 + 소수점 1개
  - 공통핀과 제어핀으로 구성됨
  - 디지털 출력





세그먼트 회로

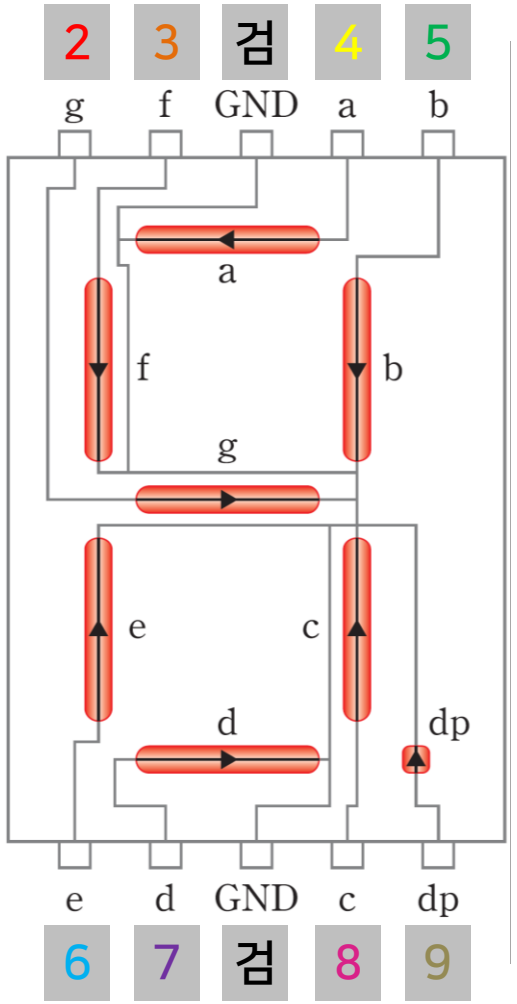

IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
ARDUINO
UNO
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM~)
RX
TX
L
ON


2 3 검 4 5
g f GND a b
a
f
b
g
e
c
d
dp
e d GND c dp
6 7 검 8 9

uno_ex05_01: 세그먼트 제어

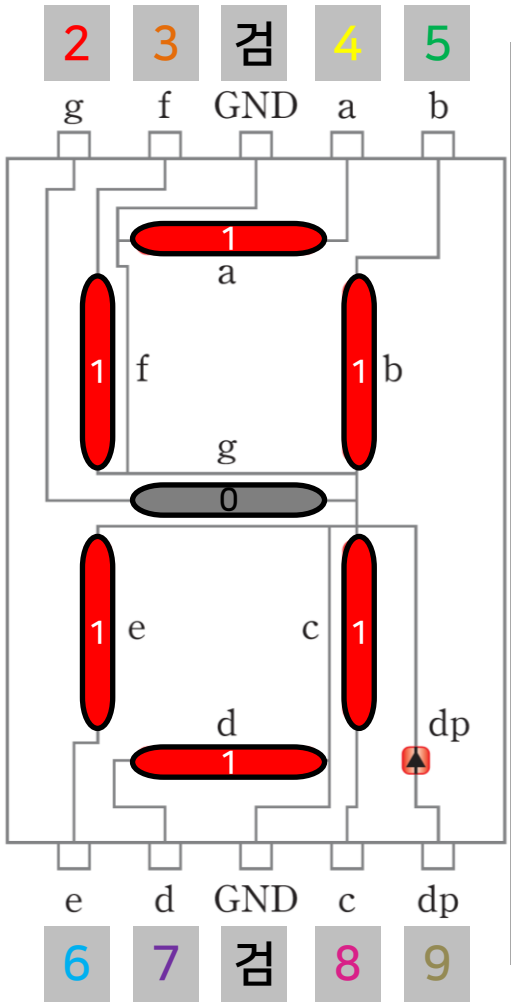
```
1  const unsigned int led_g = 2;
2  const unsigned int led_f = 3;
3  const unsigned int led_a = 4;
4  const unsigned int led_b = 5;
5  const unsigned int led_e = 6;
6  const unsigned int led_d = 7;
7  const unsigned int led_c = 8;
8  const unsigned int led_dp = 9;
9
10 void setup() {
11   // led 초기화
12   pinMode(led_a, OUTPUT);
13   pinMode(led_b, OUTPUT);
14   pinMode(led_c, OUTPUT);
15   pinMode(led_d, OUTPUT);
16   pinMode(led_e, OUTPUT);
17   pinMode(led_f, OUTPUT);
18   pinMode(led_g, OUTPUT);
19   pinMode(led_dp, OUTPUT);
20
21
22   // led 켜기
23   digitalWrite(led_a, HIGH);
24   digitalWrite(led_b, HIGH);
25   digitalWrite(led_c, HIGH);
26   digitalWrite(led_d, HIGH);
27   digitalWrite(led_e, HIGH);
28   digitalWrite(led_f, HIGH);
29   digitalWrite(led_g, HIGH);
30   digitalWrite(led_dp, HIGH);
31
32 }
33
34 void loop() {}
```

반드시 하나씩 확인!!!

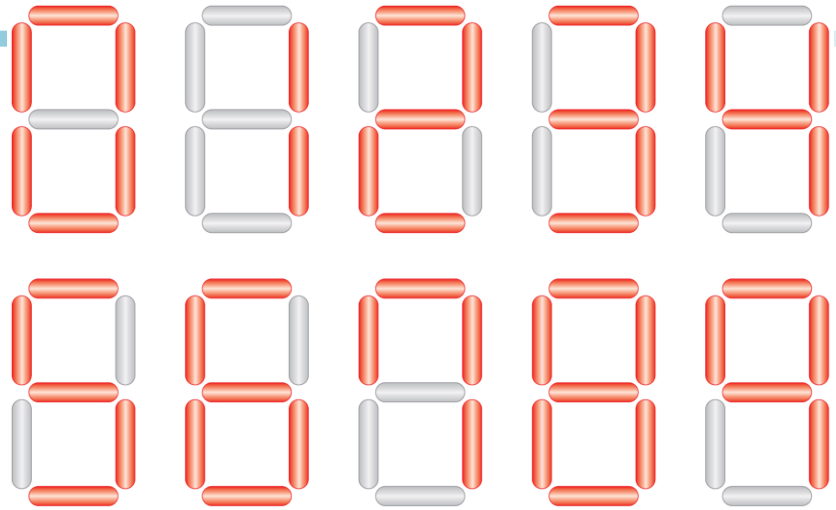


uno_ex05_02: '0' 문자 출력

```
1   const int seg[]  = {2,3,4,5,6,7,8};//소수점 제외
2   const int num0[] = {0,1,1,1,1,1,1};
3
4
5   void setup() {
6     for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
7   }
8
9   void loop() {
10    for(int i=0; i<7; i++) digitalWrite(seg[i], num0[i]);
11    delay(500);
12    for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
13    delay(500);
14  }
```



uno_ex05_03: 0~9 문자 배열에 저장하고 출력



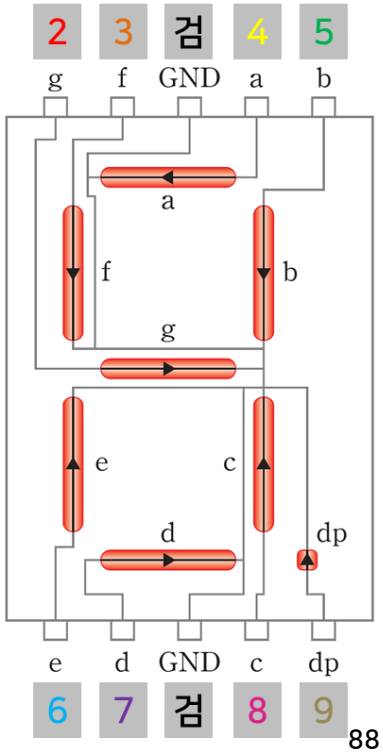
```
const int seg[]  = {2,3,4,5,6,7,8};//소수점 제외
const int numPattern[10][7] = {
  {0,1,1,1,1,1,1}, //문자 0 패턴
  {0,0,0,1,0,0,1}, //문자 1 패턴
  {1,0,1,1,1,1,0}, //문자 2 패턴
  {1,0,1,1,0,1,1}, //문자 3 패턴
  {1,1,0,1,0,0,1}, //문자 4 패턴
  {1,1,1,0,0,1,1}, //문자 5 패턴
  {1,1,1,0,1,1,1}, //문자 6 패턴
  {0,1,1,1,0,0,1}, //문자 7 패턴
  {1,1,1,1,1,1,1}, //문자 8 패턴
  {1,1,1,1,0,0,1} //문자 9 패턴
  };


void setup() {
  for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
}

void loop() {

  for(int j=0; j<10; j++){
      for(int i=0; i<7; i++) digitalWrite(seg[i], numPattern[j][i]);
      delay(1000);
      for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
  }

}
```



uno_ex05_04: 사용자 입력 받아서 해당 문자 출력

```
const int seg[]  = {2,3,4,5,6,7,8};//소수점 제외
const int numPattern[10][7] = {
  {0,1,1,1,1,1,1}, //문자 0 패턴
  {0,0,0,1,0,0,1}, //문자 1 패턴
  {1,0,1,1,1,1,0}, //문자 2 패턴
  {1,0,1,1,0,1,1}, //문자 3 패턴
  {1,1,0,1,0,0,1}, //문자 4 패턴
  {1,1,1,0,0,1,1}, //문자 5 패턴
  {1,1,1,0,1,1,1}, //문자 6 패턴
  {0,1,1,1,0,0,1}, //문자 7 패턴
  {1,1,1,1,1,1,1}, //문자 8 패턴
  {1,1,1,1,0,0,1} //문자 9 패턴
  };

int data=-1;

void setup() {

  Serial.begin(9600);//시리얼 입출력을 위한 시작

  for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
}
```

```
void loop() {

  if(Serial.available()>0){ //수신된 데이터 존재 여부 확인
    data = Serial.read();
    data -=48; //아스키코드->숫자 변환

    if(0<=data && data<=9){

      Serial.println("input:"+(String)data);
      for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
      for(int i=0; i<7; i++) digitalWrite(seg[i], numPattern[data][i]);
    }else if(data != -38) Serial.println("wrong input");
  }

}
```

// 입력 데이터가 **0~9** 범위일 경우에

// 시리얼 모니터에서 출력되는 문자열

// 먼저 모두 꺼주기

// 해당 숫자 패턴 켜기

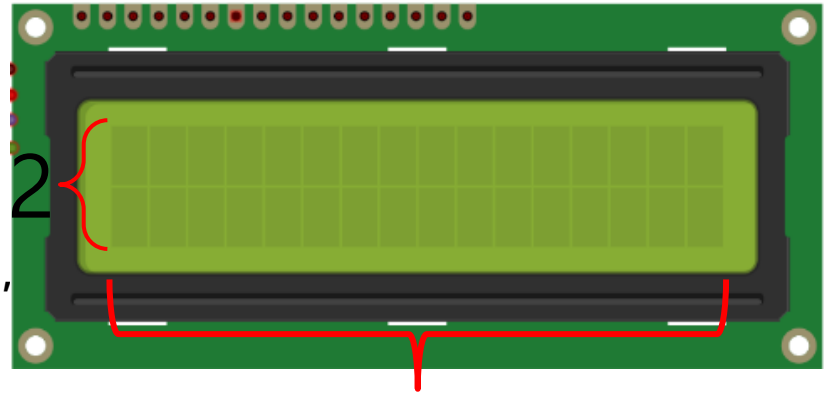
마지막 입력 엔터는 아스키코드값 "10"

// 엔터를 제외한 다른 문자가 입력되면 출력

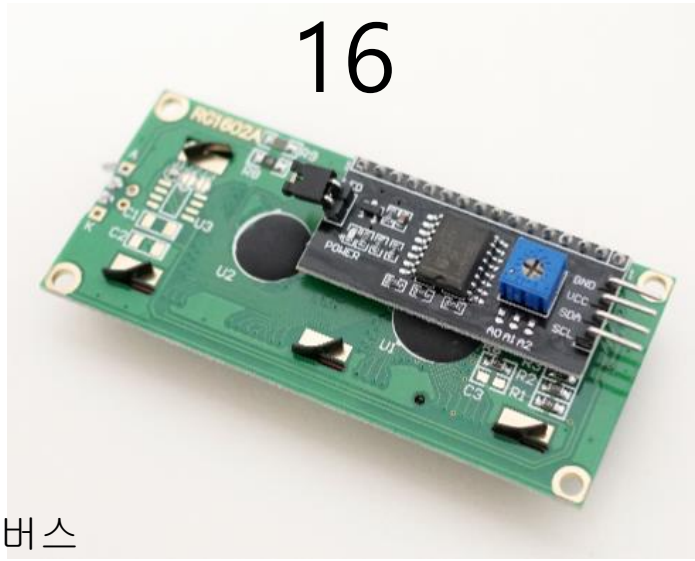
02 LCD

LCD (Liquid Crystal Display)



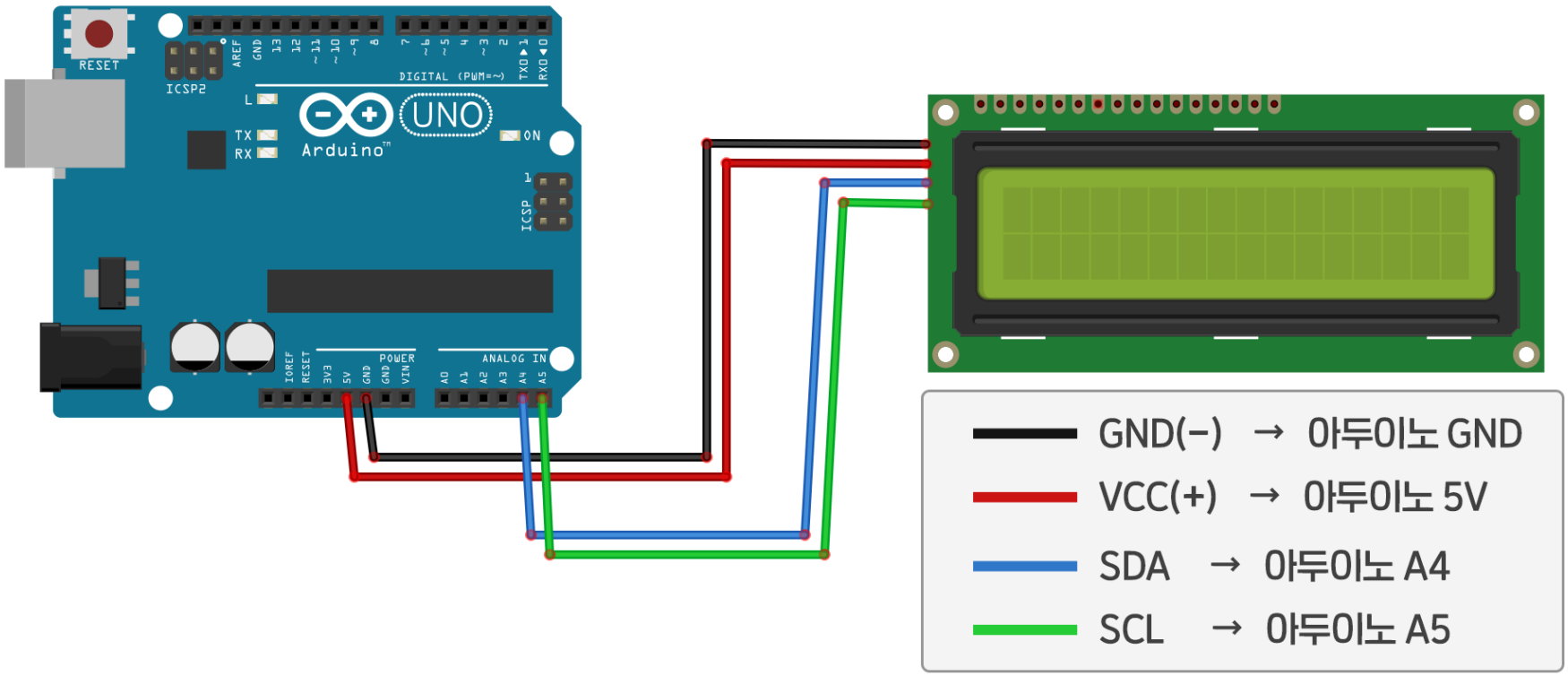


- 병렬 LCD
  - 16x2 크기(32문자)의 LCD를 흔히 볼 수 있음
- I2C LCD 모듈
  - 기존 LCD에 모듈만 결합한 형태
  - 4개의 핀 구성: GND, VCC, SDA(Serial Data Line), SCL(Serial Clock)
  - *I2C Converter에 가변 저항도 내장되어 있기 때문에 선명도 조절을 위한 가변 저항을 따로 추가할 필요없음
  - I2C 모듈이 없다면? 기존 LCD로 아두이노와 연결하여 제어할 경우 총 6개의 digital Pin을 사용해야 하기 때문에 다양한 센서와 기타 부품을 사용할 경우에는 digital port 부족으로 사용에 한계가 있음



* I2C (Inter-Integrated Circuit): 여러 장치 간의 통신을 하나의 버스(2개의 신호 라인)를 통해 처리할 수 있게 하는 통신 프로토콜

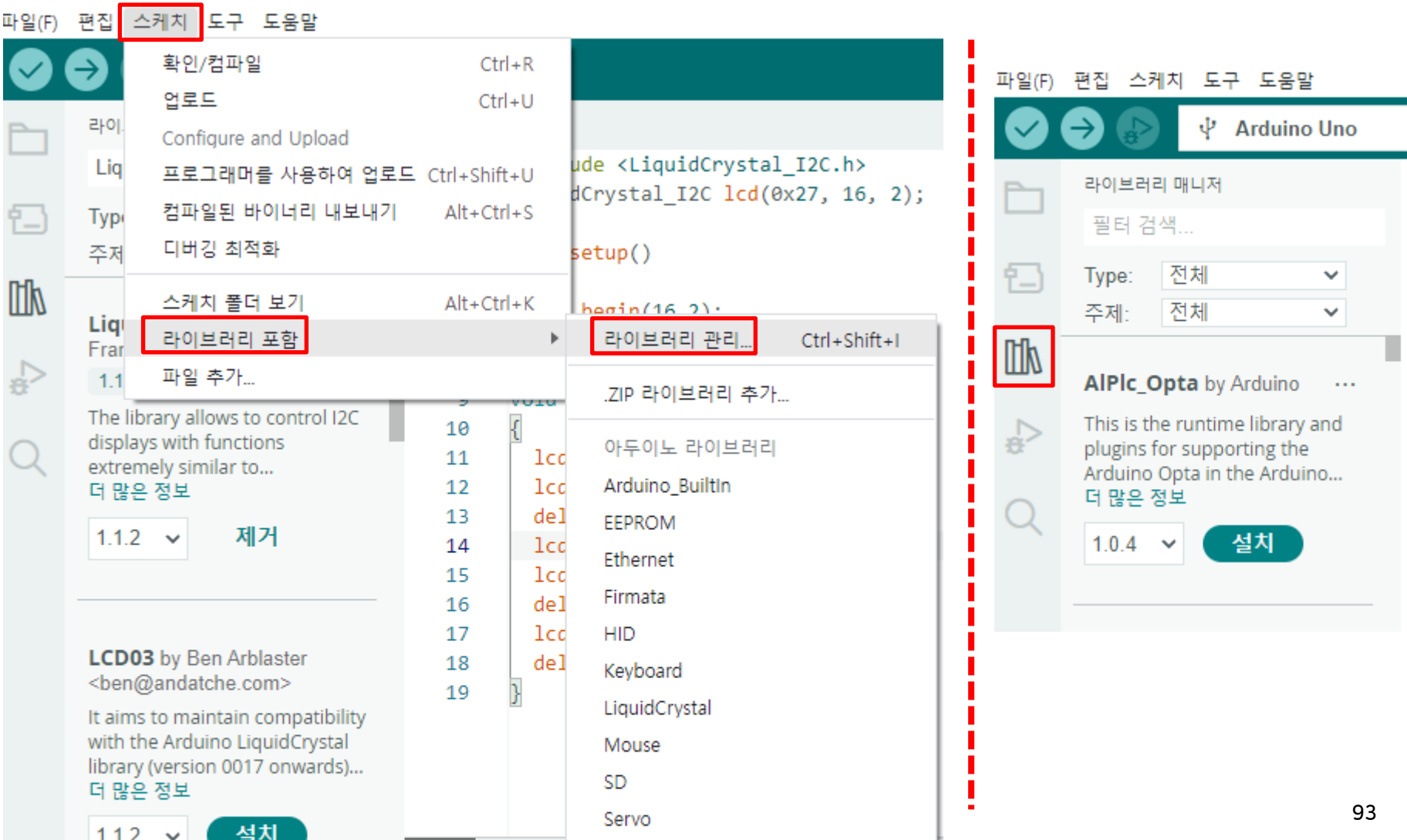
LCD 회로



- SDA (Serial Data Line) : I2C 통신에서 데이터를 주고받는 라인(데이터 라인)
- SCL (Serial Clock Line) : I2C 통신에서 클럭 신호를 전달(클럭 라인)

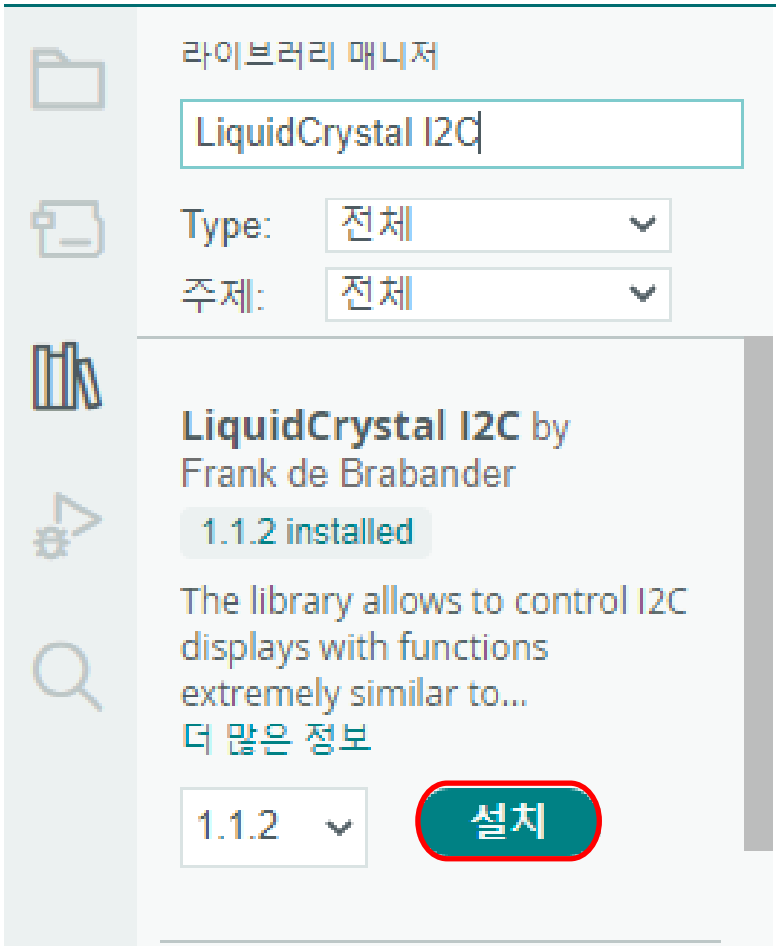
필요한 라이브러리 추가

- “스케치”->”라이브러리 포함”->”라이브러리 관리” 혹은 왼쪽 아이콘 클릭



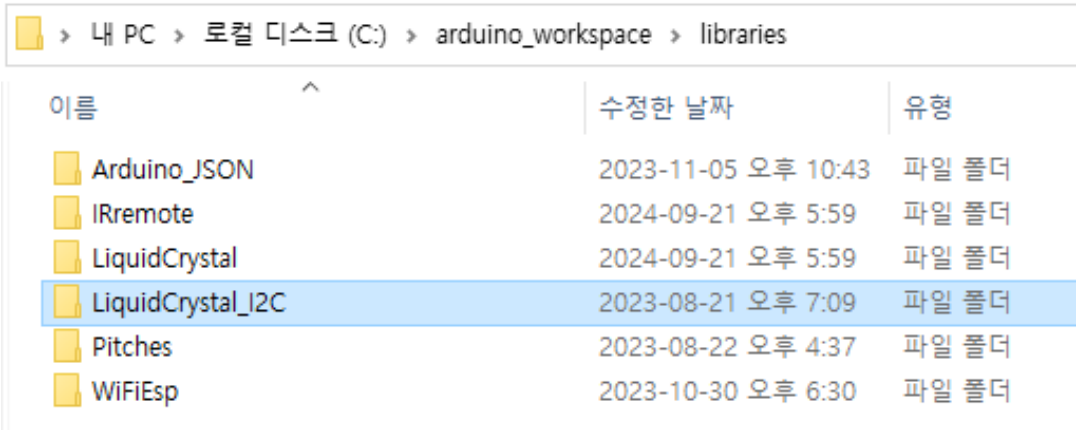
필요한 라이브러리 추가

- “LiquidCrystal I2C” 검색 후 라이브러리 “설치”버튼 클릭

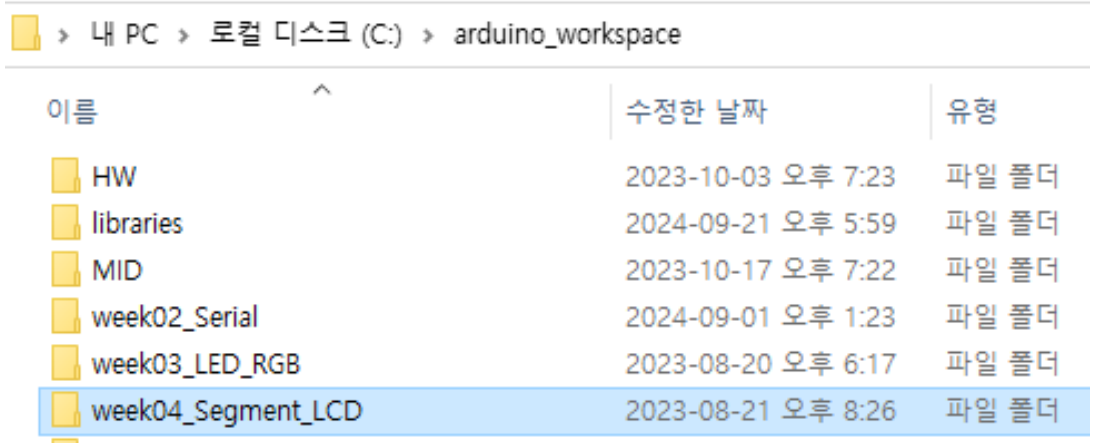


라이브러리 & 소스코드 경로 확인

라이브러리 경로



소스코드 경로



I2C LCD의 I2C 주소 확인하기 (LCD_I2C_addr.ino) : 아래 코드를 입력하고 실행하면 I2C 주소가 출력됨

```
#include <Wire.h>

void setup()
{
  Wire.begin();

  Serial.begin(9600);
  while (!Serial);
  Serial.println("\nI2C Scanner");
}

void loop()
{
  byte error, address;
  int nDevices;
  Serial.println("Scanning...");
  nDevices = 0;
  for(address = 1; address < 127;
address++ )
  {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0)
    {
      Serial.print("I2C device found at
address 0x");
      if (address<16)
        Serial.print("0");
      Serial.print(address,HEX);
      Serial.println("  !");
      nDevices++;
    }
    else if (error==4)
    {
      Serial.print("Unknow error at
address 0x");
      if (address<16)
        Serial.print("0");
      Serial.println(address,HEX);
    }
  }

  if (nDevices == 0)
    Serial.println("No I2C devices
found\n");
  else
    Serial.println("done\n");
  delay(5000);
}
```

I2C LCD의 I2C 주소 확인하기 (LCD_I2C_addr.ino)

- 디지털 핀으로 연결하는 것이 아니기 때문에 주소 반드시 확인!

출력 시리얼 모니터 ×

Message (Enter to send message to 'Arduino Uno' on 'COM3')

```
19:29:29.545 -> Scanning...
19:29:29.545 -> I2C device found at address 0x27  !
19:29:29.588 -> done
19:29:29.588 ->
19:29:34.568 -> Scanning...
19:29:34.568 -> I2C device found at address 0x27  !
19:29:34.568 -> done
19:29:34.568 ->
```

- I2C(Inter-Integrated Circuit)는 마이크로컨트롤러(아두이노 등)와 여러 장치를 연결하는 통신 규격
- **0x27**은 아두이노와 **LCD**가 **I2C** 버스에서 통신할 때 사용하는 **LCD**의 고유 주소임.
- 이 주소를 가진 장치**(LCD)**와만 통신하겠다" 라고 지정하는 **ID** 번호 같은 개념

기본 함수

- #include <LiquidCrystal_I2C.h>
  - 아두이노의 라이브러리 중 하나로 LCD 제어 라이브러리 포함
- LiquidCrystal_I2C lcd(0x27, 16, 2);
  - LCD 제어 객체 생성 : (주소, 행, 열)
- lcd.begin(16, 2);
  - LCD 크기를 지정하여 초기화, 2행(y) 16열(x)
- lcd.setCursor();
  - 문자 표시 위치 지정
  - 좌상단이 (0, 0)으로 기준점

0,0	1,0	2,0	3,0	4,0	5,0	6,0	7,0	8,0	9,0	10,0	11,0	12,0	13,0	14,0	15,0
0,1	1,1	2,1	3,1	4,1	5,1	6,1	7,1	8,1	9,1	10,1	11,1	12,1	13,1	14,1	15,1

- lcd.print()
  - 문자, 숫자 등 출력
  - Serial.print와 기본적으로 같은 방식으로 동작

LiquidCrystal I2C 라이브러리 함수

함수	기능
begin()	LCD를 시작한다.
clear()	내용을 깨끗하게 초기화(삭제)한다.
setCursor(col, row);	col, row로 커서를 이동한다.
print("Codingrun")	Codingrun 글자를 출력한다.
home()	커서를 (0, 0)으로 이동한다.
display()	내용을 표시한다.
noDisplay()	내용을 표시하지 않는다.
backlight()	백라이트를 켠다.
noBacklight()	백라이트를 끈다.
cursor()	밑줄 커서를 표시한다.
noCursor()	커서를 표시하지 않는다.
blink()	깜빡이는 커서를 표시한다.
noBlink()	커서를 표시하지 않는다.
write(ASCII 코드 값)	한 문자를 출력한다 (ASCII 코드값)
scrollDisplayLeft()	표시된 내용을 좌측으로 1칸 이동한다.
scrollDisplayRight()	표시된 내용을 우측으로 1칸 이동한다.
leftToRight()	문자 표시를 좌측에서 우측으로 설정한다.
rightToLeft()	문자 표시를 우측에서 좌측으로 설정한다.

uno_ex05_05: LCD 문자 출력

```
1  #include <LiquidCrystal_I2C.h>
2
3  LiquidCrystal_I2C lcd(0x3F ,16 ,2);
4
5  void setup()
6  {
7  lcd.begin(16,2);     // 16과 2는 아래 LCD 모듈에서 16열과 2행을 말함
8  lcd.init();          // LCD 초기화
9  lcd.backlight();     // 백라이트 온
10 lcd.clear();         // 화면 지움
11 }
12
13 void loop()
14 {
15 lcd.setCursor(5,0);       // LCD 모듈의 0번행의 5번째 열로 이동
16 lcd.print("Hi :)");
17 delay(1000);
18 lcd.setCursor(3,1);     // LCD 모듈의 1번행의 3번째 열로 이동
19 lcd.print("Coding Run");
20 delay(1000);
21 lcd.clear();
22 delay(1000);
23 }
24
25
26
```

					H	i		:	)						
			C	o	d	i	n	g		R	u	n			

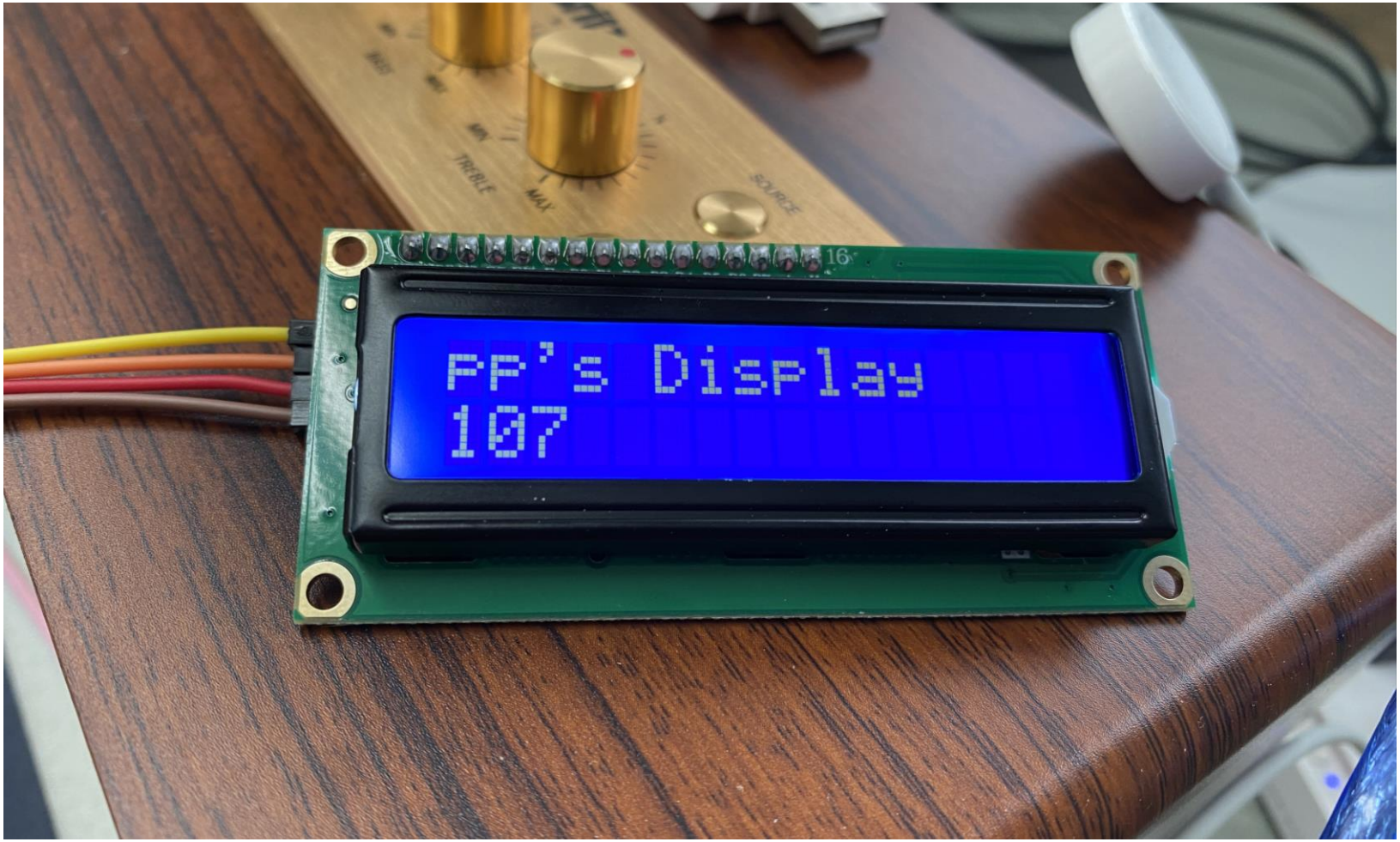
uno_ex05_06: LCD 문자 출력 2

```
1  #include <LiquidCrystal_I2C.h>
2
3  // 시간 변수
4  int time = 0;
5
6  // LCD 객체 생성 (I2C 주소 0x27, 16x2 LCD)
7  LiquidCrystal_I2C lcd(0x27, 16, 2);
8
9  void setup() {
10   lcd.begin(16, 2);   // 16x2 LCD 설정
11   lcd.init();         // 초기화
12   lcd.backlight();    // 백라이트 켜기
13   lcd.print("pp's Display");
14 }
15
16 void loop() {
17   // --- 첫 번째 기능: 간단한 문구 출력 ---
18   lcd.setCursor(0, 1);      // 두 번째 줄 맨 앞
19   lcd.print(time);
20   delay(1000);              // 1초 대기
21   time++;                 // 시간 값 증가
22
23 }
24
25
```

주의: LiquidCrystal 라이브러리는 문자열의 길이를 확인하지 않는다.
출력하려는 문자열이 LCD 화면 크기에 알맞은지 반드시 확인한다.

// 0x27번 주소를 가진 16열과 2행 LCD 모듈을 I2C 통신을 하겠다라는 의미

상태바 만들기



사용자 정의 문자 생성하기

- 그리스 문자나 온도 표시 단위, 진행 표시 등 일반 문자로 표현할 수 없는 정보를 LCD로 출력하고 싶을 때에는 사용자 정의 문자를 생성할 수 있음
- LCD를 자세히 살펴보면 각 문자는 5x8 픽셀의 사각형 블록으로 구성
- 이 픽셀들의 값을 정의한 후 그 정보를 LCD로 보내면 사용자 정의 문자를 출력
- 문자 블록의 20%를 채운 사용자 정의 문자

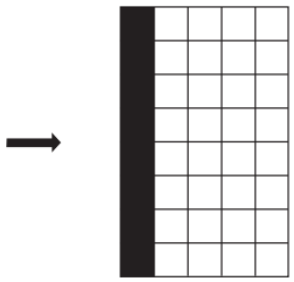
```
byte p20[8]={
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
}
```



B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0
B	1	0	0	0	0

사용자 정의 문자 생성하기

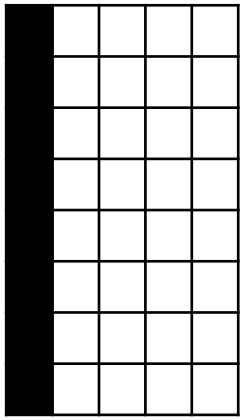
```
byte p20[8]={
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
    B10000,
}
```



- 이 코드에서는 각 문자 블록당 20%만 채우기 때문에 바이트 배열의 이름을 “p20” 지정
- 사용자 정의 문자ID는 0에서 7까지 쓸 수 있으므로 총 8개의 사용자 정의 문자 생성 가능
- Setup() 함수에서는 p20 배열을 0번 문자로 매핑하기 위해 다음과 같이 createChar() 함수 작성
  - lcd.createChar(0, p20);
- 사용자 정의 문자를 LCD로 출력할 때에는 다음과 같이 write() 함수를 사용한다.
  - lcd.write(0);

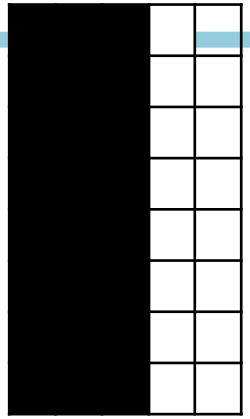
사용자 정의 문자 생성하기

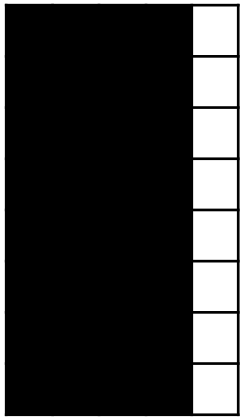
```
7   //진행 상태 바를 나타내는 바이트 배열 선언
8   byte p20[8]={
9       B10000,
10      B10000,
11      B10000,
12      B10000,
13      B10000,
14      B10000,
15      B10000,
16      B10000,
17  };
18
19  byte p40[8]={
20      B11000,
21      B11000,
22      B11000,
23      B11000,
24      B11000,
25      B11000,
26      B11000,
27      B11000,
28  };
```

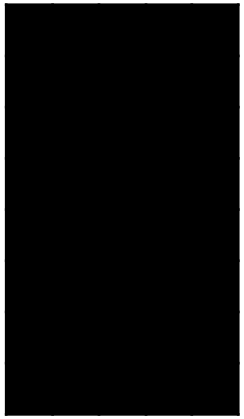


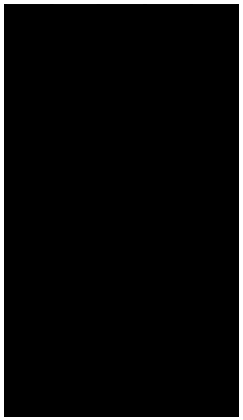
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0

```
29  byte p60[8]={
30      B11100,
31      B11100,
32      B11100,
33      B11100,
34      B11100,
35      B11100,
36      B11100,
37      B11100,
38  };
39
40  byte p80[8]={
41      B11110,
42      B11110,
43      B11110,
44      B11110,
45      B11110,
46      B11110,
47      B11110,
48      B11110,
49  };
50  byte p100[8]={
51      B11111,
52      B11111,
53      B11111,
54      B11111,
55      B11111,
56      B11111,
57      B11111,
58      B11111,
59  };
```









uno_ex05_07: 진행 상태 바 출력하기

uno_ex4-7.ino

```
1   //LCD 라이브러리 삽입
2   #include <LiquidCrystal_I2C.h>
3
4   //회로에 연결된 핀 번호를 사용하여 lcd 객체 인스턴스 생성
5   LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
7   //진행 상태 바를 나타내는 바이트 배열 선언
8   byte p20[8]={
9       B10000,
10      B10000,
11      B10000,
12      B10000,
13      B10000,
14      B10000,
15      B10000,
16      B10000,
17  };
18
19  byte p40[8]={
20      B11000,
21      B11000,
22      B11000,
23      B11000,
24      B11000,
25      B11000,
26      B11000,
27      B11000,
28  };
```

```
29  byte p60[8]={
30      B11100,
31      B11100,
32      B11100,
33      B11100,
34      B11100,
35      B11100,
36      B11100,
37      B11100,
38  };
39
40  byte p80[8]={
41      B11110,
42      B11110,
43      B11110,
44      B11110,
45      B11110,
46      B11110,
47      B11110,
48      B11110,
49  };
50  byte p100[8]={
51      B11111,
52      B11111,
53      B11111,
54      B11111,
55      B11111,
56      B11111,
57      B11111,
58      B11111,
59  };
```

uno_ex05_07: 진행 상태 바 출력하기

```
61  void setup()
62  {
63      //LCD의 행과 열을 16행 2열로 설정
64      lcd.begin(16, 2);
65
66      lcd.init();//초기화
67      lcd.backlight();//배경 라이트 on
68
69      //진행 상태 바 만들기
70      lcd.createChar(0, p20);
71      lcd.createChar(1, p40);
72      lcd.createChar(2, p60);
73      lcd.createChar(3, p80);
74      lcd.createChar(4, p100);
75  }
```

```
77  void loop()
78  {
79    lcd.clear();
80    //LCD에 문자열 출력
81    lcd.print("pp's Display");
82
83    //커서를 2열 1행으로 이동
84    lcd.setCursor(0,1);
85
86      //2열에서 진행 상태 바를 만들려고 16문자만큼 반복
87    for(int i=0; i<16; i++)
88    {
89        //각 문자마다 다섯 번 반복하여 진행 상태 바를 만듦
90        for(int j=0; j<5; j++)
91        {
92          lcd.setCursor(i, 1); //i행으로 커서 이동
93          lcd.write(j); //상태 바 업데이트
94          delay(100); //0.1초간 대기
95        }
96    }
97
98  }
```

	0	1	2	3	4
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0
B	1	1	0	0	0

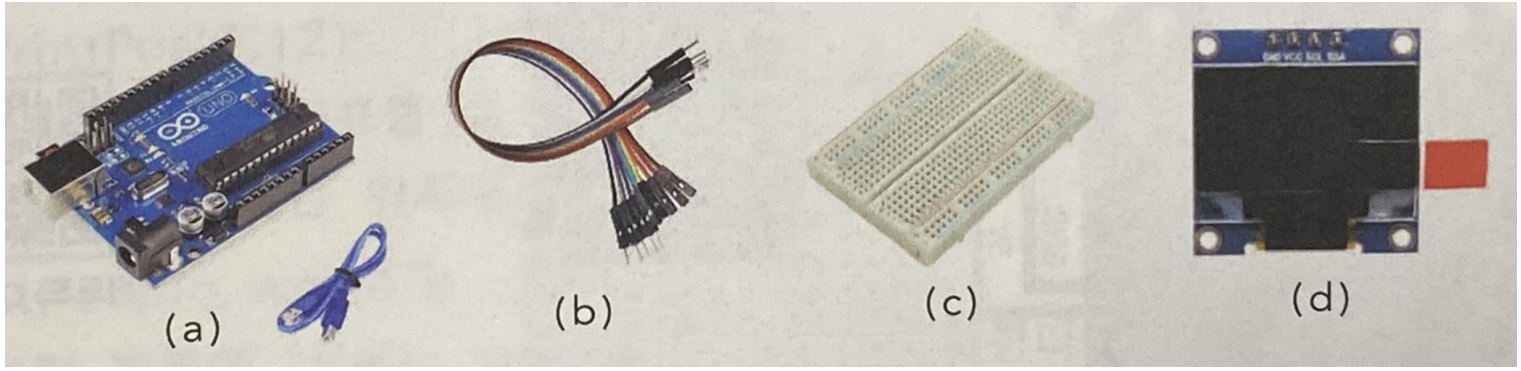
- **사용자 정의 문자(5×8 도트 패턴)를 LCD의 "슬롯"에 저장**하는 함수
- 이 함수 자체가 화면에 뭔가를 출력하지는 않음

- **저장해둔 사용자 정의 문자를 LCD에 출력**하는 함수

3. 0.96인치 OLED 디스플레이

- 개요
  - 이번 장에서는 우노 R3를 이용하여 0.96인치 OLED 디스플레이 제어에 대해 알아보자.
  - OLED 디스플레이 에 "Hellow World" 를 출력, 간단한 도형 및 에니메이션 구현 예제임.

- 사용 디바이스 : 우노 R3(a), 점퍼 케이블(b), 브레드 보드(c), 0.96인치 OLED(d)



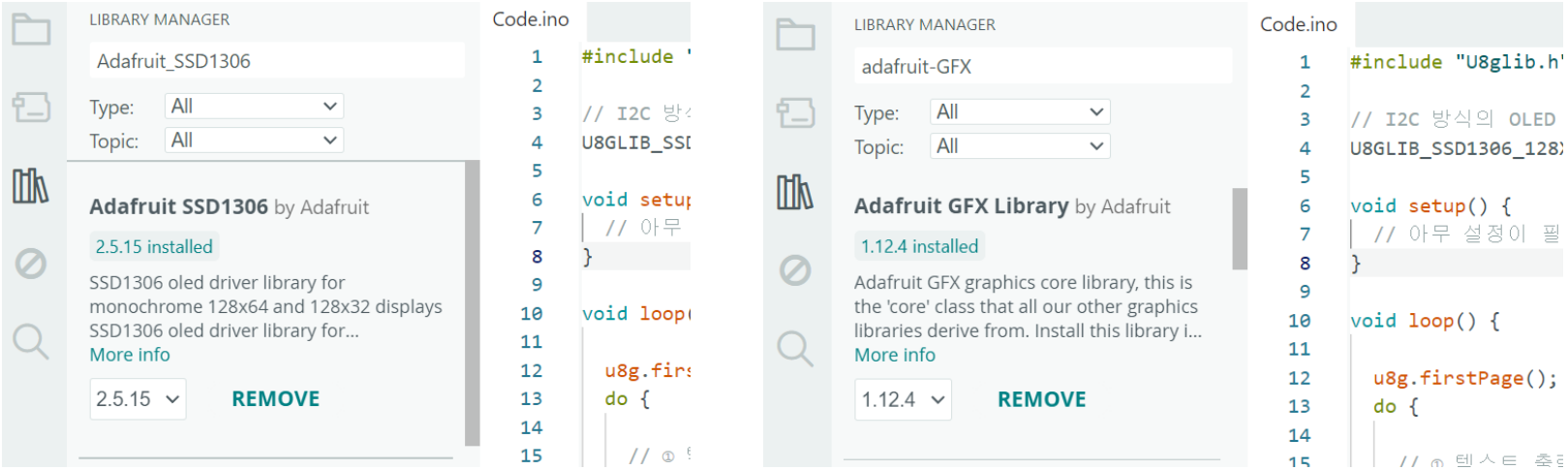
3. 0.96인치 OLED 디스플레이

- 배경 지식
  - OLED OLED는 Organic Light Emitting Diode의 약자임. 즉, 유기발광다이오드. 일반 LED와 달리 자체 발광이 가능하므로 백라이트가 필요없음
  - OLED 0.96 인치 디스플레이 모듈 웨어러블 또는 IoT를 위한 OLED로 0.96 인치의 초소형 사이즈 디스플레이 모듈임.
  - OLED LCD는 아래 표와 같은 특징을 갖으며 저전력으로 동작해 웨어러블 기기에 적합함. 또한 128x64 해상도를 갖고 있어 글자뿐 아니라 이미지 표현이 가능함.

항목	내용
동작 전압	3.3V
해상도	128 × 64
드라이버 IC	SSD1306
통신 인터페이스	I²C
크기	27 × 27 × 4.1 mm

3. 0.96인치 OLED 디스플레이

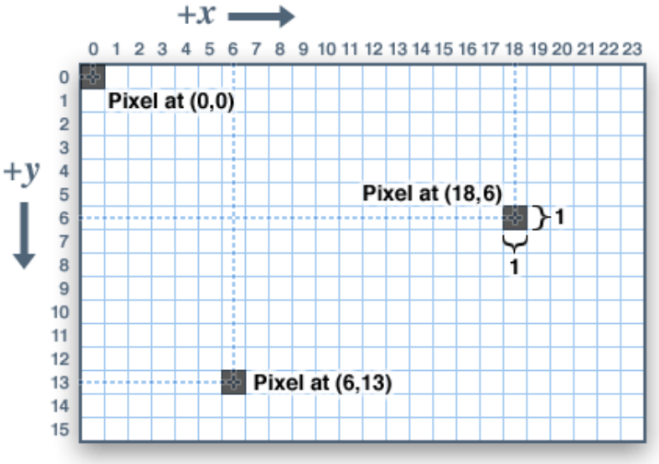
- 라이브러리 다운로드 및 설치하기
  - OLED를 사용하기 위해서는 라이브러리와 Bitmap 파일을 Hex(16진수) 행렬값으로 변환해주는 라이브러리와 헤더파일을 다운 받아야 함.
  - 단, 글씨만 디스플레이 하려면 라이브러리 파일만 다운로드 받으면 됨.
  - 아두이노 라이브러리 설치과정은 아두이노 IDE에서 스케치 – 라이브러리 포함하기 - 라이브러리 관리에서 Adafruit_SSD1306/ Adafruit-GFX-Library, U8glib 를 선택하여 다운로드 함.

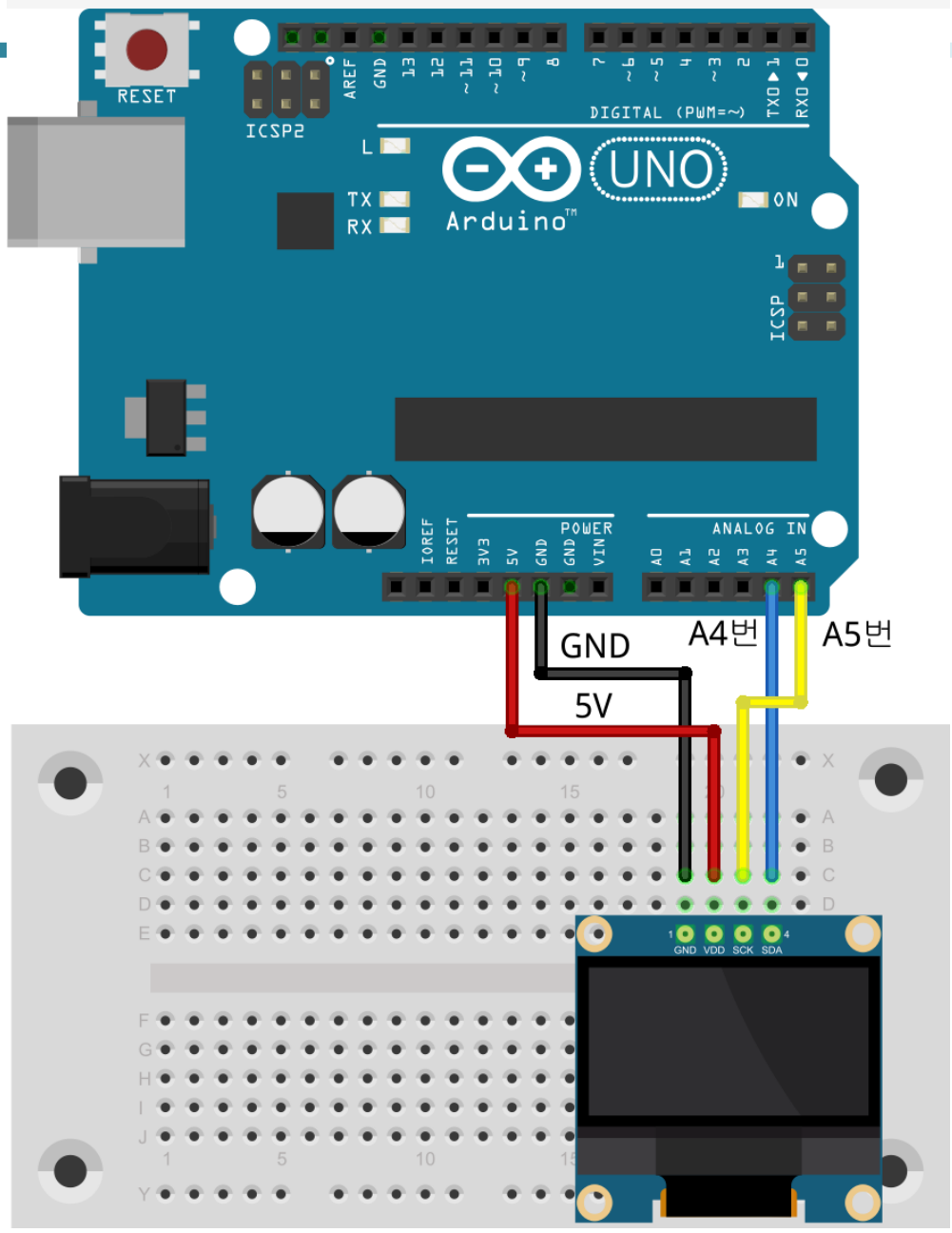


3. 0.96인치 OLED 디스플레이

- 구현 회로도

우노 R3	OLED
아날로그 4번	SDA
아날로그 5번	SCL
5V	VCC
GND	GND





3. 0.96인치 OLED 디스플레이 – 코드 함수 설명

함수 설명

- u8g.firstPage() : 화면의 첫 번째 페이지를 초기화한다. U8glib 은 페이지 모드에서 디스플레이를 업데이트를 하여 이 함수를 이용해 페이지의 시작을 알린다.
- do (... while (u8g.nextPage()) : OLED 디스플레이의 모든 페이지를 순차적으로 그리는 함수 루프이다. do-while 구조 안에서 페이지에 그릴 내용을 정의하고, u8g.nextPage()를 호출해 다음 페이지로 넘어간다. 마지막 페이지까지 그리면 루프가 종료된다.
- u8g.setFont(u8g_font_unifont) : OLED 디스플레이에 출력될 문자의 폰트를 설정한다. 이 코드에서는 미리 정의된 u8g_font_unifont 폰트를 사용한다.
- u8g.setPrintPos(0,12) : 출력할 문자의 시작 좌표를 설정한다. 여기서는 (0, 12) 좌표로 설정하여, 왼쪽 상단에 서 조금 아래로 내려간 위치에서 문자가 출력되도록 한다.
- u8g. print("Hello, world") : 현재 설정된 좌표에 "Hello, world" 문자열을 출력한다.
- u8g. drawStr(0,32, "Hello, world!") : 특정 좌표 (0, 32)에 "Hello, world!"라는 문자열을 출력한다. 8g.print()와 유사하지만, drawStr 함수는 좌표와 문자열을 한 번에 지정하여 출력한다.

3. 0.96인치 OLED 디스플레이 – 텍스트 출력 및 간단한 도형 그리기

```
#include "U8glib.h"
// I2C 방식의 OLED 사용 시 (SSD1306 128x64 기준)
U8GLIB_SSD1306_128X64 u8g(U8G_I2C_OPT_NONE);  //초기화

void setup() {
  // 아무 설정이 필요 없어도 OK
}

void loop() {

  u8g.firstPage(); //첫 페이지 초기화
  do {

    // ① 텍스트 출력
    u8g.setFont(u8g_font_unifont);    // 폰트 설정, 미리 정의된 u8g_font_unifont 폰트를 사용
    u8g.setPrintPos(0, 14); //출력할 문자의 시작 좌표를 설정
    u8g.print("Hello, OLED!"); // 현재 설정된 좌표에 "Hello, world" 문자열을 출력

    u8g.drawStr(0, 30, "Text + Drawing"); //특정 좌표 (0, 32)에 "Hello, world!"라는 문자열을 출력

    // ② 사각형 그리기
    u8g.drawFrame(0, 40, 40, 20);        // 외곽 사각형, drawFrame(시작 X, 시작 Y, 가로 크기, 세로 크기)
    u8g.drawBox(45, 40, 20, 20);         // 꽉 찬 사각형, drawBox(시작 X, 시작 Y, 가로 크기, 세로 크기)

    // ③ 원 그리기
    u8g.drawCircle(90, 50, 10);          // 중심 (90,50), 반지름 10

    // ④ 직선(line) 그리기
    //u8g.drawLine(0, 0, 127, 63);         // 대각선 직선

    // ⑤ 점(pixel) 찍기
    u8g.drawPixel(120, 10);

  } while (u8g.nextPage());
  delay(500);
}
```

u8g.drawFrame(x, y, w, h);
x: 사각형 왼쪽 위 모서리의 x좌표 (0~127)
y: 사각형 왼쪽 위 모서리의 y좌표 (0~63)
w: 사각형 가로 길이 (픽셀 단위)
h: 사각형 세로 길이 (픽셀 단위)

u8g.drawBox(x, y, w, h);
x: 사각형 왼쪽 위 모서리의 x좌표 (0~127)
y: 사각형 왼쪽 위 모서리의 y좌표 (0~63)
w: 사각형 가로 길이 (픽셀 단위)
h: 사각형 세로 길이 (픽셀 단위)

u8g.drawCircle(x, y, r);
x: 원 중심 x좌표
y: 원 중심 y좌표
r: 반지름 (픽셀 단위)

u8g.drawPixel(x, y);
x: 점의 x좌표
y: 점의 y좌표

3. 0.96인치 OLED 디스플레이 실습 예제 - 에니메이션

```
#include "U8glib.h"

U8GLIB_SSD1306_128X64 u8g(U8G_I2C_OPT_NONE);

int x = 0;            // 공의 x좌표
int dir = 1;          // 방향 (1=오른쪽, -1=왼쪽)

void setup(){}

void loop(){
  u8g.firstPage();
  do {
    // 공 그리기
    u8g.drawDisc(x, 30, 8);  // (x,30) 위치에 반지름 8짜리 공
  } while (u8g.nextPage());


  // 좌표 이동
  x += dir;

  // 화면 끝에 닿으면 방향 전환
  if (x > 120) dir = -1;
  if (x < 0) dir = 1;

  delay(10);   // 프레임 속도
}
```

요약

- 세그먼트 LED
  - 막대 하나 당 디지털 출력
  - 선과 핀 번호 정확하게 매칭
- LCD
  - 16X2 문자 출력
  - 문자 한 개 당 5X8 데이터 출력
  - 핀 번호가 아닌 출력 버스 주소 확인
  - print함수 등 LiquidCrystal 라이브러리에 정의된 함수 사용

목차

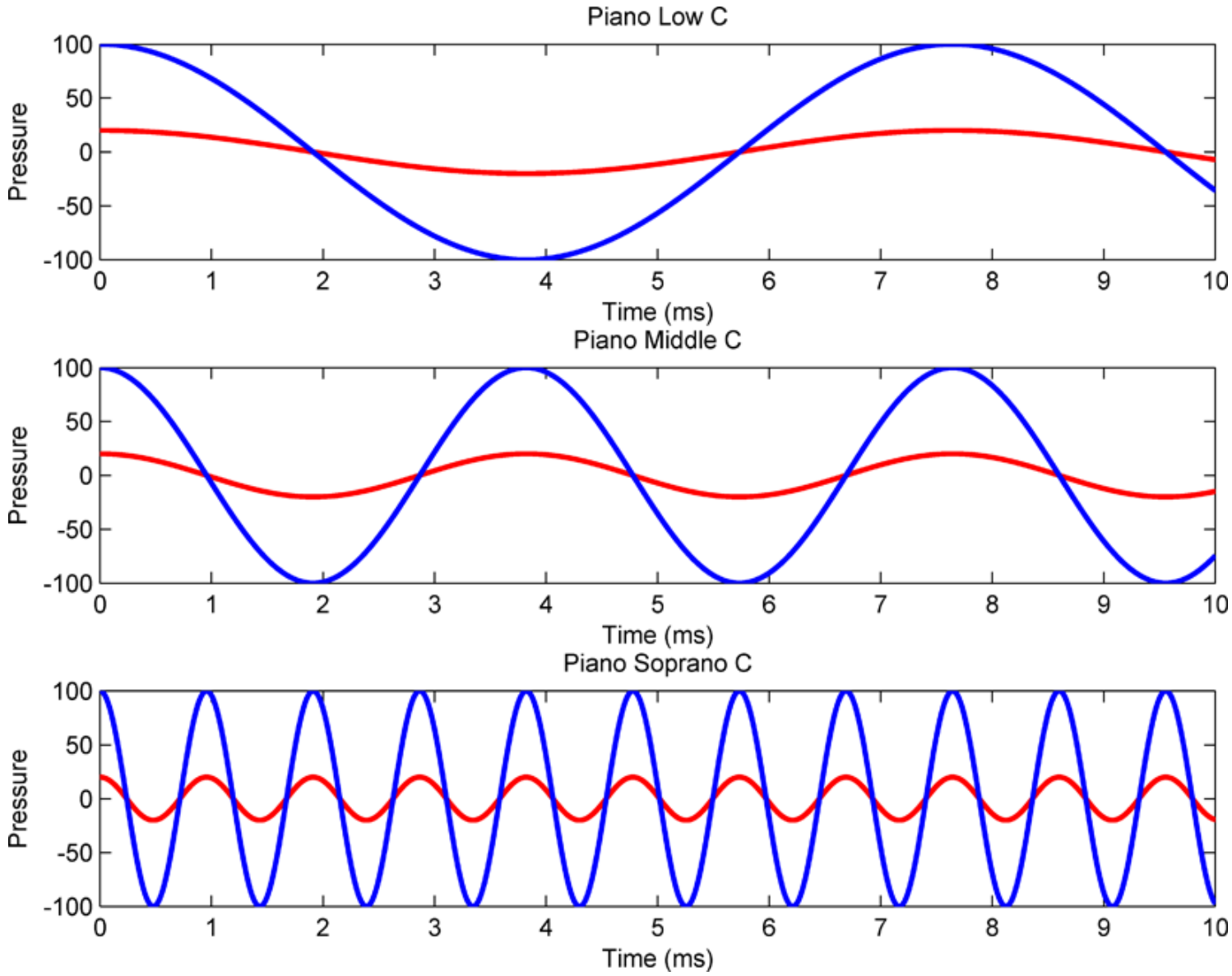
1. 스피커 작동원리
2. 능동부저
3. 수동부저

01 스피커 작동 원리

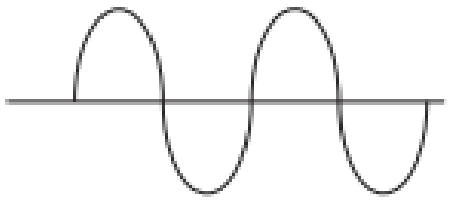
음파의 속성

- 압력파(pressure wave) : 압력 크기의 변화가 만들어 내어 매개물을 통해 전달되는 파동
  - 소리는 공기로 전달되는 압력파
  - 스피커, 드럼, 종 등 물체가 진동하면 주변의 공기도 진동
  - 공기 분자는 진동하면서 주변 분자에 에너지를 전달하고 주변의 공기 분자도 함께 진동
  - 진동 입자의 연쇄반응을 통해 인간의 고막으로 소리를 전달
- 주파수(frequency): 공기 입자가 얼마나 빨리 앞뒤로 진동하는지 나타내는 것
  - 주파수가 높아질 수록 음은 높아지고, 낮아질 수록 음도 낮아짐
  - 중간 도 음의 주파수=261.63Hz
    - 1초 동안 정확히 261.63번 진동하면서 도 음을 만든다는 뜻
- 진폭(amplitude): 진동의 크기
  - 진폭이 클 수록 소리가 크고, 진폭이 작을 수록 소리가 작음
  - 진폭은 스피커로 흐르는 전류의 양으로 조절 가능

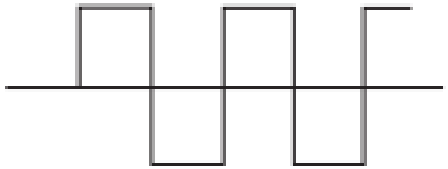
다양한 주파수와 진폭의 음파



사인파와 네모파



▶ 사인파



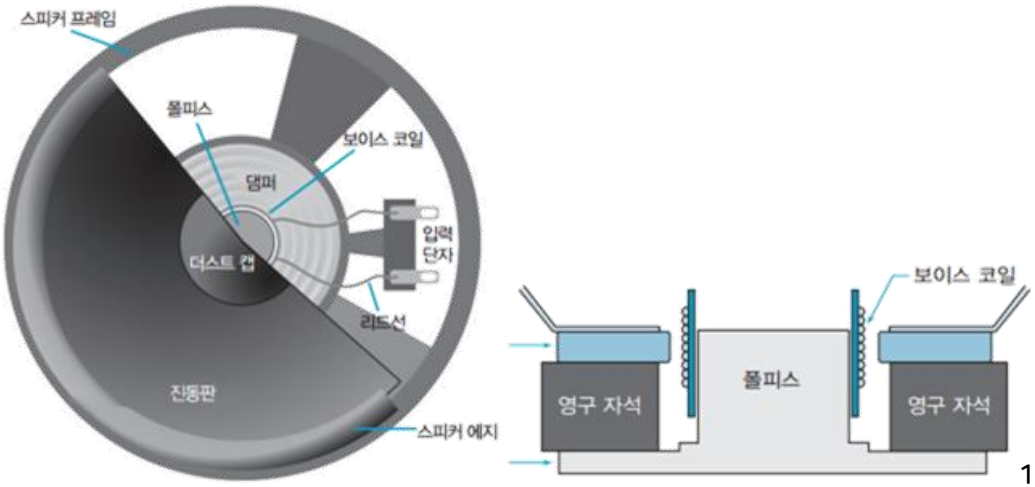
▶ 네모파

- 실제 소리는 사인 곡선 형태의 파형으로 나타냄 ➔ 아날로그 파형
- 아두이노 우노는 실제 소리와 비슷한 사인파를 만들지 못함
- 네모파로도 소리를 만드는 압력파를 생성할 수는 있음 ➔ 디지털 파형(네모파)
  디지털 주기 파형으로 HIGH값과 LOW값 사이 를 즉시 바꾸는 형태
- 아날로그 형태의 사인파처럼 소리가 예쁘지는 않다.
- MP3 등 음악이나 영화도 고해상도의 네모파로 만든 사인파
- DAC(Digital to Analog Converter)를 사용하면 고해상도의 사인파를 만들 수 있고, 컴퓨터에 저장된 모든 음악 파일은 이런 고해상도의 디지털 사인파로 구성된 것

스피커의 구조

스피커의 내부 구조

- 모터와 마찬가지로 스피커도 전기를 운동으로 바꾸기 위해 전자기력을 사용
- 전압 신호(사인파 또는 네모파)가 보이스 코일로 보내지면 전류의 변화가 시작되어 자기장을 유도
- 이 자기장의 영향으로 영구자석이 보이스 코일을 끌어당겼다 밀어냈다 하면서 폴피스와 보이스코일에 연결된 진동판을 앞뒤로 떨리게 함
- 앞뒤로 움직이는 이 떨림이 스피커 앞의 공기를 진동시켜 음파가 생성되어 우리 귀까지 소리 전달



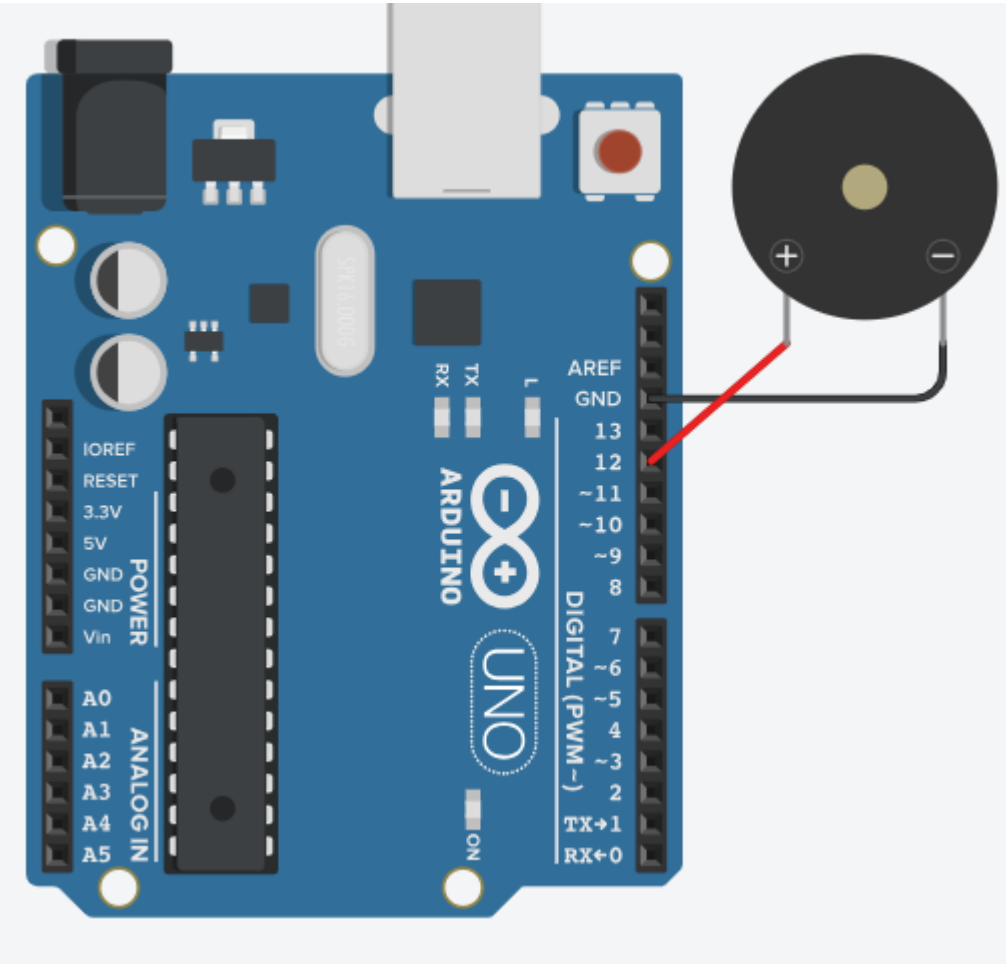
© GNU Free Documentation License

부저

- 부저 (Buzzer, beeper)
  - 단순한 소리를 내는 스피커 같은 장치
  - 간단하게 소리를 생성 가능
  - 아두이노 디지털 핀에 연결해서 디지털 출력으로 제어 가능
- 능동부저 : 전류만 흐르면 자동으로 소리 출력
- 수동부저 (피에조 부저) : 소리 + 헤르츠 지정
  - 아두이노 uno의 경우 31~65,535Hz의 음 출력 가능

02 능동부저

능동부저 회로도





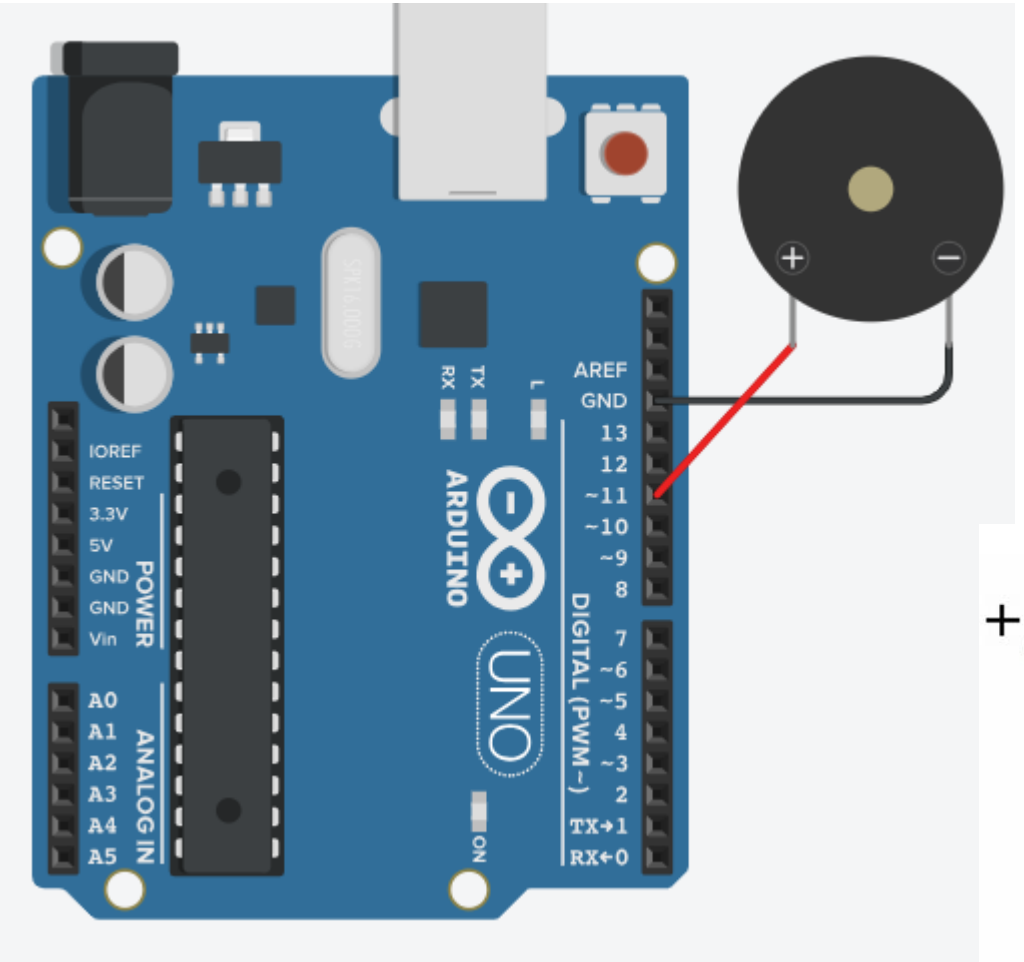
긴 쪽 +
짧은 쪽 -

uno_ex07_01: 능동부저 예제

```
1   int piezo = 12;
2
3   void setup() {
4     pinMode(piezo, OUTPUT);
5   }
6
7   void loop() {
8
9     digitalWrite(piezo, HIGH);
10    delay(1000);
11    digitalWrite(piezo, LOW);
12    delay(1000);
13
14  }
```

03 수동부저

수동부저 회로도


AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM~)
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
ARDUINO
UNO
TX
RX
L
ON


+/- 극성 표시되어 있음

tone( ) 함수

tone 함수

- 지정한 핀으로 지정한 주파수의 구형파(네모파) 출력
- 매개변수
  - 첫 번째 : 구형파를 출력할 핀
  - 두 번째 : 재생할 음의 주파수
  - 세 번째 : 음 재생 시간, 생략하면 noTone() 함수 호출 때까지 재생 계속
- 하드웨어 타이머를 사용하는 논블로킹(non-blocking) 함수 ➔ "논블로킹"은 코드가 멈추지 않는다는 뜻
  - 재생을 시작하면 바로 반환하며, 다른 작업을 수행할 수 있음

tone() 함수는 이 하드웨어 타이머를 이용해
예를 들어 440Hz라면,
→ 1초에 440번 핀을 HIGH/LOW로 바꿔서 스피커를 진동함
이게 바로 소리의 진동(=주파수) 이 됨

4. tone( ) 함수의 한계

- tone( ) 함수는 PWM 신호를 사용
  - 하드웨어 타이머 사용
  - 3번과 11번 핀의 PWM 신호 출력은 사용할 수 없음(* 테스트상 되긴 함, 가급적 사용 금지)
- tone( ) 함수는 구형파를 사용
  - 구형파로 스피커 구동은 가능하지만, 정현파만큼 자연스럽지는 않음
    - 아두이노 듀에와 같이 DAC이 포함된 아두이노 보드 사용
    - 음악 파일 재생을 위한 확장 실드 사용
- tone( ) 함수는 한 번에 하나의 핀으로만 음 재생 가능
  - 여러 개 스피커를 구동하기 위해서는 타이머 관련 인터럽트를 직접 제어해야 할 수 있음

uno_ex07_02: 수동부저 예제

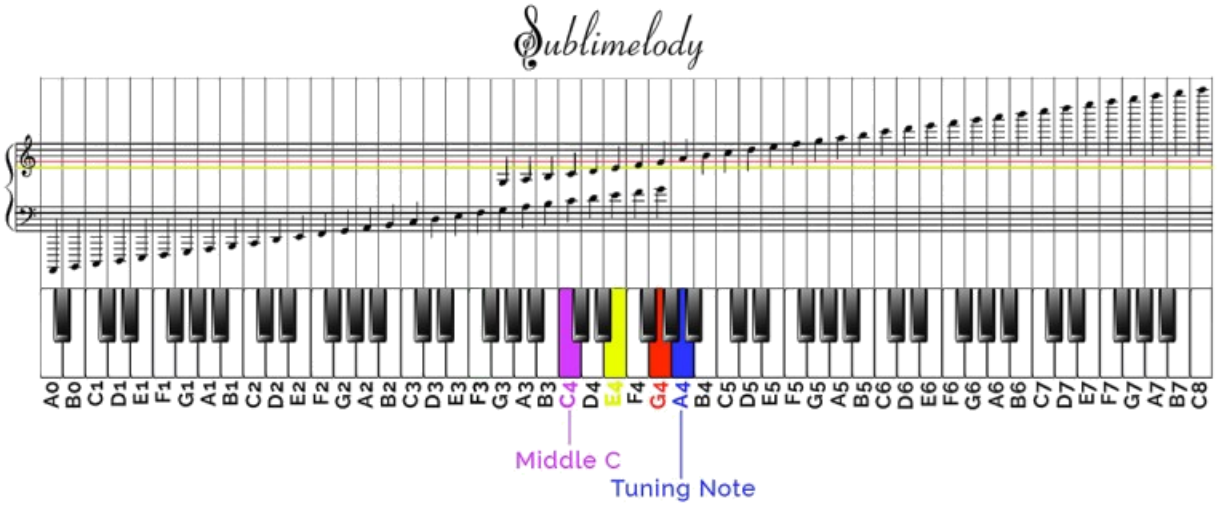
```
1   int piezo = 10;
2
3   int numTones = 8;
4   int tones[] = {261, 294, 330, 349, 392, 440, 494, 523};  // 각 주파수(Hz): 도(261), 레(294), 미(330),
5    //도 레 미 파 솔 라 시 도                                  파(349), 솔(392), 라(440), 시(494), 도(523)
6
7   void setup() {
8     pinMode(piezo, OUTPUT);
9   }
10
11  void loop() {
12    delay(1000);
13
14      for (int i = 0; i < numTones; i++) {
15        tone(piezo, tones[i]);
16        delay(500);
17      }
18      noTone(piezo);
19  }
```

“pitches” 라이브러리 추가

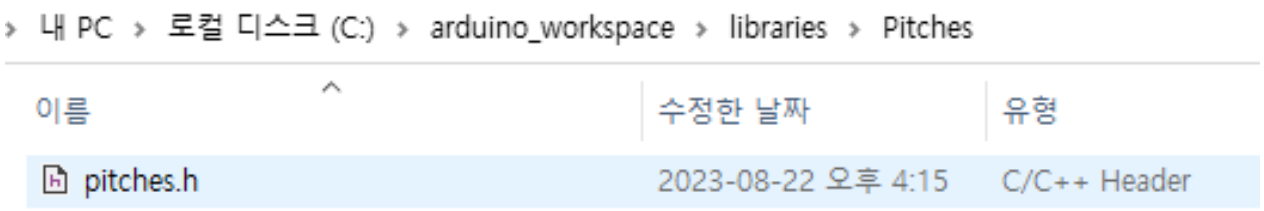
- pitches.h : 음계를 저장한 헤더파일

```
/*************************************************
 * Public Constants
 * pitches.h
 *************************************************/

#define NOTE_B0  31
#define NOTE_C1  33
#define NOTE_CS1 35
#define NOTE_D1  37
#define NOTE_DS1 39
#define NOTE_E1  41
   ⋮
#define NOTE_CS8 4435
#define NOTE_D8  4699
#define NOTE_DS8 4978
```



- “Pitches” 폴더 생성 후 하위에 “pitches.h” 파일 추가



uno_ex07_03: 음악재생

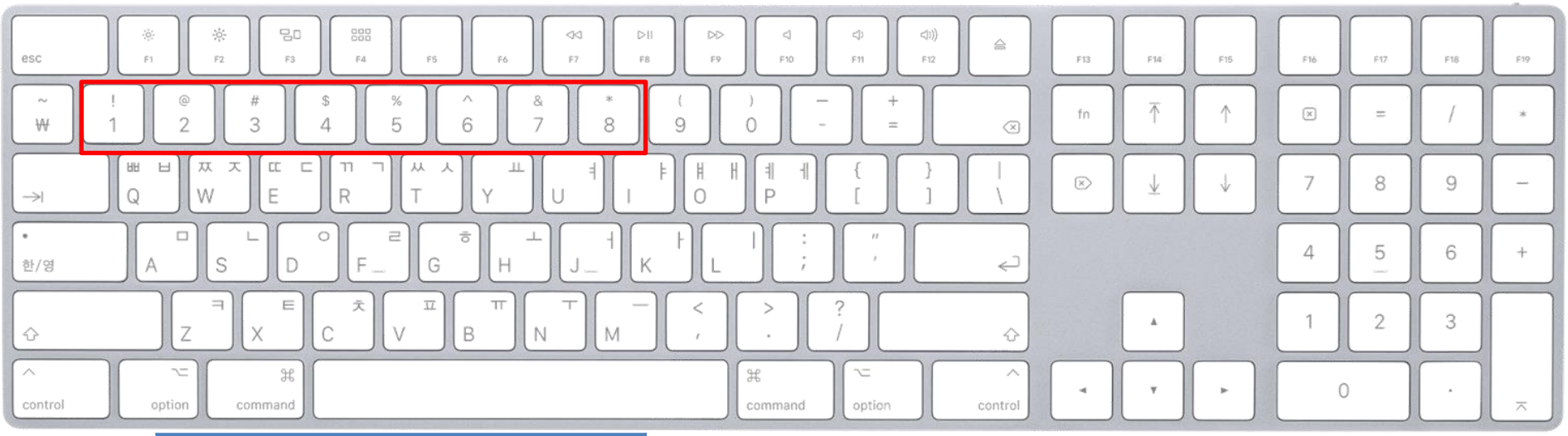
```
1   #include <pitches.h> //계이름이 정의된 헤더 파일 삽입
2
3   const int SPEAKER=10; //10번 핀을 사용하는 SPEAKER 상수 정의
4
5   //연주할 음이 저장된 배열 notes 선언
6   int notes[]= {
7       NOTE_A4, NOTE_E3, NOTE_A4, 0,
8       NOTE_A4, NOTE_E3, NOTE_A4, 0,
9       NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
10      NOTE_E4, NOTE_E3, NOTE_A4, 0
11  };
12
13  //음의 지속 시간이 저장된 배열 times 선언. 단위는 ms
14  int times[]= {
15      250, 250, 250, 250,
16      250, 250, 250, 250,
17      125, 125, 125, 125, 125, 125, 125, 125,
18      250, 250, 250, 250
19  };
```

0은 소리를 내지 않는 것, 즉 쉼표를 의미

uno_ex07_03: 음악재생

```
21  void setup()
22  {
23      pinMode(SPEAKER, OUTPUT);
24  }
25
26  void loop()
27  {
28      //계이름, 지속 시간 배열 순서대로 음악 재생
29      for(int i=0; i<20; i++)
30      {
31          tone(SPEAKER, notes[i], times[i]);
32          delay(times[i]);
33      }
34  }
```

uno_ex07_04: 키보드 피아노



키보드	음계	아스키코드
1	도	49
2	레	50
3	미	51
4	파	52
5	솔	53
6	라	54
7	시	55
8	도	56

uno_ex07_04: 키보드 피아노

```
#include <pitches.h> //계이름이 정의된 헤더 파일 삽입

const int SPEAKER=10; //10번 핀을 사용하는 SPEAKER 상수 정의

int tones[] = {261, 294, 330, 349, 392, 440, 494, 523};
//도, 레, 미, 파, 솔, 라, 시, 도

int input =-1;
void setup()
{
  Serial.begin(9600);
   pinMode(SPEAKER, OUTPUT);
}

void loop()
{
  if(Serial.available()>0){  // 시리얼로 입력이 들어오면

    input = Serial.read();          // 입력(아스키 코드값) 읽기
    input -=48; //아스키코드->숫자 변환  // '0' 문자(48)를 빼서 숫자로 변환

    if(1<=input && input<=8){   // 입력값이 1~8 사이면

      tone(SPEAKER, tones[input-1], 300); // 해당 음 재생
      delay(300);      // 0.3초 유지

    }else if(input != -38) Serial.println("wrong input");

  }

}
```

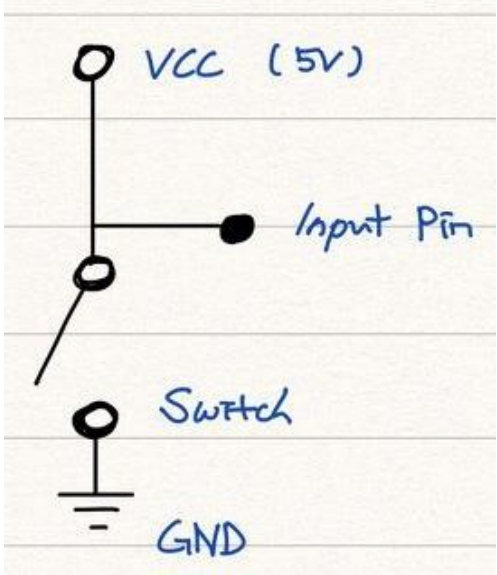
목차

1. 디지털 입력
2. 아날로그 입력
3. 다중 연결

01 디지털입력 : 택트 스위치, 가변저항

디지털 입력값을 읽는 방법

문제: 버튼을 누르면 필요한 작업을 수행하도록 하려고 한다. 어떻게 하면 버튼 눌림을 디지털 입력값으로 감지할 수 있을까?



- 해결 방법
  - 버튼을 누르면 회로가 연결되어 당연히 HIGH값으로 인식 될 것이다.
  - 버튼을 누르지 않았을 때는 입력으로 지정된 해당 입출력 핀이 어디에도 연결되지 않은 채 붕 떠 있는(플로팅) 상태가 된다.
    - 핀자체의 칩에서는 0V, 5V인지 1V인지 인식할 수가 없어서 입력 단자 주위에 정전기나 잡음에 의해서 오류가 생기게 됨.
  - 이 문제는 풀다운 저항으로 해결할 수 있다(풀다운 저항: 낮은 전압에 (Low) 연결되어 있는 저항)

디지털 입력값을 읽는 방법 – 풀다운 저항 사용하기

❖ Pull up 저항

- 높은 전압 (High) 에 연결되어 있는 저항

❖ Pull down 저항

- 낮은 전압 (Low) 에 연결되어 있는 저항

❖ 플로팅 현상 (Floating)

- 연결되어 있지 않은 상태
- 전류가 흐르는지 알 수 없는 상태

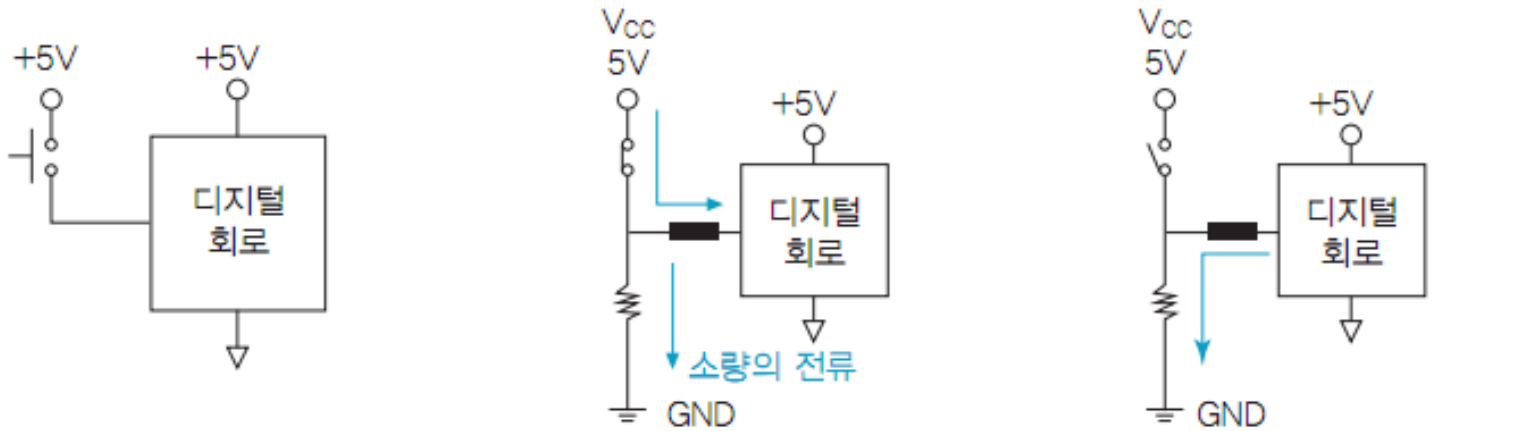


노이즈, 정전기에 취약.
잡음에 의한 오동작 가능성이 큼

안정적인 회로가 필요 ➔ 풀업 및 풀다운 저항 사용

디지털 입력값을 읽는 방법 – 풀다운 저항 사용하기

그림 2-5 버튼의 입력값과 풀다운 저항



(a) 풀다운 저항 사용 안 함

(b) 풀다운 저항 사용 – 버튼 누름(ON)

(c) 풀다운 저항 사용 – 버튼 누르지 않음(OFF)

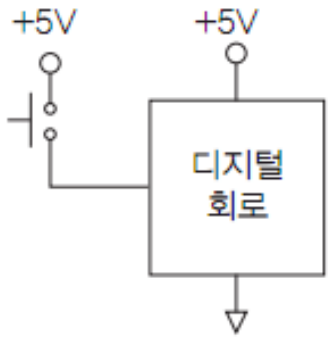
버튼 또는 스위치	누름(ON)	누르지 않음(OFF)
풀다운 저항 사용 안 함	5V(High)	플로팅
풀다운 저항 사용	5V(High)	0V(Low)

풀다운 저항을 사용하면

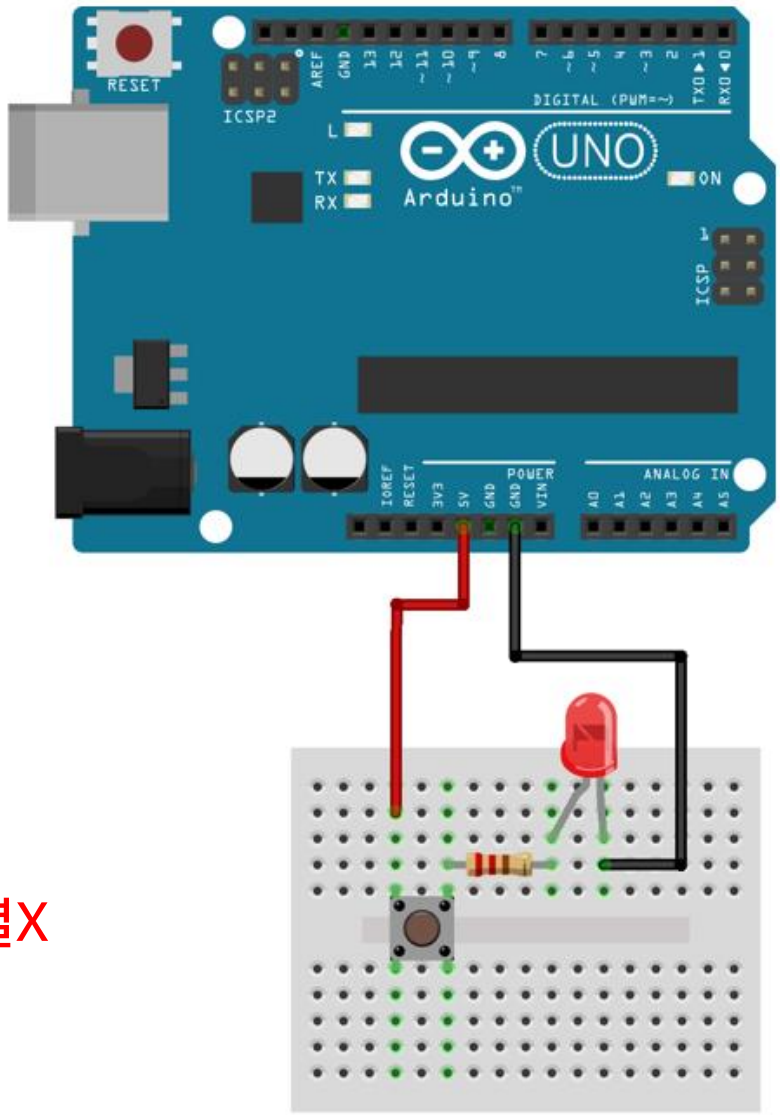
- 버튼을 누르면 전류는 저항이 거의 없는 5V 경로를 따라 흐른다.
- 버튼을 누르지 않으면 전류는 저항이 높은 GND 경로를 따라 흐른다.
- 따라서 풀다운 저항은 단지 "전압을 0V로 잡아주는 역할"을 함

디지털 입력 회로도

- 풀다운 저항이 적용되지 않은 버튼 회로



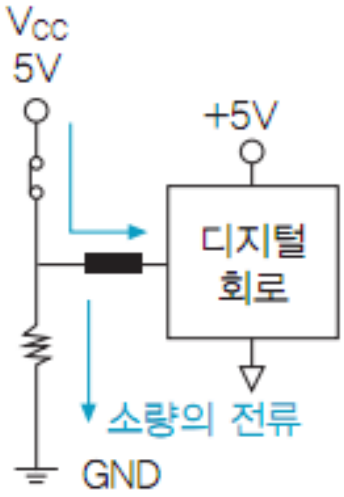
(a) 풀다운 저항 사용 안 함



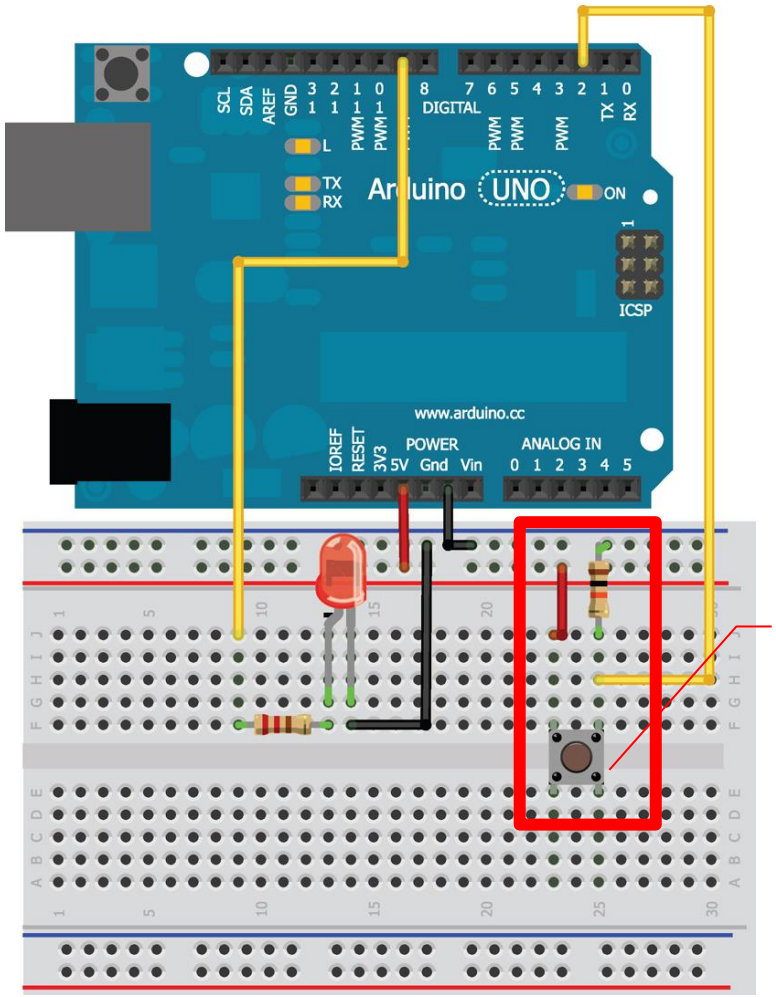
- 아두이노 5V와 GND는 절대 저항없이 연결X

디지털 입력값을 읽는 방법 – 소프트웨어적으로 버튼-LED 제어

- 풀다운 저항이 적용된 버튼 회로



(b) 풀다운 저항 사용
– 버튼 누름(ON)



4개의 핀 모두
간섭없이 연결

디지털 입력값을 읽는 방법 – 소프트웨어적으로 버튼-LED 제어

```
1   const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2   const int BUTTON=2; //버튼을 2번 핀에 지정
3
4   void setup()
5   {
6       pinMode(LED, OUTPUT);
7       //LED(9번 핀)를 출력으로 지정
8       pinMode(BUTTON, INPUT);
9       //BUTTON(2번 핀)을 입력으로 지정
10  }
11
12  void loop()
13  {
14      if(digitalRead(BUTTON)==LOW)
15      {
16          digitalWrite(LED, LOW);
17      }
18      else
19      {
20          digitalWrite(LED, HIGH);
21      }
22  }
```

- If문으로 버튼 값을 검사하여 LED를 작동시킨다.
  - 버튼이 안 눌리면

    (digitalRead(BUTTON)==LOW)

    LED를 끄고,

    digitalWrite(LED, LOW);
  - 그렇지 않으면(버튼이 눌리면)

    LED를 켠다.

    digitalWrite(LED, HIGH);

디지털 입력값을 읽는 방법 – 소프트웨어적으로 버튼-LED 제어

- 일반 버튼 동작 (모멘터리, 순간식)
  - 버튼 누르면 LED 켜짐
  - 버튼 떼면 LED 꺼짐

→ 버튼을 계속 누르고 있어야 LED가 켜져 있음

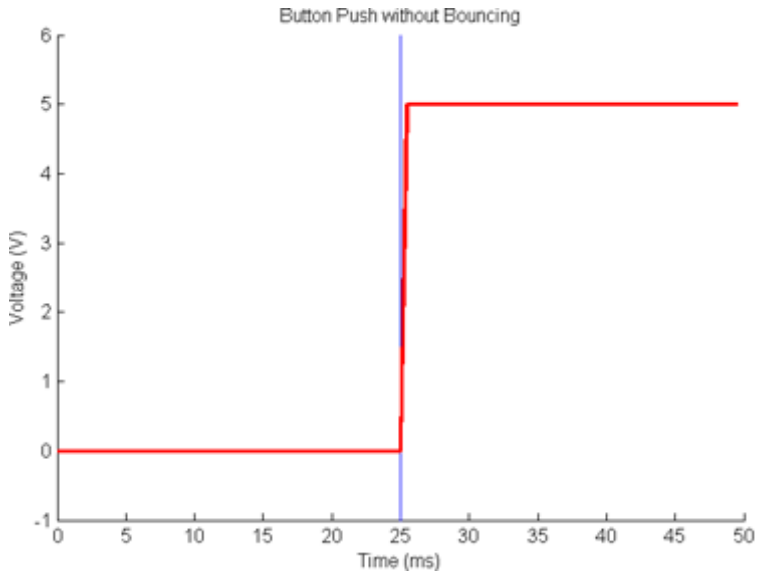
- 토글 동작
  - 버튼 한 번 누르면 LED 계속 켜짐 (버튼 떼도 유지)
  - 다시 버튼 누르면 LED 꺼짐
  - 또 누르면 다시 켜짐 …

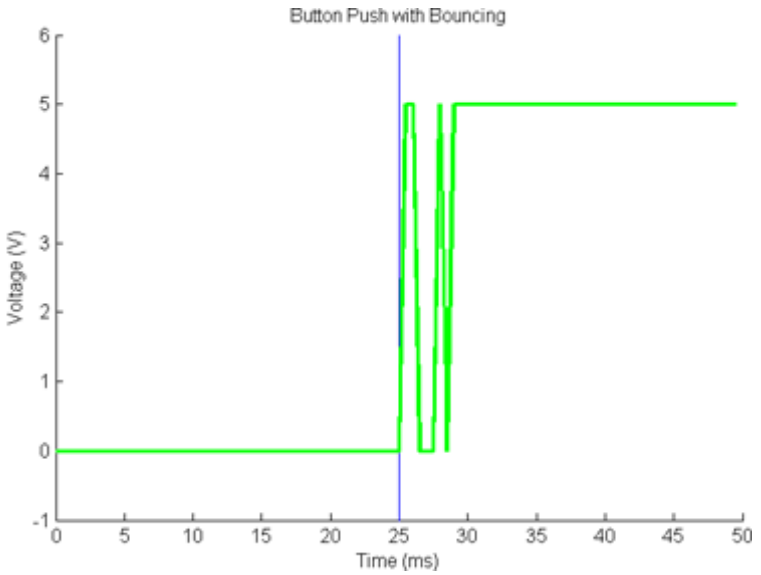
→ 버튼을 “전원 스위치”처럼 쓰는 방식

스위치 바운싱(채터링 또는 진동 잡음)

스위치 바운싱(채터링 또는 진동 잡음)

- 스위치 바운싱은 한 번 누른 동작이 여러 번 신호로 인식되는 현상
- 버튼의 기계적인 현상
- 값이 변경되는 것을 알아내기 어렵다.
- 실제로 버튼을 눌러도 예상대로 작동하지 않는 경우가 많다.





소프트웨어적 해결방법

소프트웨어로 스위치 바운싱 처리

- 버튼의 스위치 바운싱 해결 방법
- 디바운싱이라고도 부른다.

1. 이전 버튼 상태와 현재 버튼 상태를 저장한다.
2. 현재 버튼 상태를 읽어 온다.
3. 입력값이 이전 버튼 상태와 다를 때는 바운싱이 끝날 때 까지 5ms 동안 기다린다.
4. 5ms 후에 버튼 상태를 다시 읽어서 그 값을 현재 버튼 상태로 저장한다.
5. 이전 버튼 상태가 LOW이고 현재 버튼 상태가 HIGH라면 LED 상태를 전환한다.
6. 이전 버튼 상태의 변수값에 현재 버튼 상태를 입력한다.
7. 단계 2를 반복한다.

디지털 입력값을 읽는 방법 – 스위치 바운싱 처리

```
1  const int LED=9; //9번 핀을 사용하는 LED 상수 정의
2  const int BUTTON=2; //버튼을 2번 핀에 지정
3
4  boolean lastButton=LOW; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
5  boolean currentButton=LOW; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
6  boolean ledOn=false; //현재 LED의 점멸 상태를 Boolean형 변수로 선언
7
8  void setup()
9  {
10     pinMode(LED, OUTPUT); //LED(9번 핀)를 출력으로 지정
11     pinMode(BUTTON, INPUT); //BUTTON(2번 핀)을 입력으로 지정
12 }
13
14 /*
15 * 스위치 디바운싱 함수
16 * debounce() 함수에 이전 버튼 상태를 인수로 사용하여
17 * 디바운싱 처리된 현재 버튼 상태값을 반환
18 */
19
20 boolean debounce(boolean last)
21 {
22     boolean current=digitalRead(BUTTON); //현재 버튼 상태를 확인
23     if(last!=current) //이전 버튼 상태와 현재 버튼 상태가 다름 //버튼을 누른 상태임
24     {
25         delay(5); //5ms 동안 기다림
26         current=digitalRead(BUTTON); //current에 현재 버튼 상태를 저장
27     }
28     return current; //current 변수값을 반환
29 }
```

디지털 입력 값을 읽는 방법 – 스위치 바운싱 처리

```
32  void loop()
33  {
34      currentButton=debounce(lastButton); //디바운싱 처리된 버튼값을 읽음
35      if(lastButton==LOW&&currentButton==HIGH) //버튼을 누름
36      {
37          ledOn=!ledOn; //LED 점멸 상태를 나타내는 ledOn 변수값을 바꿈
38      }
39      lastButton=currentButton; //이전 버튼값을 현재 버튼값으로 변경
40
41      digitalWrite(LED, ledOn); //LED 점멸 상태를 바꿈
42  }
```

02 아날로그 입력

가변저항 사용

- 가변저항? 전류의 세기를 사용자 입력으로 변하는 저항

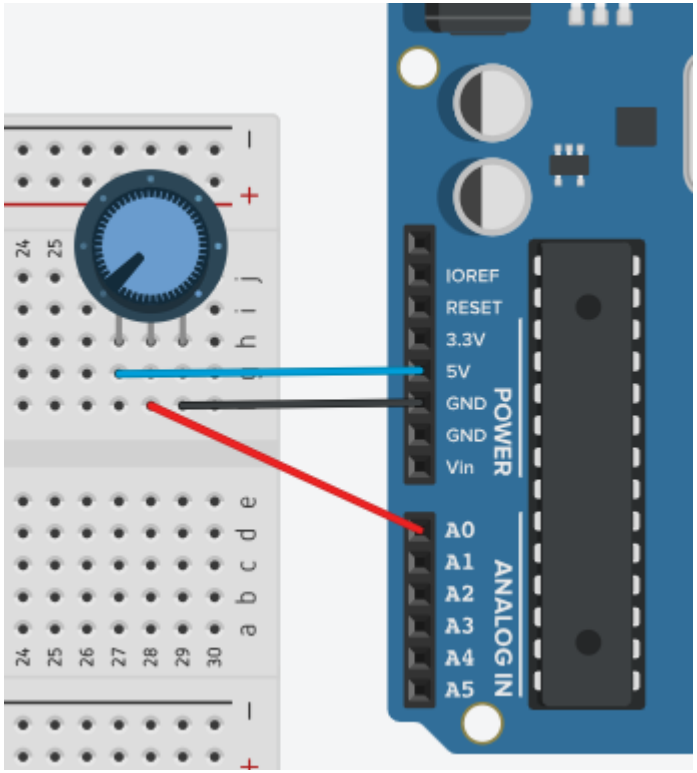
  ex)오디오의 볼륨을 제어하거나 LED 조명의 밝기를 제어
- 양 끝의 두 단자: 전원(5V)과 그라운드(GND)에 연결
- 가운데 단자: 와이퍼(Wiper)라고하여 가변저항의 변경된 저항 값을 읽을 수 있는 단자
- 저항은 극성이 없는 전자소자이기 때문에 전원과 그라운드는 서로 바꿔서 연결해도 무방

		
가변저항 구성요소	전원, 그라운드 순서로 연결	그라운드, 전원 순서로 연결

uno_ex6_3: 아날로그 값 읽기



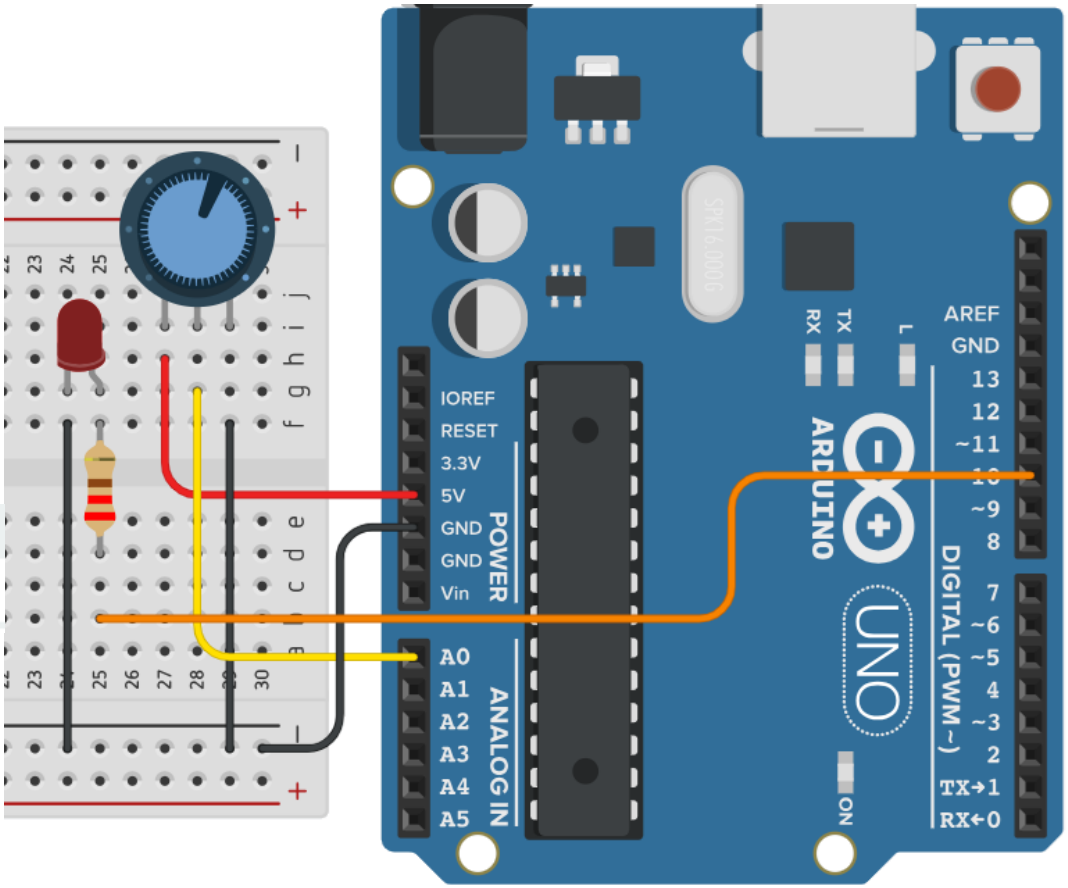
가변저항 구성요소 | 전원, 그라운드 순서로 연결 | 그라운드, 전원 순서로 연결




```
1  const int INPUT_PIN = A0;
2
3  int input_value;
4
5  void setup() {
6    pinMode(INPUT_PIN, INPUT);
7    Serial.begin(9600);
8  }
9
10 void loop() {
11
12   input_value = analogRead(INPUT_PIN);//0~1023
13
14
15   Serial.println("INPUT="+(String)input_value);
16   delay(500);
17 }
```

uno_ex6_4: LED 밝기 조절

모듈명	모듈 PIN	아두이노 PIN
가변저항	VCC(1)	5V
	GND(3)	GND
	INPUT(2)	A0
LED	양극	10
	음극	GND



출력 시리얼 모니터 ×

Message (Enter to send message to 'Arduino Uno' on 'COM3')

```
INPUT=1003      OUTPUT=250
INPUT=1003      OUTPUT=250
INPUT=143       OUTPUT=35
INPUT=143       OUTPUT=35
INPUT=246       OUTPUT=61
INPUT=348       OUTPUT=87
INPUT=430       OUTPUT=107
```

uno_ex6_4: LED 밝기 조절

uno6_4.ino

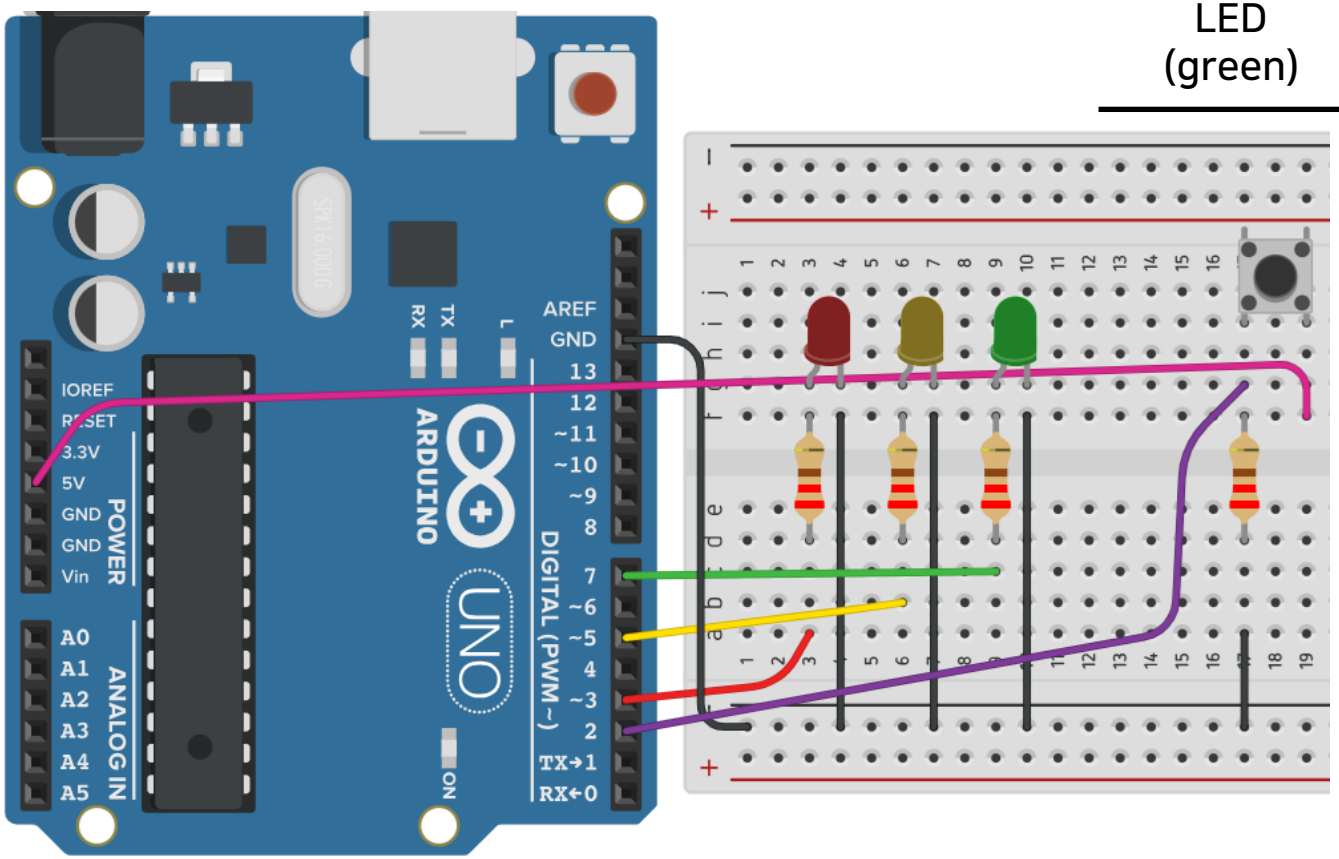
```
1   const int INPUT_PIN = A0;
2   const int OUTPUT_PIN=10;
3   int input_value;
4   int output_value;
5
6
7   void setup() {
8     pinMode(INPUT_PIN, INPUT); //0~1023
9     pinMode(OUTPUT_PIN, OUTPUT); //0~255
10    Serial.begin(9600);
11  }
12
13  void loop() {
14
15    input_value = analogRead(INPUT_PIN);
16    output_value =  input_value/4;
17
18    Serial.println("INPUT="+(String)input_value+"\t OUTPUT="+(String)output_value);
19    analogWrite(OUTPUT_PIN, output_value);
20
21    delay(500);
22  }
```

입력 : 0 ~ 1023 범위
출력 : 0 ~ 255 값

03 다중 연결

입력모듈 1개 : 출력모듈 N

모듈명	모듈 PIN	아두이노 PIN
스위치	VCC	5V
	GND	GND
	INPUT	2
LED (red)	양극	3
	음극	GND
LED (yellow)	양극	5
	음극	GND
LED (green)	양극	7
	음극	GND



입력모듈 1개 : 출력모듈 N

uno6_5.ino

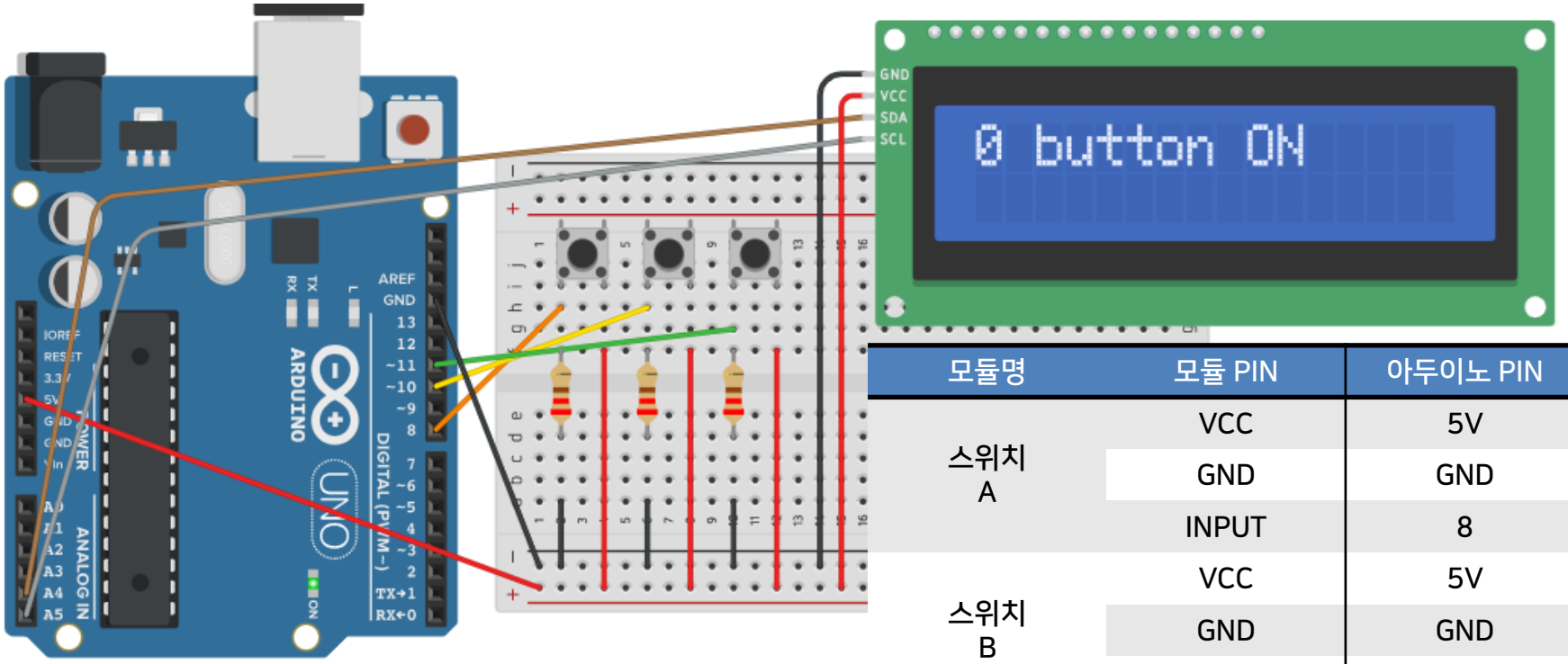
```
1   const int INPUT_PIN=2; //버튼을 2번 핀에 지정
2   const int OUTPUT_PIN[]={3,5,7}; //3,5,7번 핀을 사용하는 LED 상수 정의
3
4   boolean lastButton=LOW; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
5   boolean currentButton=LOW; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
6   int count=0;
7   void setup()
8   {
9       pinMode(INPUT_PIN, INPUT);
10      for(int i=0; i<3; i++)pinMode(OUTPUT_PIN[i], OUTPUT);
11      Serial.begin(9600);
12  }
13
14  boolean debounce(boolean last)
15  {
16      boolean current=digitalRead(INPUT_PIN); //현재 버튼 상태를 확인
17      if(last!=current) //이전 버튼 상태와 현재 버튼 상태가 다름
18      {
19          delay(5); //5ms 동안 기다림
20          current=digitalRead(INPUT_PIN); //current에 현재 버튼 상태를 저장
21      }
22      return current; //current 변수값을 반환
23  }
```

입력모듈 1개 : 출력모듈 N

```
25  void loop()
26  {
27      currentButton=debounce(lastButton); //디바운싱 처리된 버튼값을 읽음
28      if(lastButton==LOW&&currentButton==HIGH) //버튼을 누름
29      {    //순간적으로 버튼이 안 눌리다가(LOW → HIGH 변화가 생길 때)" 버튼을 눌렀다고 인식
30          for(int i=0; i<3; i++)digitalWrite(OUTPUT_PIN[i], 0);   //모든 LED 끄기
31          if(count==2)count=0;
32          else count++;
33          Serial.println(count);
34      }
35      lastButton=currentButton; //이전 버튼값을 현재 버튼값으로 변경
36                                //lastButton을 현재 상태로 변경
37      digitalWrite(OUTPUT_PIN[count], 1); //LED 점멸 상태를 바꿈
38  }
39
```

버튼을 누를 때마다 **LED**가 **0 → 1 → 2 →** 다시 **0** 순으로 켜짐.
Serial.println(count);로 현재 **LED** 인덱스가 시리얼 모니터에 출력됨.

입력모듈 N개 : 출력모듈 1개



모듈명	모듈 PIN	아두이노 PIN
스위치 A	VCC	5V
	GND	GND
	INPUT	8
스위치 B	VCC	5V
	GND	GND
	INPUT	10
스위치 C	VCC	5V
	GND	GND
	INPUT	11
LCD	GND	GND
	VCC	5V
	SDA	A4
	SCL	A5

입력모듈 N개 : 출력모듈 1개

```
#include <LiquidCrystal_I2C.h>

//initialize lcd
LiquidCrystal_I2C lcd(0x20,16,2);

//스위치
const int BTN[]={8,10,11};

boolean lastButton[]={0,0,0}; //이전 버튼의 눌림 상태를 Boolean형 변수로 선언
boolean currentButton[]={0,0,0}; //현재 버튼의 눌림 상태를 Boolean형 변수로 선언
int choice=0;

void debounce()
{

    for(int i=0; i<3; i++) {
      currentButton[i] = digitalRead(BTN[i]);

      if(lastButton[i]!=currentButton[i]) //이전 버튼 상태와 현재 버튼 상태가 다름
      {
          delay(5); //5ms 동안 기다림
          currentButton[i] = digitalRead(BTN[i]);
      }
    }
}

```

0X27로 변경

입력모듈 N개 : 출력모듈 1개

```
27  void setup()
28  {
29    //lcd.begin(16,2);
30    lcd.init();
31    lcd.clear();
32    lcd.backlight();
33
34    //버튼 초기화
35    for(int i=0; i<3; i++)
36      pinMode(BTN[i], INPUT);
37
38  }
39
40  void loop()
41  {
42    debounce(); //디바운싱 처리된 버튼값을 읽음
43      for(int i=0; i<3; i++){
44        if(lastButton[i]==LOW&&currentButton[i]==HIGH) //버튼을 누름
45        {
46          lcd.clear();
47          lcd.print((String)i+" button ON");
48
49          delay(300);
50        }
51        lastButton[i]=currentButton[i]; //이전 버튼값을 현재 버튼값으로 변화
52      }
53  }
```

요약

- 디지털 입력
- 아날로그 입력
- 다중 연결 (입력모듈:출력모듈)
  - 1:N
  - N:1

목차

1. 조도 센서
2. 초음파 센서
3. 온도 센서

01 조도센서

저항에 의한 전압 분배 회로

아날로그 센서

- 물리량 변화에 따라 저항 성질이 바뀌는 재료를 활용하여 센서 제작
- 가변저항과 같은 방식으로 전원을 가하여 저항값을 알아내고 물리량 파악
- 저항 성질이 바뀌는 재료를 사용하는 센서
  - 힘 센서, 구부림 센서 : 누르거나 구부릴 때 저항 성질이 변함
  - 조도 센서 : 빛의 양에 따라 저항 성질이 변함
  - 서미스터(thermistor) : 온도에 따라 저항 성질이 변함
  - 초음파 센서 : 반사물에 반사되어 돌아오는 시간을 측정해 거리를 측정

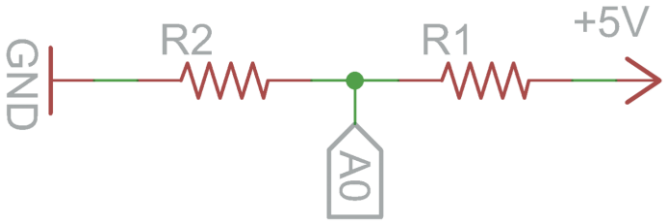
빛에 따라 저항이 변하는 이유

- 조도센서 재질
  - 조도센서는 반도체(대표적으로 CdS: 황화카드뮴)로 만들어져 있음
  - 반도체는 전기가 잘 통할 수도 있고, 잘 안 통할 수도 있음
- 어두울 때
  - 전자가 대부분 제자리에 갇혀 있어서 움직이지 못함
  - 전기가 잘 안 통해서 저항이 커짐.
- 밝을 때(빛을 받을 때)
  - 빛 속에는 에너지를 가진 작은 입자(광자)가 있음.
  - 이 빛이 반도체에 닿으면, 갇혀 있던 전자가 에너지를 받아서 풀려남
  - 풀려난 전자가 움직이며 전류가 흐름 → 전기가 잘 통함 → 저항이 작아짐.

저항에 의한 전압 분배 회로

전압 분배 회로

- 2개의 저항을 사용하여 사용한 저항 크기에 따라 입력 전압 일부만 출력
- 2개 저항 중 하나를 가변저항으로 사용하면 손잡이를 돌려 출력 전압을 연속적으로 변경 가능



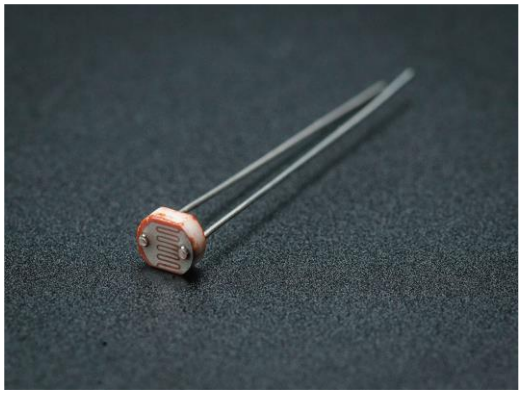
전압 분배 식

- $V_{out} = V_{in}\left(\frac{R2}{R1+R2}\right)$
- R1, R2 저항 크기에 따라 출력 전압(V_{out}) 결정

조도 센서

- 광량에 따라 저항값이 변함
- 완전히 어두울 때 약 200kΩ

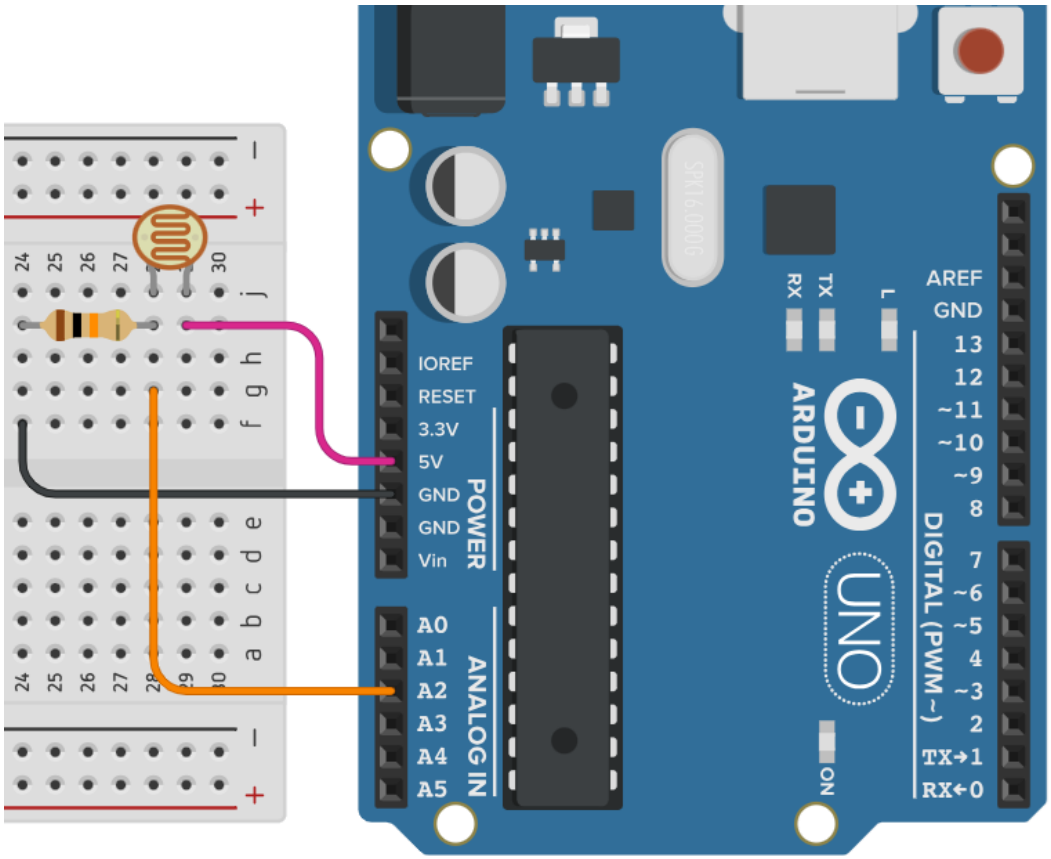
  빛이 가득 찬 상태일 때 약 5kΩ



조도 센서

조도 센서 연결 회로

- 10kΩ 저항 사용(밝기 변화 전체를 고르게 감지하기 좋은 절충값)
- 손으로 조도 센서를 가리거나 플래시로 빛 조절하며 시리얼 모니터 출력 확인



사용 함수

- analogRead
  - 물리량 변화에 따른 센서의 저항값을 디지털 값으로 변환하여 읽기
  - 0~1023 사이 값

- map
  - output = map(value, fromLow, fromHigh, toLow, toHigh)
  - 입력값(value)의 범위(fromLow~fromHigh)를 다른 범위(toLow~toHigh)로 변환 출력

- constrain
  - output = constrain(value, min, max)
  - 입력값(value)이 최소 min 값, 최대 max 값을 가지도록 제한하여 출력

uno09_01.ino : 범위 내 광량 측정

```
1   const int INPUT_PIN = A2;
2   const int MIN_LIGHT = 100;
3   const int MAX_LIGHT = 700;
4   int val = -1;
5
6   void setup() {
7     Serial.begin(9600);
8     pinMode(INPUT_PIN, INPUT);
9   }
10
11  void loop() {
12
13    val = analogRead(INPUT_PIN);
14    val = map(val, MIN_LIGHT, MAX_LIGHT, 0, 255);
15    val = constrain(val, 0, 255);
16    Serial.println(val);
17    delay(500);
18
19  }
```

analogRead(INPUT_PIN)

•조도센서에서 들어오는 전압(0~5V)을 10비트 값으로 읽음

•즉, 0~1023 범위의 숫자가 들어옴

- 0 → 0V
- 1023 → 5V

map(val, MIN_LIGHT, MAX_LIGHT, 0, 255)

•map() 함수는 **입력값 범위를 다른 범위로 변환**

•여기서는 조도센서의 실제 동작 범위가 100~700 정도라서 그 구간을 **0~255**로 바꾸는 역할을 함

예를 들어,

•val = 100 (어두움) → 0

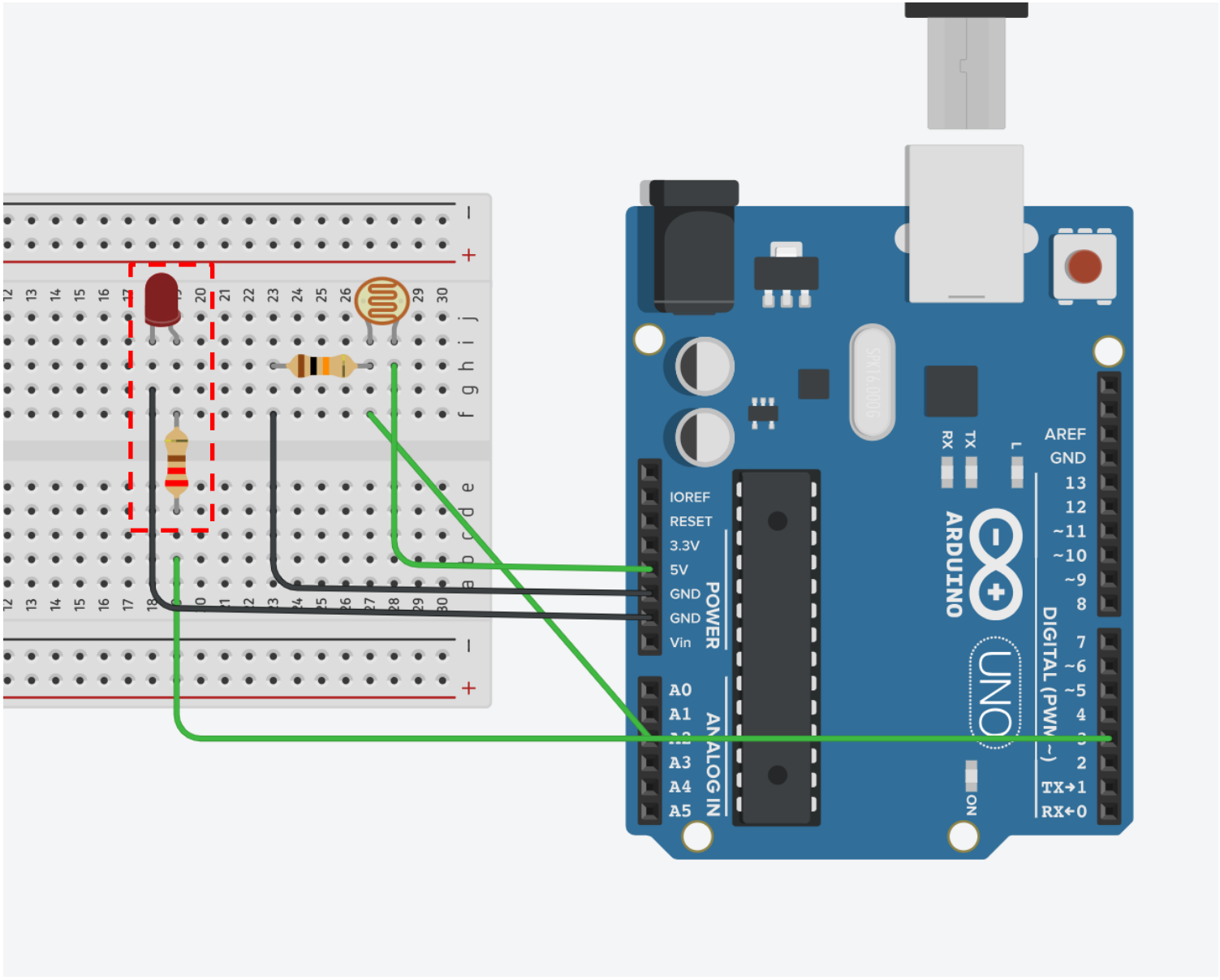
•val = 700 (밝음) → 255

constrain(val, 0, 255)

• map 계산 과정에서 값이 0보다 작거나 255보다 큰 데이터가 들어올 수 있음. 그 경우 강제로 0~255 범위 안으로 조정함

- map() 함수는 입력값의 범위를 다른 범위로 변환하는 역할을 하며, constrain() 함수는 변환된 값이 지정된 범위를 벗어나지 않도록 제한하는 역할을 한다. 따라서 map()으로 100~700의 값을 0~255로 변환한 후, constrain()을 사용하여 최종값을 0~255 범위로 제한한다.

광량을 측정하여 LED 켜기 & 끄기 - 회로



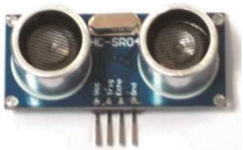
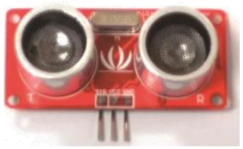
uno09_02.uno : 광량을 측정하여 LED 켜기 & 끄기

```
1   const int INPUT_PIN = A2;
2   const int OUTPUT_PIN = 3;
3
4   const int MIN_LIGHT = 100;
5   const int MAX_LIGHT = 800;
6   int val = -1;
7
8   void setup() {
9     Serial.begin(9600);
10    pinMode(INPUT_PIN, INPUT);
11    pinMode(OUTPUT_PIN, OUTPUT);
12
13  }
14
15  void loop() {
16
17    val = analogRead(INPUT_PIN);
18    val = map(val, MIN_LIGHT, MAX_LIGHT, 0, 255);
19    val = constrain(val, 0, 255);
20
21    Serial.println(val);
22
23    if(val<50) digitalWrite(OUTPUT_PIN, 1);
24    else digitalWrite(OUTPUT_PIN, 0);
25
26    delay(500);
27
28  }
```

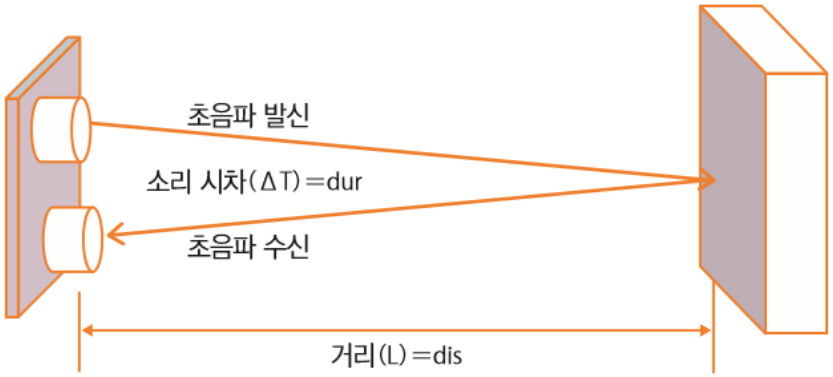
02 초음파 센서

초음파 센서

- 초음파 센서는 사람의 귀로는 들을 수 없는 고주파 소리를 출력한 후, 반사물에 반사되어 돌아오는 시간을 측정해 거리를 측정
- 사람, 동물, 물건 등이 다가오고 멀어지는 것을 감지하거나, 물이 불어나는 정도를 측정하거나, 전장까지의 높이를 측정하는 등 다양하게 활용
- 측정할 수 있는 거리의 범위는 수 센티미터에서 3~4m까지

제품 이름	HC-SR04	SEN136B5B
핀 수	4	3
핀의 의미 (정면 왼쪽부터)	Vcc: 5V Trig: 송신 쪽 핀 Echo: 수신 쪽 핀 Gnd: 접지	SIG: 송수신 핀 VCC: 5V GND: 접지
측정 거리 범위	2cm~4m	3cm~4m
제품		

초음파 센서 거리 측정



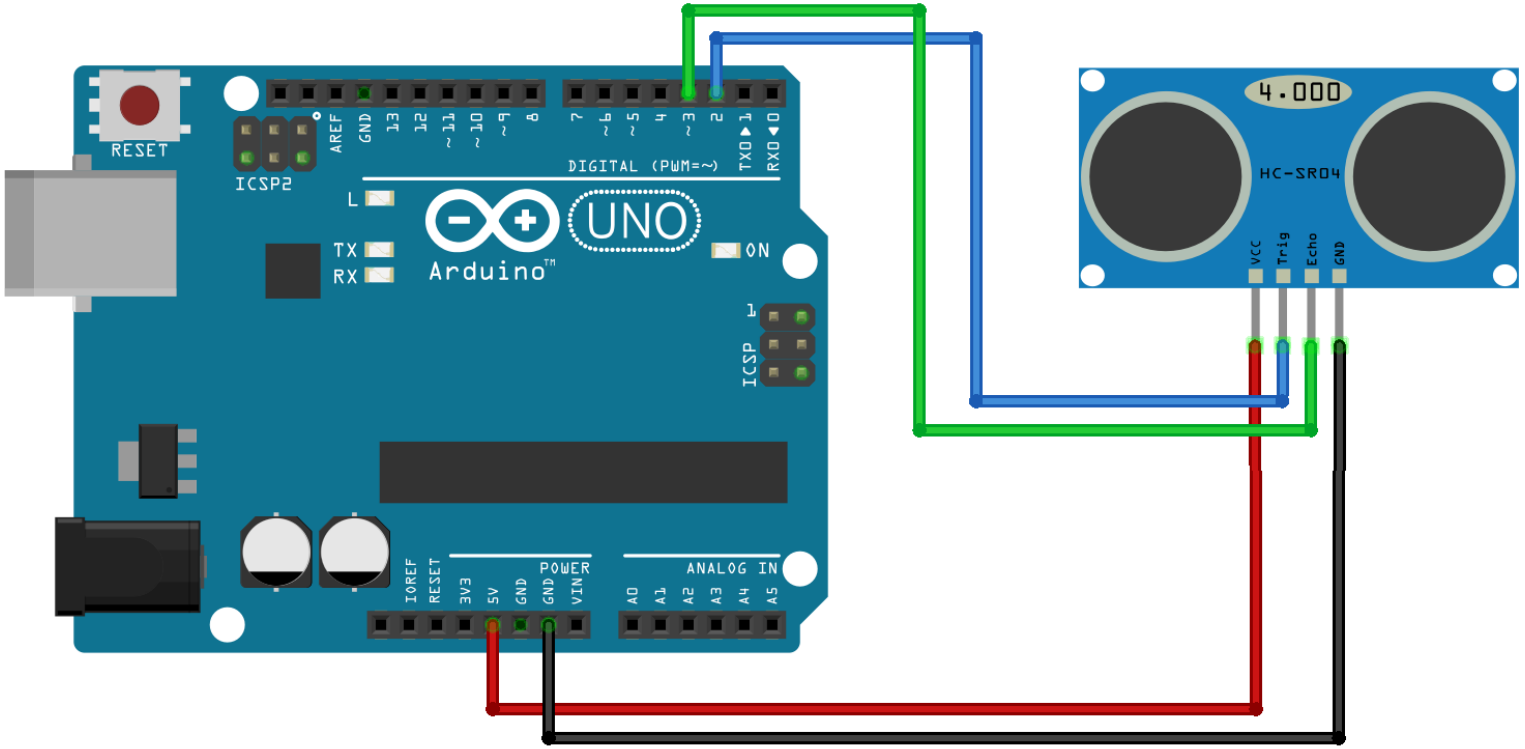
- 장애물까지의 거리(L), 장애물에 반사된 초음파가 돌아오는 데 걸리는 시간(T), 음속(C) 사이의 관계를 다음과 같은 식으로 정리할 수 있음
- 초음파 거리 센서는 디지털 신호 HIGH와 LOW의 전환을 통해 송수신기(발신 쪽과 수신 쪽)에서 시간 차를 읽을 수 있음

초음파 거리 센서에 사용하는 식

L = C × T / 2
L은 장애물까지의 거리
C는 음속(간이식은 331 + 0.6 × t: 단위 m/s) : t는 기온
T는 초음파를 발신한 직후부터 수신하기까지 걸리는 시간

초음파 센서 회로

- 5V → 5V (VCC)
- Trig → D2
- Echo → D3
- GND → GND



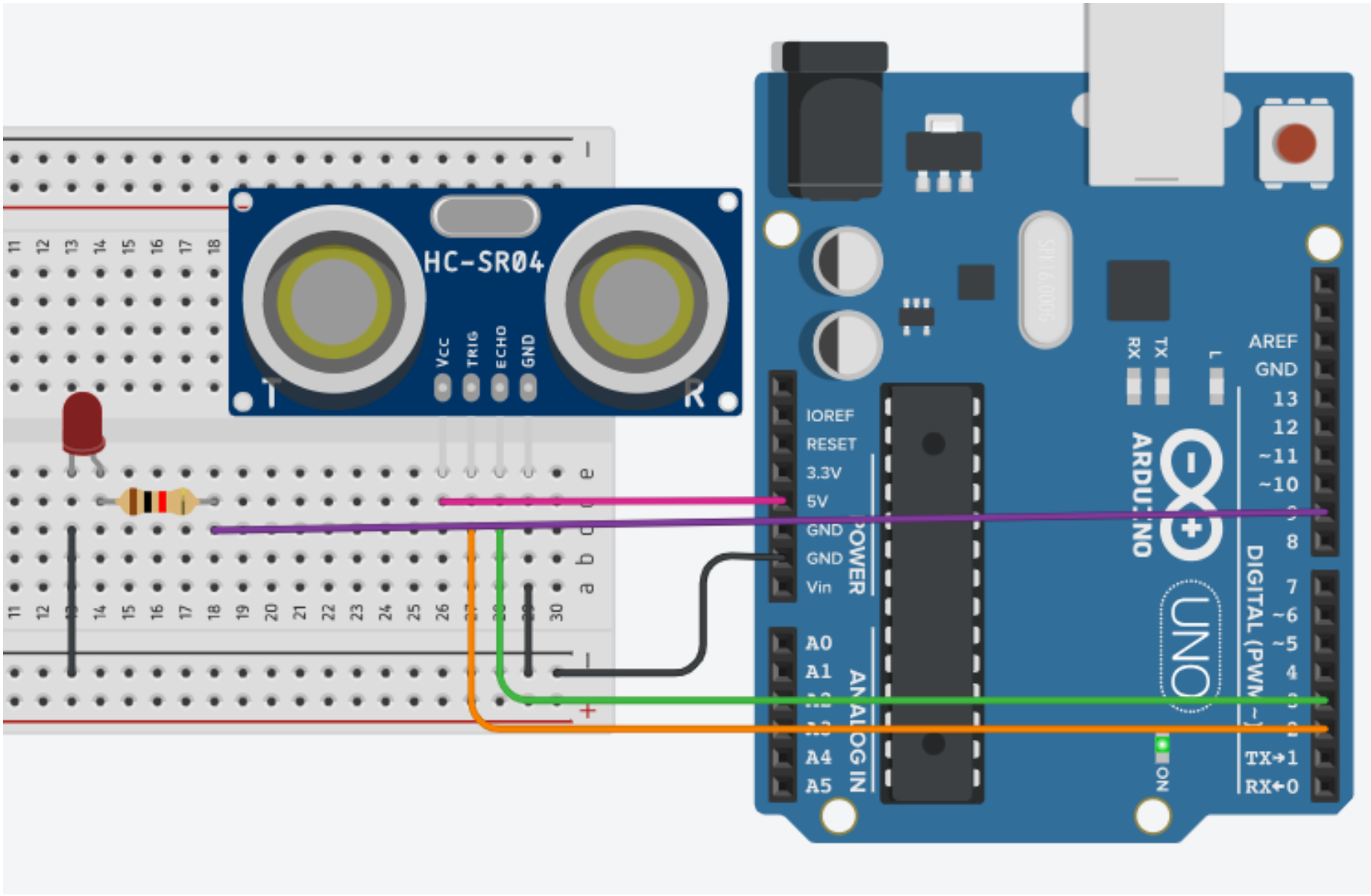
uno09_03.ino : 거리 측정

```
1   #define TRIGPIN 2       // 트리거(송신쪽) 핀     // 트리거 핀 (초음파 신호를 내보내는 쪽)
2   #define ECHOPIN 3       // 에코(수신쪽) 핀       // 에코 핀 (반사파를 받는 쪽)
3   #define CTM 10          // HIGH인 시간(μ 초)  // 트리거 신호를 HIGH로 유지할 시간 (μs)
4
5   int dur;              // 시간 차(μ초)
6   float dis;            // 거리(cm)
7
8   void setup() {
9     Serial.begin(9600);
10    pinMode(TRIGPIN, OUTPUT);     // 트리거 핀을 디지털 출력으로 설정
11    pinMode(ECHOPIN, INPUT);      // 에코 핀을 디지털 입력으로 설정
12  }
13  void loop() {
14
15
16    digitalWrite(TRIGPIN, HIGH);      // 트리거 핀을 HIGH로 10μs 동안 유지 →
17    delayMicroseconds(CTM);           초음파를 '딱 한 번' 발사
18    digitalWrite(TRIGPIN, LOW);
19    dur = pulseIn(ECHOPIN, HIGH);       // HIGH가 되기까지 걸리는 시간을 측정
20    dis = (float) dur * 0.017;          // 음속을 사용해서 거리 계산
21    Serial.print(dis);
22    Serial.println(" cm");
23    delay(500);
24
25  }
```

// 3. 거리 계산 (단위: cm)
// 음속 약 340m/s → 1μs 동안 약 0.034cm 이동
// 왕복 거리이므로 0.034 / 2 = 0.017

- 아두이노가 **Trig에 10μs 정도 HIGH 신호**를 보냄.
- 센서가 **40kHz 초음파를 발사함.**
- **초음파를 발사하는 순간부터 Echo가 HIGH가 됨.**
- 초음파가 물체에 반사되어 센서로 돌아옴.
- **돌아오는 순간 Echo가 LOW가 됨.**

uno09_04.ino : 사람 인식하면 LED 켜기



uno09_04.ino
: 사람 인식하면 LED 켜기

```
#define TRIGPIN 2       // 트리거(송신쪽) 핀
#define ECHOPIN 3       // 에코(수신쪽) 핀
#define LEDPIN 9        // LED 핀

#define CTM 10          // HIGH인 시간(μ 초)

int dur;              // 시간 차(μ초)
float dis;            // 거리(cm)

void setup() {
  Serial.begin(9600);
  pinMode(TRIGPIN, OUTPUT);     // 트리거 핀을 디지털 출력으로 설정
  pinMode(ECHOPIN, INPUT);      // 에코 핀을 디지털 입력으로 설정
  pinMode(LEDPIN, OUTPUT);     // LED 핀을 디지털 출력으로 설정

}
void loop() {


  digitalWrite(TRIGPIN, HIGH);
  delayMicroseconds(CTM);
  digitalWrite(TRIGPIN, LOW);
  dur = pulseIn(ECHOPIN, HIGH);      // HIGH가 되기까지 걸리는 시간을 측정
  dis = (float) dur * 0.017;         // 음속을 사용해서 거리 계산
  Serial.print(dis);
  Serial.println(" cm");


  if(dis<50) { //장애물이 가까이오면
    digitalWrite(LEDPIN, HIGH); //LED 5초간 켜기
    delay(5000);
    digitalWrite(LEDPIN, LOW);
  }

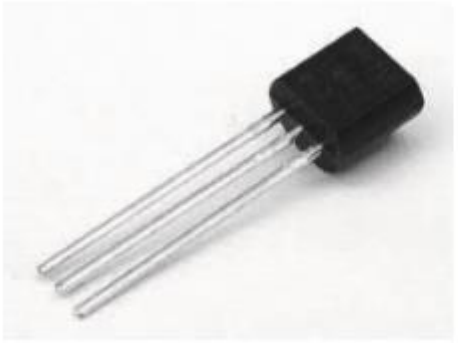
  delay(500);

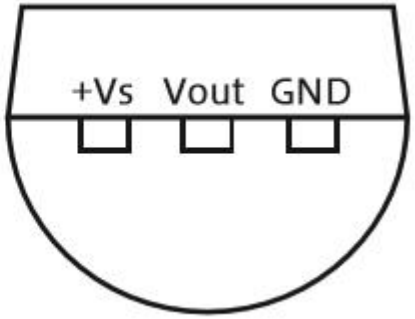
}
```

03 온도센서

온도센서

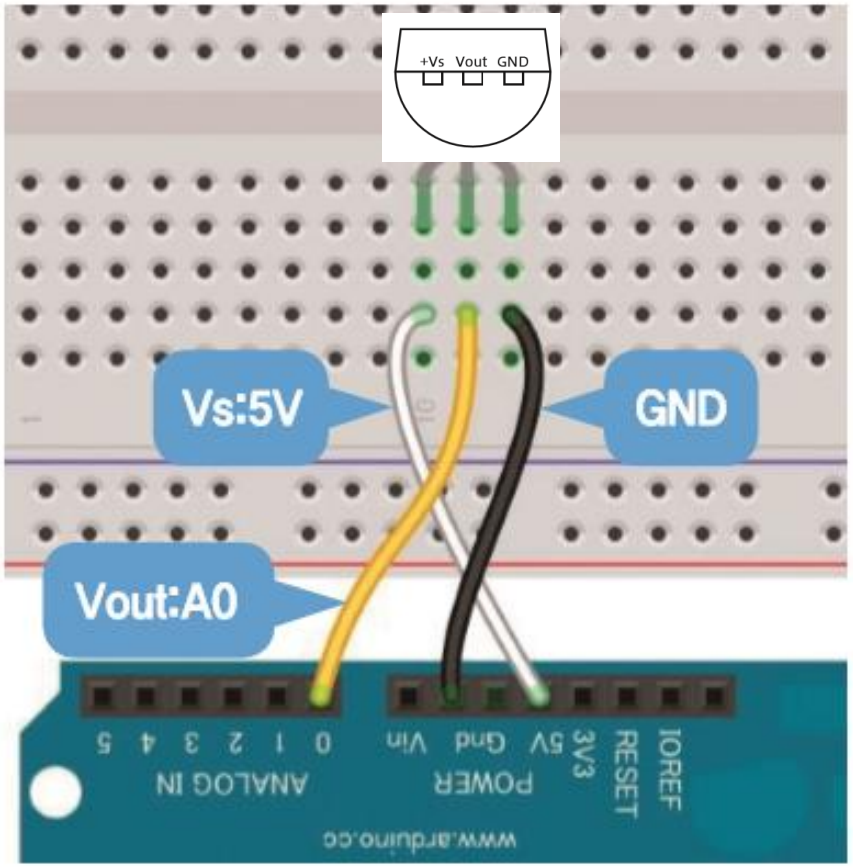
- 온도 센서는 바깥 온도 변화에 따라 저항 값이 바뀌는 아날로그 센서
- 이러한 특성을 통해 섭씨온도(단위: ℃)나 화씨온도(단위: °F)로 변환 계산을 함
- 핀이 3개 : 양쪽 끝 핀이 전원 전압(+Vs)과 접지(GND,그라운드)이고, 가운데 핀이 출력 전압(Vout)





회로

- 온도 값이 출력되는 Vout 핀은 아날로그 입력 포트 A0 핀에 연결
- 반드시 주의하여 연결! 쇼트남



출력전압을 통한 온도계산

•아두이노의 analogRead() 함수는 **0~1023** 사이의 정수 값을 반환함
•0 → 0V
•1023 → 기준 전압 Vs(보통 5V)
•즉, 아날로그 입력을 **디지털로 변환(ADC)** 한 값임.

- 스케치를 작성하기 전에 먼저 온도 센서의 출력 사양을 확인
- 사양을 보면 온도 센서의 출력 전압(Vout) 의 관계식은 아래와 같음

```
V = Vout * Vs / 1024
```

- 이 식에서 Vs는 전원 전압
- 아두이노 우노의 전원은 5V이므로 다음과 같이 바꿔 쓸 수 있음

```
V = Vout * 5 / 1024
```

- 출력 전압을 통한 온도계산(섭씨기준, C)
  - 온도센서의 종류에 따라 다름

* 차이점은 온도가 0도일때 TMP36은 0.5V를 출력하지만 LM35의 경우에는 0V를 출력한다.

  - TMP36

```
C = (V-0.5) / 0.01;
```

  - LM35

```
C = V / 0.01;
```

출력전압을 통한 온도계산

<TMP36 온도센서>

- TMP36센서는 출력 전압이 온도에 비례함
- 주요 특징:
  - 0°C일 때 출력 전압: 0.5 V
  - 온도 1°C 상승 → 출력 전압 10 mV 증가
    (즉, 1°C → 0.01 V)

- 공식 의미
- $C = \frac{V-0.5}{0.01}$
  - V: 센서에서 측정한 전압 (단위: V)
  - 0.5를 빼는 이유: 센서가 0°C일 때 0.5 V를 출력하므로, 기준점을 0°C로 맞추기 위해 0.5를 뺌
  - 0.01로 나누는 이유: 전압 0.01 V(10 mV)가 온도 1°C에 해당하기 때문

uno10_1.uno

ADC는 **Analog-to-Digital Converter**의 약자임
즉, **아날로그 신호를 디지털 신호로 변환하는 장치**를 의미함

•**아날로그 신호**: 연속적인 값, 예를 들어 온도 센서에서 나오는 0~5V 전압
•**디지털 신호**: 컴퓨터가 처리할 수 있는 0과 1로 된 값, 예를 들어 0~1023 범위의 정수

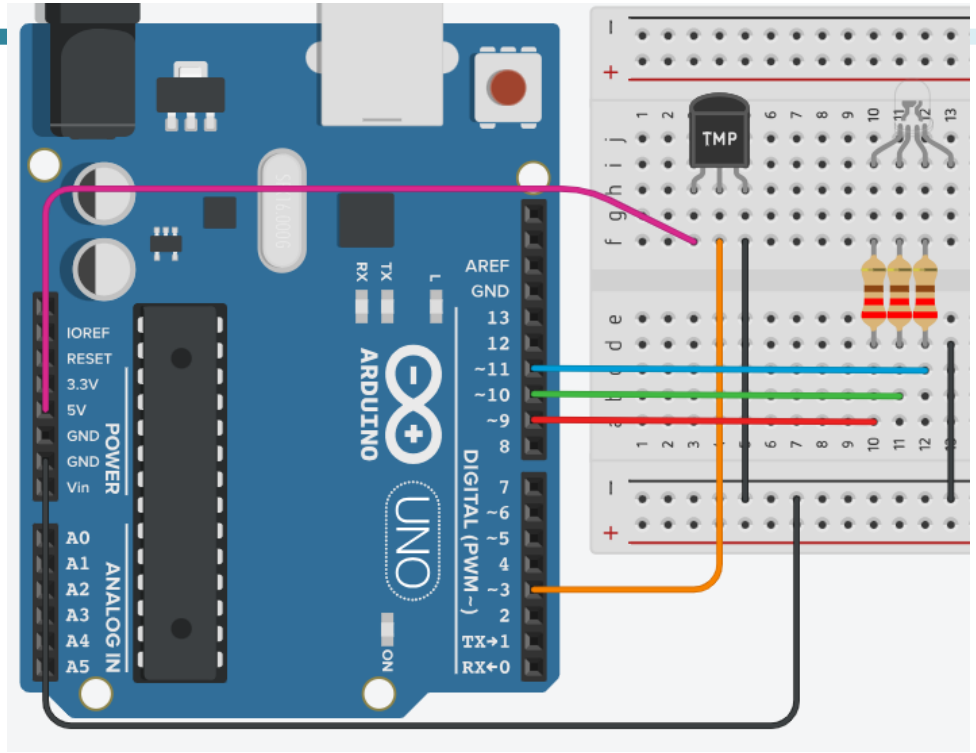
그래서 아두이노에서 `analogRead()`를 쓰면 ADC를 통해 전압을 디지털 값으로 변환해서 읽음

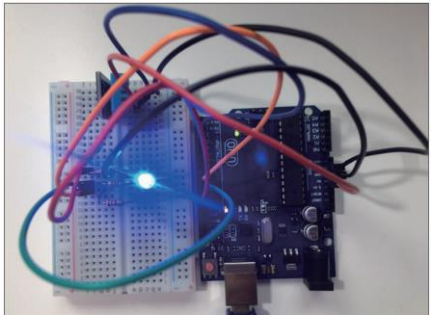
```
1  int tmp_sensor = A0;
2
3  int Vout;
4  float V;
5  float tempC,tempF;
6
7  void setup(){
8    Serial.begin(9600);
9    pinMode(tmp_sensor, INPUT);
10 }
11
12 void loop(){
13
14   Vout = analogRead(tmp_sensor);    // 1. 아날로그 값 읽기 (0~1023)
15
16   V = Vout * 5.0 / 1024.0;          // 2. ADC 값 → 전압 변환
17
18   tempC = (V - 0.5) / 0.01;//TMP36
19   //tempC = V / 0.01;//LM35
20
21   tempF = tempC *1.8 +32; //화씨
22
23   Serial.print("value: ");
24   Serial.print(Vout);
25
26   Serial.print("/voltage:");
27   Serial.print(V);
28
29   Serial.print("/Temperature:");
30   Serial.print(tempC);
31   Serial.print("C, ");
32   Serial.print(tempF);
33   Serial.println("F");
34   delay(1000);
35
36 }
```

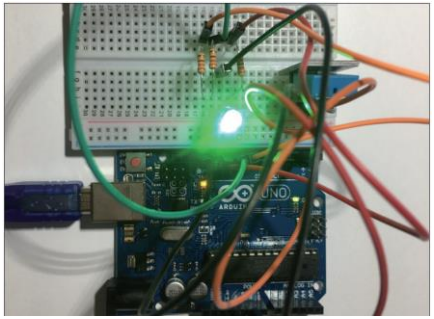
온도에 따른 LED 변화

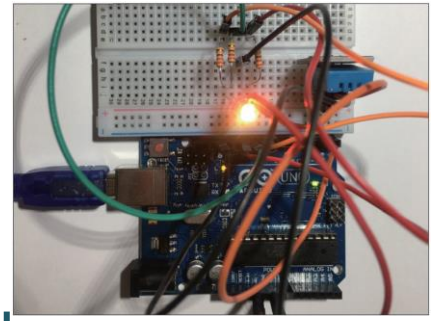
- ~10℃ -> BLUE
- 10~17℃ -> GREEN
- 17~23℃ -> ORANGE
- 23℃~ -> RED

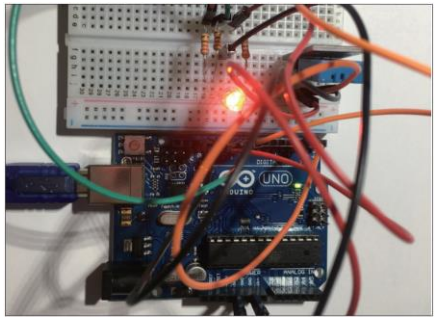
- Pin: A0 -> 9,10,11











10℃ 17℃ 23℃

uno10_2.uno

```
int tmp_sensor = A0;

int Vout;
float V;

float tempC;

void setup(){

  Serial.begin(9600);
  pinMode(tmp_sensor, INPUT);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}

void allOFF(){
    analogWrite(9,0);
    analogWrite(10,0);
    analogWrite(11,0);
}

void loop(){

  Vout = analogRead(tmp_sensor);
  V = Vout * 5.0 / 1024.0;
  tempC = (V - 0.5) / 0.01;//TMP36
  //tempC = V / 0.01;//LM35
  Serial.println(tempC);
  if(tempC>=23) {//RED
    allOFF();
    analogWrite(9,255);
    analogWrite(10,0);
    analogWrite(11,0);
  }
  else if(tempC>=17) {//ORANGE
    allOFF();
    analogWrite(9,255);
    analogWrite(10,128);
    analogWrite(11,0);
  }
  else if(tempC>=10) {//GREEN
    allOFF();
    analogWrite(9,0);
    analogWrite(10,255);
    analogWrite(11,0);
  }
  else {//BLUE
    allOFF();
    analogWrite(9,0);
    analogWrite(10,0);
    analogWrite(11,255);
  }
}
```

목차

1. 화재감지 센서
2. 자이로 센서
3. 스테핑 모터
4. L298D DC 모터

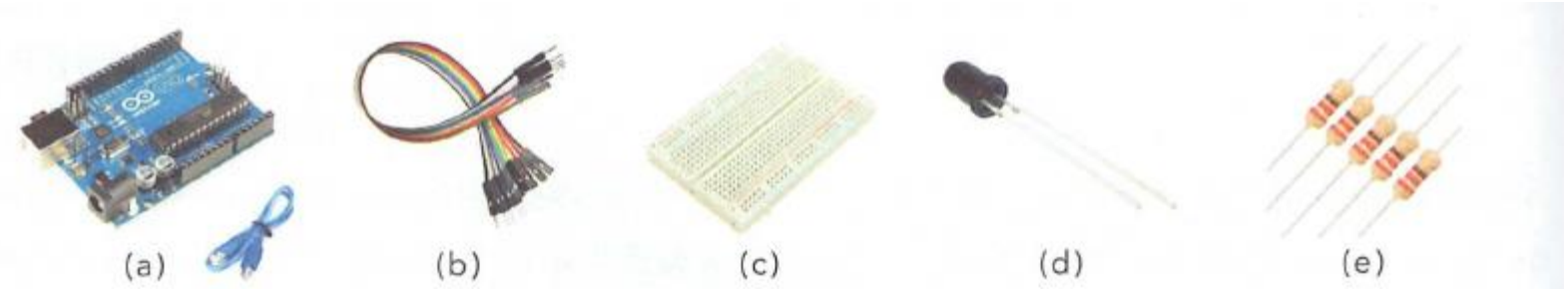
1. 화재 감지 센서

개요

- 아두이노를 활용하여 화재 감지 센서 제어에 대하여 알아봄
- 화재 감지 센서에서 출력되는 아날로그 전압을 디지털 값으로 AD 변환하여 아두이노 시리얼 모니터를 통하여 확인함.

사용 디바이스

- 아두이노 우노 보드(a), 점퍼 케이블(b), 브레드 보드(c), 화재감지센서(d), 저항_10k(e)



1. 화재 감지 센서

- 화재감지센서(포토트랜지스터)
  - 화재 감지 센서 또는 불꽃 감지 센서라고 불리며 적외선 및 빛을 감지하여 전기에너지로 변환 시켜 증폭하는 작용을 하는 센서
  - 다리가 긴 쪽 : 에미터(Emitter)
  - 다리가 짧은 쪽 : 컬렉터(Collector)
  - 센서는 적외선(IR) 스펙트럼에서 발생하는 불꽃의 빛을 판별함. 주로 760nm~1100nm 범위의 적외선을 감지할 수 있음. 또한 이산화탄소발생 구간에서 적외선이 방출되는 에너지가 커지는 특성을 이용하여 화재 여부를 감지함.

CO_2는 특정 적외선 파장(보통 **4.26μm** 근처)을 흡수하거나 방출하는 특성이 있음

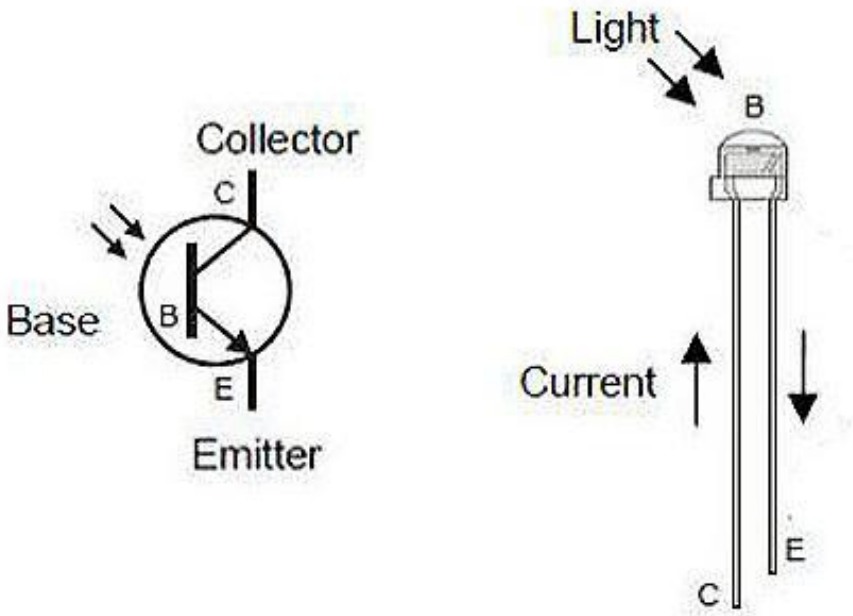
불꽃은 여러 파장의 빛을 동시에 냄
불이 타면 눈에 보이는 **가시광선(빨강, 노랑)** 만 나오는 게 아니라 **적외선(IR), 자외선(UV)** 같은 보이지 않는 파장도 함께 나옴



1. 화재 감지 센서

- 화재감지센서(포토트랜지스터)
  - 다리가 긴 쪽 : 에미터(Emitter)
  - 다리가 짧은 쪽 : 컬렉터(Collector)

구분	기능
Collector(컬렉터)	들어온 적외선을 감지 → 전류 변화 발생
Emitter(에미터)	전류가 빠져나가는 경로, 감지된 전류를 출력



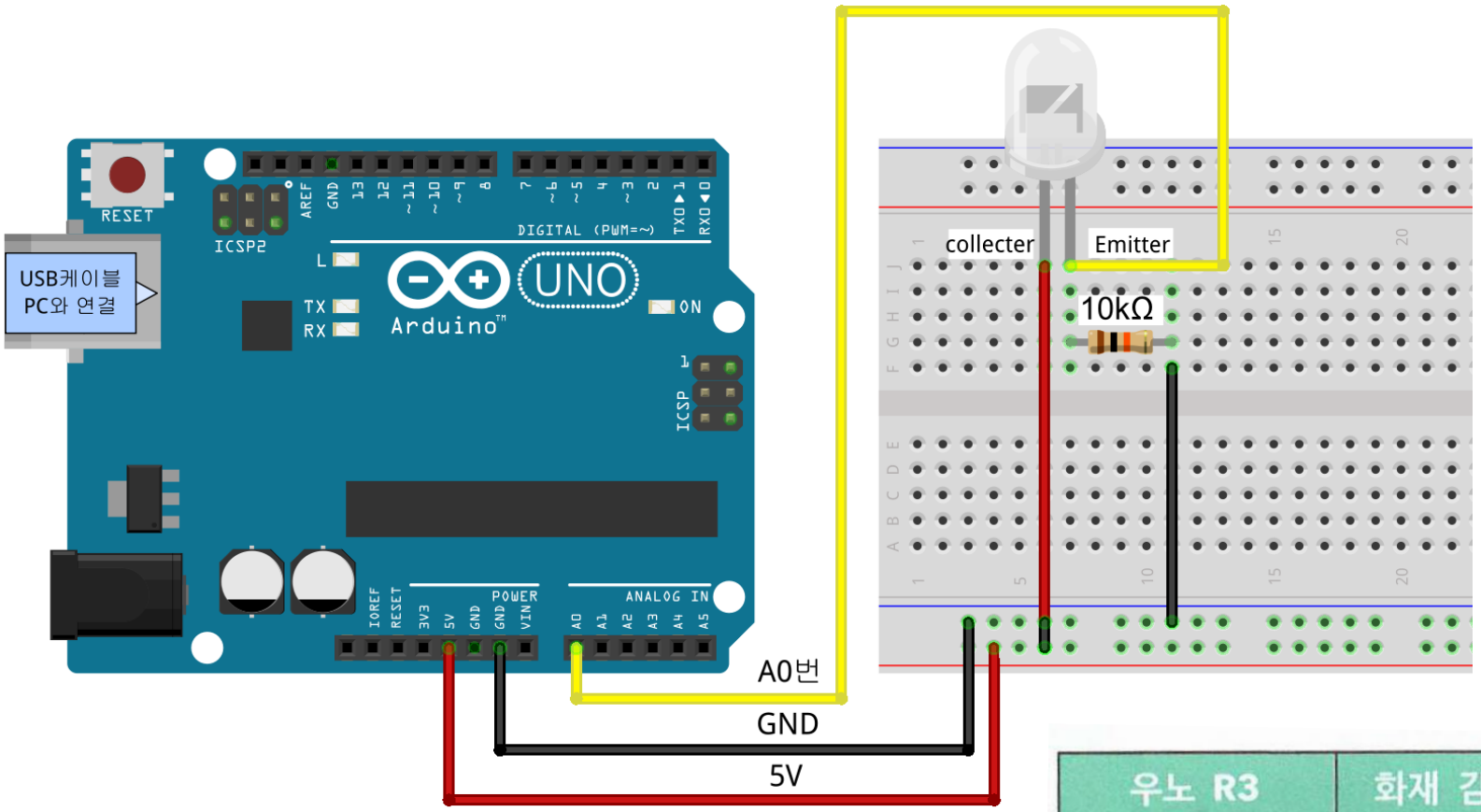
1. 화재 감지 센서

- 화재 감지 센서 – 감지 각도
  - 화재 감지 센서의 감지 각도는 아래 그림과 같이 일반적으로 60도이며 이 범위에 해당하는 각도에서만 불꽃 감지가 가능함.



  - 센서의 감지 범위를 넓히고 효과적으로 커버하기 위해서 화재 감지 센서를 높은 위치에 설치하거나 각 센서가 서로 겹치는 범위를 가지도록 배치하여 사각지대를 최소화 해야 함.

2. 화재 감지 센서 - 회로도



우노 R3	화재 감지 센서
아날로그 0번핀	에미터 핀
5V핀	브레드보드 5V
GND핀	브레드보드 GND

3. 화재 감지 센서 – 실습 코드

```
#define flame A0 // 센서 핀 A0로 설정
int val ;
void setup() {  // 초기화
  Serial.begin(9600); // 시리얼통신속도 설정
  pinMode(flame, INPUT);  // 센서 핀 입력 핀 설정
}
void loop() { // 무한루프
  val = analogRead(flame);  // 센서의 아날로그 값을 읽어와 변수에 저장
  Serial.println(val);  // 변수 값을 시리얼모니터에 출력
  delay(200); // 딜레이 200 ms
}
```

3. 화재 감지 센서 – 실습 코드

- 센서 출력 = 전압(아날로그 신호)
  - 포토트랜지스터가 적외선을 감지하면

    → 전류가 변하고

    → 그 전류가 전압 변화로 바뀌어서 A0 핀에 전달
- 예 :
  - 적외선 강함 → 전압 높음
  - 적외선 약함 → 전압 낮음

- analogRead()는 그 전압을 측정 → 숫자로 변환
  - 아두이노의 ADC(Analog to Digital Converter)가 0~5V 전압을

    0~1023 사이의 정수로 변환

4. 화재 감지 센서 – 실습 참고 영상



5. 자이로 센서

- 개요
  - 아두이노를 활용한 자이로 센서 모듈 제어에 대하여 알아봄
  - 자이로 센서 모듈과 우노 간의 I2C 통신을 통하여 자이로와 기울기 X, Y, Z 축 값을 얻은 후 아두이노 스케치 시리얼 모니터를 통해 확인함.

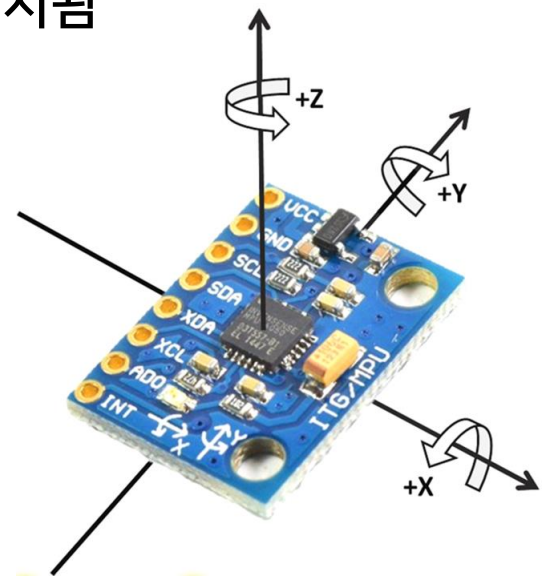
- 사용 디바이스
  - 아두이노 우노 보드, 점퍼 케이블, 자이로 센서 모듈

5. 자이로 센서

- MPU6050 이란?
  - MPU6050은 6축(6DOF, Degrees of Freedom) 모션 트래킹 장치로, 3축 자이로스코프와 3축 가속도계를 하나의 칩에 결합한 센서임.
  - 이 센서는 주로 드론, 로봇, 게임 장비, 웨어러블 디바이스 등에서 자세 추적, 모션 감지, 관성 측정 등 다양한 용도로 사용됨.
  - 3축 자이로스코프 및 가속도계 등의 주요 기능을 가짐
  - 3축 자이로스코프 : 각속도(회전 속도)를 측정하며, 각 축(X, Y, Z)의 회전율을 감지함. 단위는 도/초(degrees per second, dps)
  - 3축 가속도는 가속도를 측정하여 각 축(X, Y, Z)에서의 가속도를 감지함. 단위는 g(중력 가속도)임.

5. 자이로 센서 – 원리

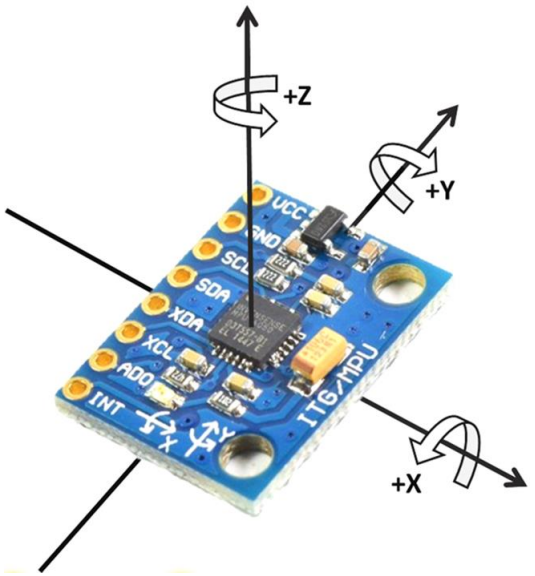
- 3축 가속도계(Accelerometer)의 원리
  - 동작 원리: MEMS 질량-스프링 구조
  - 가속도계는 내부에 작은 진자(질량, Mass)와 스프링 구조가 있는 MEMS(Micro Electro Mechanical System) 로 이루어져 있음
  - 센서가 움직이거나 기울어지면→ 내부의 미세 질량이 관성 때문에 움직임
  - 그 이동 거리가 정전용량(Capacitance) 변화로 감지됨
  - 정전용량 변화를 전기 신호로 변환
  - X, Y, Z 축 각각의 가속도를 계산하여 기울어져 있는 정도를 나타냄



5. 자이로 센서 – 원리

- 3축 자이로스코프(Gyroscope)의 원리
  - 동작 원리: 진동하는 MEMS 구조의 Coriolis Effect(코리올리 힘)
  - MPU6050의 자이로는 진동형 MEMS 자이로임.
  - 내부에 아주 작은 구조물이 일정 방향으로 심하게 진동하고 있음
  - 센서가 회전하면, 회전하는 물체의 진동 방향이 변하며 코리올리 힘이 발생
  - 이 힘이 진동 방향을 약간 변화시키고
  - 그 변화량을 정전용량 혹은 저항 변화로 감지
  - X, Y, Z 각 축에 대한 각속도(°/s) 측정

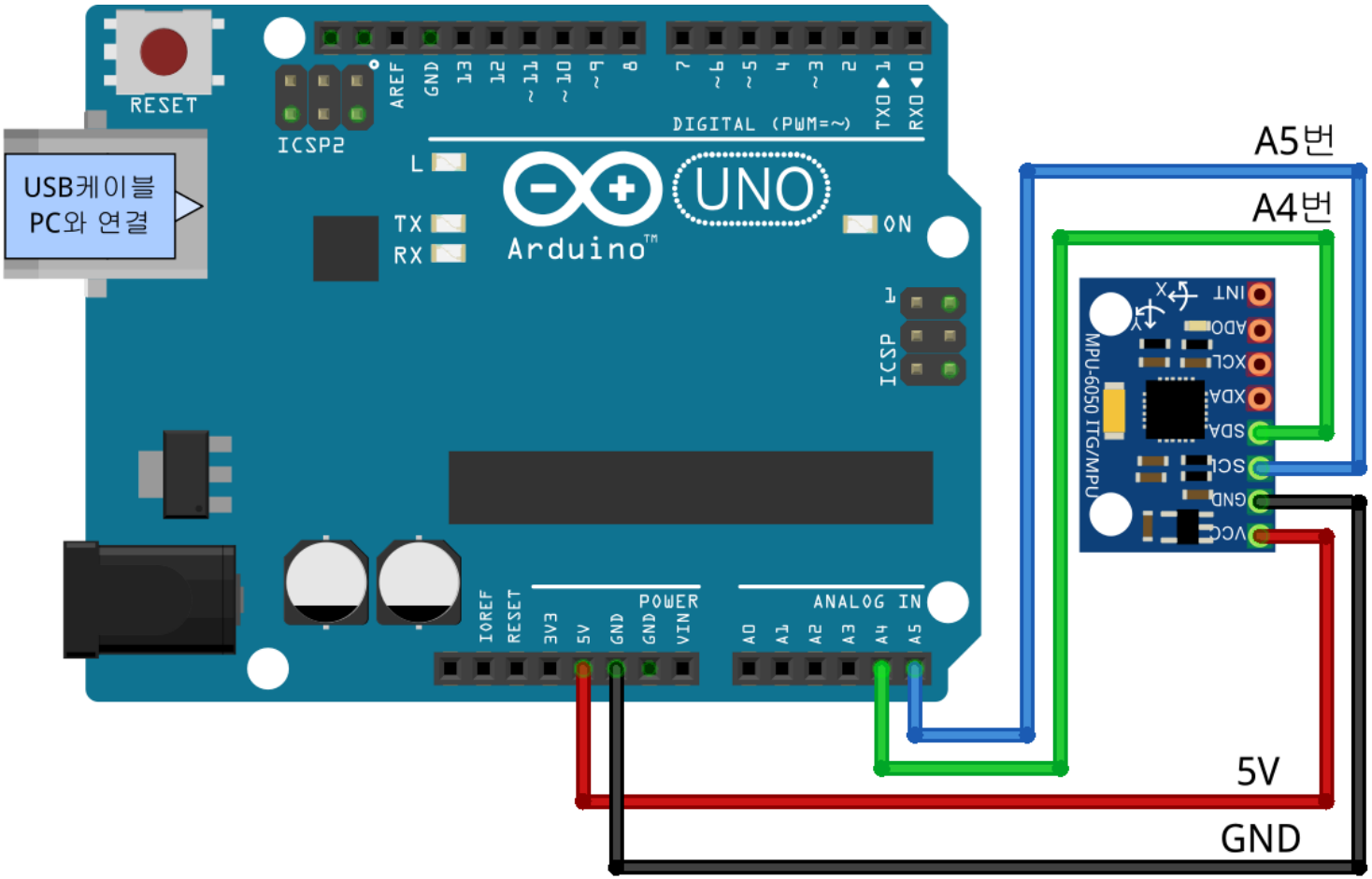
(각속도: 물체가 얼마나 빨리 회전하는지 나타내는 값)



5. 자이로 센서 – 통신

- I2C 통신이란?
  - I2C(Inter-Integrated Circuit)는 SDA(Data) : 데이터 송수신, SCL(Clock) : 클록 신호, 두 선을 이용해 여러 장치와 동시에 통신하는 버스 방식
- 핵심 특징
  - 선 두 개만 사용
  - 여러 장치를 하나의 버스에 연결 가능
  - 각 장치는 고유 주소(I2C Address) 가 있음
  - 마스터(보통 아두이노)와 슬레이브(센서) 구조

5. 자이로 센서 – 회로도


USB케이블
PC와 연결
RESET
ICSP2
AREF
GND
13
12
11
10
9
8
7
6
5
4
3
2
TX→1
RX←0
DIGITAL (PWM=~)
L
TX
RX
UNO
Arduino™
ON
ICSP
IOREF
RESET
3V3
5V
GND
GND
VIN
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
A5번
A4번
INT
ADO
XCL
XDA
SDA
SCL
GND
VCC
MPU-6050 ITG/MPU
5V
GND

5. 자이로 센서 – 실습 코드

```
/* I2C 통신을 위해 Wire.h 헤더 파일을 include 한다. */
#include <Wire.h>

// MPU6050의 I2C 주소(기본값은 0x68)
// ADO 핀을 HIGH로 하면 주소는 0x69가 됨
const int MPU_addr = 0x68;


// MPU6050 센서로부터 읽을 16비트 데이터 변수 선언
int16_t AcX, AcY, AcZ;  // 가속도 센서 값 (X, Y, Z축)
int16_t Tmp;            // 온도 센서 값
int16_t GyX, GyY, GyZ;  // 자이로 센서 값 (X, Y, Z축)


void setup() // 초기화 함수 (아두이노 전원이 켜지면 한 번 실행됨)
{
  Wire.begin(); // I2C 통신 시작 (SDA/SCL 핀 활성화)

  // MPU6050을 깨우기 위한 설정
  Wire.beginTransmission(MPU_addr); // MPU6050에 데이터 전송 시작
  Wire.write(0x6B);                 // PWR_MGMT_1 레지스터 선택 : 전원관리와 절전모드 관리
  Wire.write(0);                    // 0을 쓰면 sleep 모드 해제 → 센서 활성화
  Wire.endTransmission(true);       // 전송 끝

  Serial.begin(9600); // 시리얼 통신 초기화 (PC와 데이터 주고받기)
}
```

5. 자이로 센서 – 실습 코드

```
/*
 MPU6050 모듈은 총 6축(3축 가속도 + 3축 자이로)을 측정할 수 있으며
 이 데이터를 이용해 기울기, 회전 등 다양한 동작을 해석할 수 있다.
 PC의 시리얼 모니터에서 값을 출력하여 확인할 수 있다.
*/
void loop() // 무한 반복 (센서 데이터를 계속 읽음)
{
  // --------------- MPU6050의 레지스터 처음 위치 설정 ---------------
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);  // ACCEL_XOUT_H 레지스터 주소 (가속도 X축 상위 바이트가 저장된 주소)
  Wire.endTransmission(false);
  // false → 반복 시작(restart condition), 연속 읽기 준비

  // --------------- 총 14바이트 요청 ---------------
  // ACCEL X(2) + Y(2) + Z(2) + TEMP(2) + GYRO X(2) + Y(2) + Z(2)
  Wire.requestFrom(MPU_addr, 14, true);
// MPU6050은 하나의 16비트 데이터를 상위 8비트와
하위 8비트로 나누어 보냄
  // ----------- 가속도 데이터 읽기 -----------
  AcX = Wire.read() << 8 | Wire.read();  // X축
  AcY = Wire.read() << 8 | Wire.read();  // Y축
  AcZ = Wire.read() << 8 | Wire.read();  // Z축

  // ----------- 온도 데이터 읽기 -----------
  Tmp = Wire.read() << 8 | Wire.read();
```

바이트 위치	내용	설명
0-1	ACCEL_X	X축 가속도 (2바이트)
2-3	ACCEL_Y	Y축 가속도 (2바이트)
4-5	ACCEL_Z	Z축 가속도 (2바이트)
6-7	TEMP	온도 (2바이트)
8-9	GYRO_X	X축 각속도 (2바이트)
10-11	GYRO_Y	Y축 각속도 (2바이트)
12-13	GYRO_Z	Z축 각속도 (2바이트)

5. 자이로 센서 – 실습 코드

```
// ----------- 자이로 데이터 읽기 -----------
  GyX = Wire.read() << 8 | Wire.read();  // X축 회전속도
  GyY = Wire.read() << 8 | Wire.read();  // Y축 회전속도
  GyZ = Wire.read() << 8 | Wire.read();  // Z축 회전속도


  // --------------- 시리얼 출력 ---------------
  Serial.println(F("MPU-6050"));

  Serial.print(F("accel x,y,z: "));
  Serial.print("AcX = "); Serial.print(AcX);
  Serial.print(" | AcY = "); Serial.print(AcY);
  Serial.print(" | AcZ = "); Serial.print(AcZ);

  Serial.print(F("temperature: "));
  Serial.print(" | Tmp = ");
  Serial.print(Tmp / 340.00 + 36.53);
  // 데이터시트에 있는 온도 변환 공식 적용

  Serial.print(F("gyro x,y,z: "));
  Serial.print(" | GyX = "); Serial.print(GyX);
  Serial.print(" | GyY = "); Serial.print(GyY);
  Serial.print(" | GyZ = "); Serial.println(GyZ);

  delay(333); // 0.33초마다 한 번씩 데이터를 출력
}
```

센서 로우 데이터

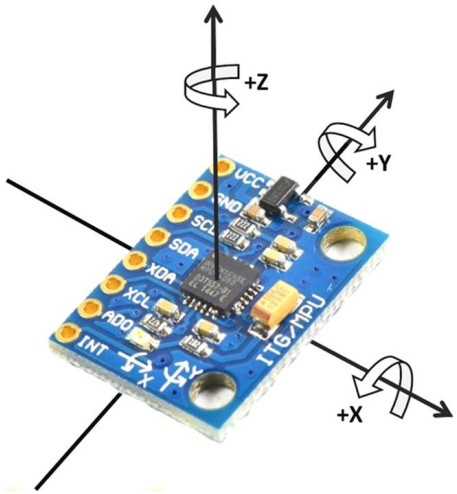
```
Gyro x,y,z :  X = 157 | Y = 526 | Z = 522 Accel x,y,z: X = 648 | Y = 16328 | Z = -1088 temperature: 30.08
Gyro x,y,z :  X = -421 | Y = -265 | Z = -73 Accel x,y,z: X = 712 | Y = 15984 | Z = -1176 temperature: 30.13
Gyro x,y,z :  X = -354 | Y = 357 | Z = 41 Accel x,y,z: X = 588 | Y = 16232 | Z = -776 temperature: 30.13
Gyro x,y,z :  X = -350 | Y = 168 | Z = -231 Accel x,y,z: X = 840 | Y = 15860 | Z = -1080 temperature: 30.13
Gyro x,y,z :  X = -359 | Y = 380 | Z = -272 Accel x,y,z: X = 708 | Y = 16120 | Z = -1064 temperature: 30.13
Gyro x,y,z :  X = -537 | Y = -9 | Z = -252 Accel x,y,z: X = 812 | Y = 16152 | Z = -1064 temperature: 30.22
Gyro x,y,z :  X = -368 | Y = -45 | Z = 33 Accel x,y,z: X = 720 | Y = 15996 | Z = -1080 temperature: 30.13
Gyro x,y,z :  X = -2070 | Y = 495 | Z = -624 Accel x,y,z: X = 804 | Y = 16464 | Z = -1464 temperature: 30.22
Gyro x,y,z :  X = -10315 | Y = -1608 | Z = -4464 Accel x,y,z: X = -240 | Y = 16480 | Z = 748 temperature: 30.32
Gyro x,y,z :  X = -16229 | Y = -539 | Z = -7701 Accel x,y,z: X = -832 | Y = 16368 | Z = 2964 temperature: 30.13
Gyro x,y,z :  X = -11727 | Y = -2215 | Z = 1546 Accel x,y,z: X = -2556 | Y = 12708 | Z = 9812 temperature: 30.27
Gyro x,y,z :  X = -14777 | Y = -6738 | Z = 2760 Accel x,y,z: X = -1348 | Y = 11716 | Z = 11980 temperature: 30.18
Gyro x,y,z :  X = -4802 | Y = -3595 | Z = 954 Accel x,y,z: X = -1420 | Y = 8724 | Z = 13976 temperature: 30.22
Gyro x,y,z :  X = -8448 | Y = -4116 | Z = 718 Accel x,y,z: X = -440 | Y = 6156 | Z = 15132 temperature: 30.22
Gyro x,y,z :  X = -5705 | Y = -1601 | Z = 2407 Accel x,y,z: X = 144 | Y = 5044 | Z = 16480 temperature: 30.18
Gyro x,y,z :  X = -1581 | Y = -230 | Z = -3580 Accel x,y,z: X = 228 | Y = 2204 | Z = 16964 temperature: 30.32
Gyro x,y,z :  X = -1874 | Y = -5215 | Z = -1929 Accel x,y,z: X = 1060 | Y = 1664 | Z = 17196 temperature: 30.22
Gyro x,y,z :  X = -12329 | Y = -914 | Z = 4275 Accel x,y,z: X = 1692 | Y = 380 | Z = 17860 temperature: 30.32
Gyro x,y,z :  X = 278 | Y = 2372 | Z = -16 Accel x,y,z: X = 2244 | Y = -352 | Z = 17096 temperature: 30.22
Gyro x,y,z :  X = 1361 | Y = 6882 | Z = -6603 Accel x,y,z: X = 1076 | Y = 768 | Z = 16348 temperature: 30.18
Gyro x,y,z :  X = 1425 | Y = 2317 | Z = -1294 Accel x,y,z: X = 1256 | Y = 704 | Z = 17100 temperature: 30.27
Gyro x,y,z :  X = -986 | Y = -5755 | Z = 5637 Accel x,y,z: X = 1012 | Y = 652 | Z = 17220 temperature: 30.32
Gyro x,y,z :  X = 1062 | Y = 104 | Z = -391 Accel x,y,z: X = 1104 | Y = 1672 | Z = 17180 temperature: 30.27
Gyro x,y,z :  X = 3419 | Y = -2418 | Z = 5645 Accel x,y,z: X = 928 | Y = 2220 | Z = 16448 temperature: 30.27
Gyro x,y,z :  X = 5108 | Y = -1476 | Z = -2156 Accel x,y,z: X = 776 | Y = 2680 | Z = 17724 temperature: 30.32
Gyro x,y,z :  X = -1212 | Y = 410 | Z = -1691 Accel x,y,z: X = 1056 | Y = 2336 | Z = 17040 temperature: 30.37
Gyro x,y,z :  X = -883 | Y = -459 | Z = -806 Accel x,y,z: X = 1060 | Y = 2604 | Z = 17416 temperature: 30.32
Gyro x,y,z :  X = -2987 | Y = -1823 | Z = -434 Accel x,y,z: X = 1340 | Y = 1724 | Z = 15784 temperature: 30.32
Gyro x,y,z :  X = -6591 | Y = 12 | Z = 778 Accel x,y,z: X = 1464 | Y = 1784 | Z = 17900 temperature: 30.46
Gyro x,y,z :  X = -5191 | Y = 13 | Z = 834 Accel x,y,z: X = 1784 | Y = -1400 | Z = 16368 temperature: 30.27
Gyro x,y,z :  X = -15233 | Y = -11148 | Z = 4250 Accel x,y,z: X = 1388 | Y = -3032 | Z = 17676 temperature: 30.32
Gyro x,y,z :  X = -7297 | Y = 11439 | Z = 5311 Accel x,y,z: X = 3880 | Y = -9036 | Z = 13184 temperature: 30.37
Gyro x,y,z :  X = -4737 | Y = 3346 | Z = 932 Accel x,y,z: X = 1836 | Y = -10548 | Z = 13308 temperature: 30.37
Gyro x,y,z :  X = -10874 | Y = 3201 | Z = -2876 Accel x,y,z: X = 508 | Y = -12864 | Z = 11048 temperature: 30.41
Gyro x,y,z :  X = -4214 | Y = 593 | Z = -142 Accel x,y,z: X = 1372 | Y = -15272 | Z = 7252 tempe
```

로우 데이터로부터 가속도 계산

- MPU6050의 가속도 센서(accelerometer) 는 물체가 받는 가속도를 측정하는데, 그 기준 단위가 g(gravity)임.
- 예) 2바이트는 -32768 ~ 32768범위 데이터
  - 아무 움직임 없이 가만히 둘 때
    → Z축에서 약 +1g(실제 중력 값)
  - 기울이면 X축이나 Y축에서 g 값이 달라짐
  - 기기를 평평하게 놓으면
  - Z축 +1g, X,Y 0g
  - X축을 90° 기울이면
  - X축 +1g, Z축 0g

$$\text{가속도 (g)} = \frac{\text{Raw Value (AcX, AcY, AcZ)}}{\text{Sensitivity Scale Factor}}$$

범위	Scale Factor (LSB/g)
±2g	16384
±4g	8192
±8g	4096
±16g	2048



로우 데이터로부터 자이로스코프 계산

- MPU6050 같은 자이로스코프 센서는 각속도(angular velocity)를 측정함. 각속도란, 단위 시간당 회전하는 속도이고 단위는 보통 °/s (degrees per second, 초당 회전 각도)로 나타냄.
- ±250°/s 기준일 때
  - ±32768 raw → ±250°/s 기준 ➔ 아래 공식으로 계산할 수 있음.

$$\text{deg/s} = \text{raw} \times \frac{250}{32768}$$

범위	Scale Factor (LSB/°/s)
±250°/s	131
±500°/s	65.5
±1000°/s	32.8
±2000°/s	16.4

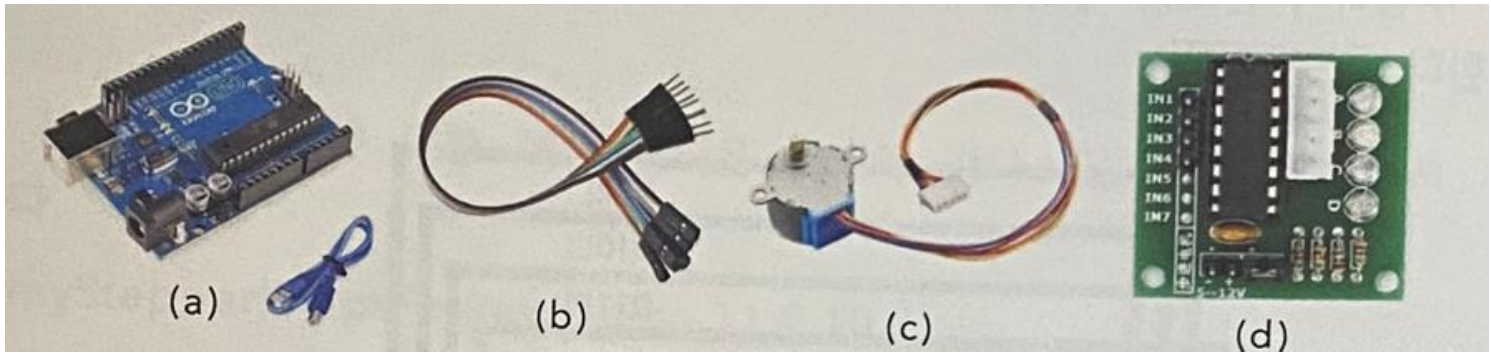
3. 스테핑 모터

개요

- 아두이노를 활용하여 스테핑 모터 제어에 대하여 알아봄
- 우노에서 스테핑 모터로 펄스를 보내 0.5초마다 정방향, 역방향으로 변화시켜 구동하는 동작을 구현

사용 디바이스

- 아두이노 우노 보드(a), 점퍼 케이블(b), 스테핑 모터(c), 모터드라이버(d)

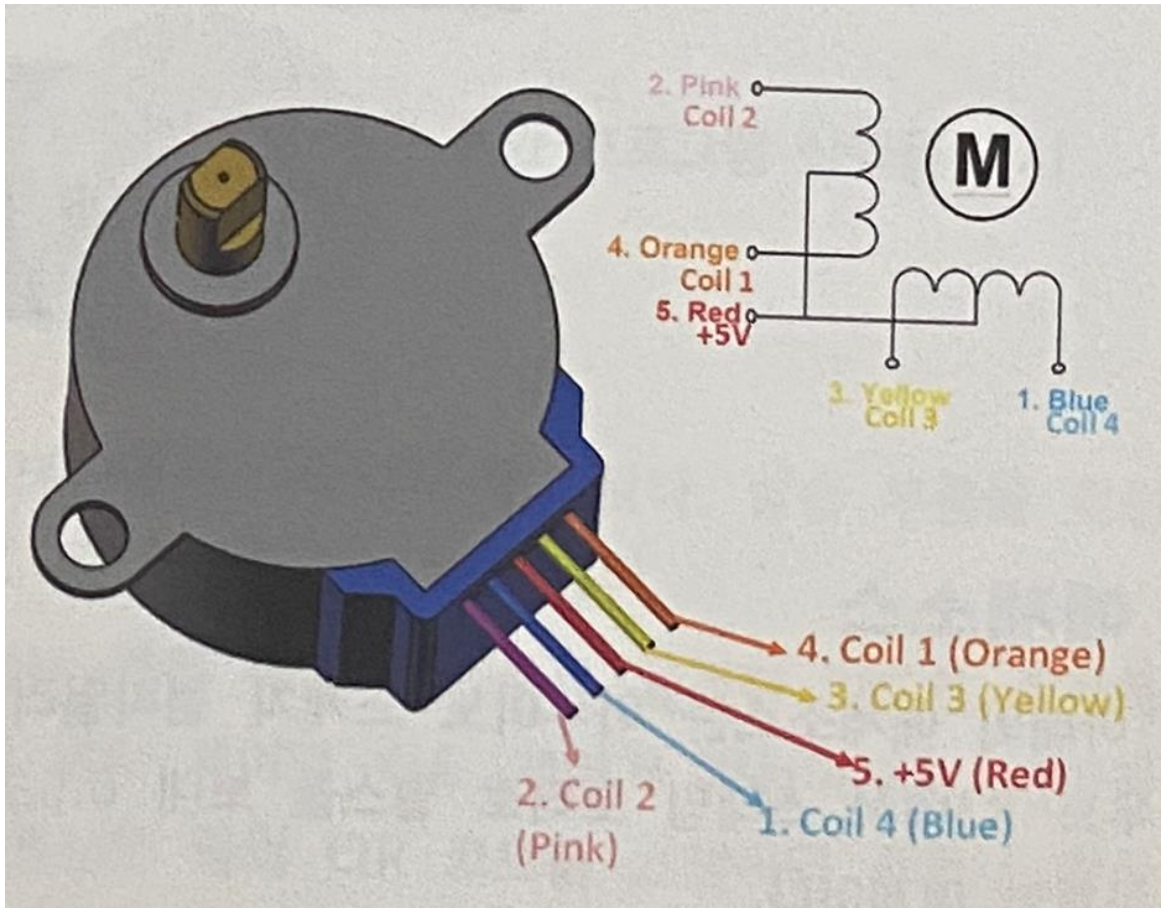


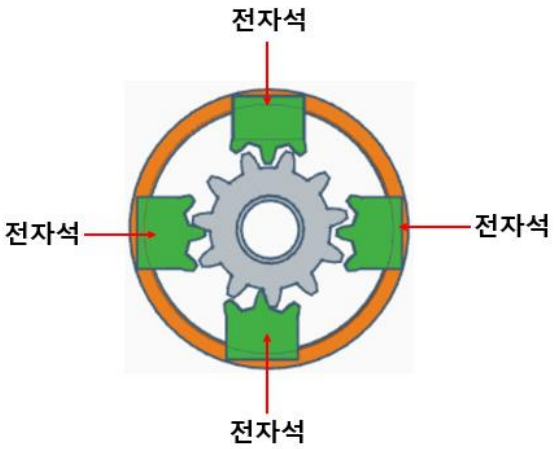
3. 스테핑 모터

- 스테핑 모터 배경 지식
  - 스테핑 모터는 전원을 입력하고 펄스를 넣어줘야만 회전하는 모터
  - 또한 <그림 1>처럼 구조를 띄고 있으며, 전원핀과 펄스 입력핀을 확인해 배선할 수 있음.
  - 특히 스테핑 모터 구동을 위해서는 1, 2, 3, 4번핀에 펄스를 입력해야 함. 보통 스테핑 모터축은 1펄스당 1.8도의 회전을 하는데 이 수치는 스테핑 모터마다 다를 수 있음(실습 예제는 28BYJ-48 모터: 한 펄스당 회전 각도는 0.1758°)
  - 펄스 단위로 움직이는 모터이므로 정확한 각도, 속도 제어가 가능함. (예를 들어 90도를 움직인다고 했을 때 펄스를 50개만 넣어 주면 됨)
  - 속도 또한 1분에 30바퀴를 회전한다고 했을 때 1분당 6000펄스 초당 100펄스를 넣어 준다면 1분에 30바퀴를 정확히 회전할 수 있음.

3. 스테핑 모터

스테핑 모터 구조





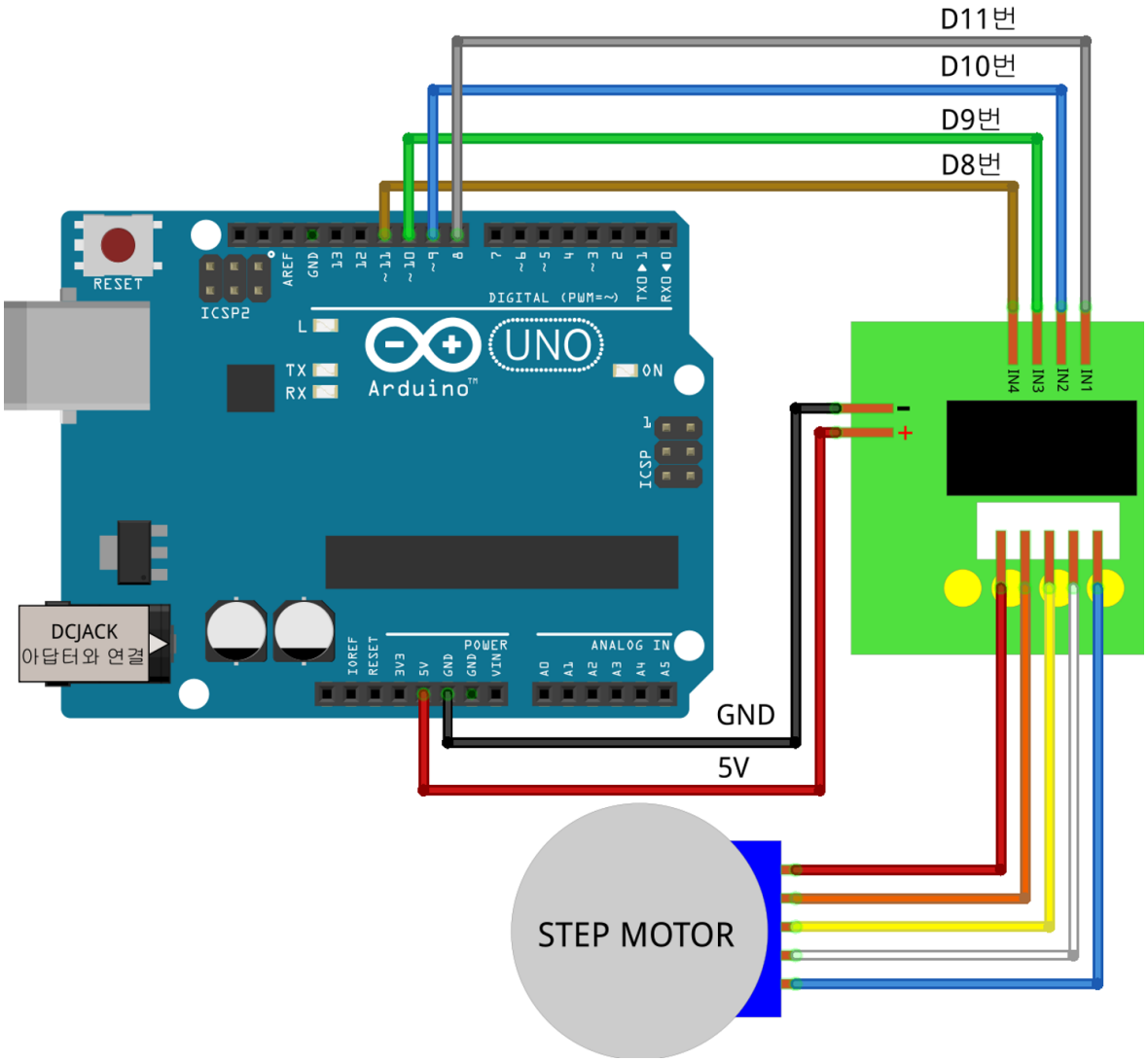
<그림 1. 스테핑 모터 구조>

3. 스테핑 모터

- 스테핑 모터 구현 소스 코드 및 구현
  - 우노 R3 디지털 8번핀을 모터 드라이버의 IN1에 연결하고 디지털 9번핀을 모터 드라이버 IN2핀에 연결
  - 우노 R3 디지털 10번핀과 디지털 11번핀 또한 모터 드라이버 IN3핀과, IN4핀에 각각 연결
  - 우노 R3의 5V핀은 모터 드라이버 5V핀에 우노 R3의 GND핀은 모터 드라이버 GND핀에 연결하고 스테핑 모터는 모터 드라이버의 커넥터에 알맞게 꽂으면 됨. 또한 모터 구동 시 우노 R3의 USB 전원만으로는 무리가 될 수 있으니 배터리 케이스나 DC 전원 아답터 등을 사용하는 것을 권장함.
  - 모터 드라이버의 5V핀과 GND핀에 전원을 투입할 때 반대로 전원을 투입하지 않도록 주의. 5V, GND핀에 전원을 입력할 때는 반드시 극성에 유의해 연결함.

3. 스테핑 모터

- 스테핑 모터 구현 회로도



3. 스테핑 모터 – 실습 코드

```
#include <Stepper.h>  // 스테핑 모터 라이브러리
// 상수 정의
const int stepsPerRevolution = 2048; //스테핑 모터가 한 바퀴(360°)를 돌기 위해 필요한 스텝 수
// 2048: 한 바퀴(360도), 1024: 반바퀴(180도)
// 핀 연결 (IN4, IN2, IN3, IN1 순서)
Stepper myStepper(stepsPerRevolution, 11, 9, 10, 8);//myStepper 객체 생성, 핀 설정
void setup() {
  // 모터 속도 설정 (rpm)
  myStepper.setSpeed(14);//모터 속도를 14 rpm(분당 회전 수)으로 설정
}
void loop() {
  // 정방향 회전: 시계 반대 방향
  myStepper.step(stepsPerRevolution); //step() 매개변수 > 0 : 정방향
  delay(500);  // 500ms 지연
  // 역방향 회전: 시계 방향
  myStepper.step(-stepsPerRevolution); //매개변수 < 0 : 역방향
  delay(500);  // 500ms 지연
}
```

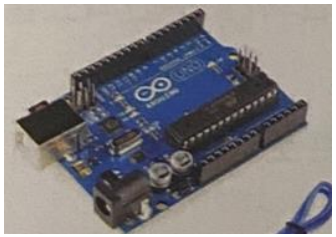
3. 스테핑 모터 – 실습 코드

- Stepper myStepper (steps Per Revolution, 11,9,10,8) : 8,10,9,11 숫자들은 스테핑 모터의 4개의 제어 핀을 아두이노 보드의 디지털 핀 번호에 연결하기 위한 것임.
- StepsPerRevolution : 모터의 회전당 스텝 수를 나타내는 변수임. 예를 들어 스테핑 모터가 한바퀴(360도) 회전하기 위해 몇 개의 스텝을 실행해야 하는지를 나타냄. 이 값은 모터의 해상도에 따라 다르며, 일반적으로 1024, 2048 값 등으로 설정
- myStepper.setSpeed(value) : 아두이노 함수에서 스테핑 모터를 제어할 때 사용하는 함수임. 이 함수는 스테핑 모터의 회전 속도를 설정하는 데 사용됨.
- delay(500) : 스테핑 모터가 동작 후 0.5초 동안 대기함. Delay() 함수는 일정 시간 동안 프로그램 실행을 일시정지시킴

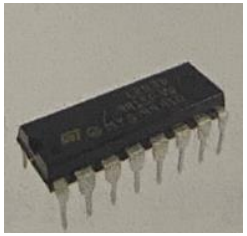
4. L298D DC 모터

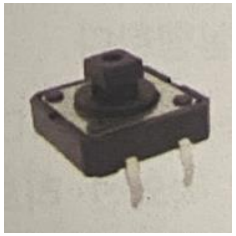
- 개요
  - 우노 R3를 이용하여 L293D 모터 드라이버와 DC모터 제어함
  - 프로그래밍 동작은 모터 드라이버인 L293D으로 DC모터의 반시계 방향 회전, 시계 방향 회전, 정지를 시킬 수 있는 예제임.

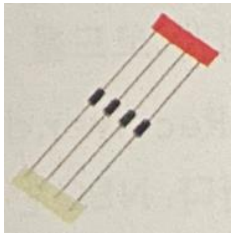
- 사용 디바이스 : 우노 R3(a), DC 모터(b), L298D(c), 스위치(d), 1N4001 다이오드(e), 저항 10k(f)


(a)


(b)


(c)


(d)

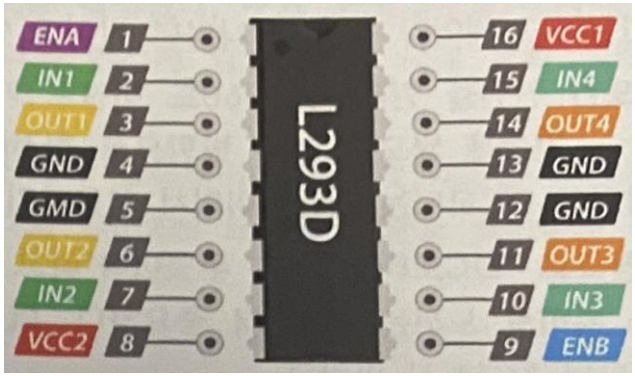

(e)


(f)

1N4001 다이오드 : 모터에 전류를 흘렸다가 끊으면 모터 코일에 역기전력(back EMF)이 발생. 모터에서 발생하는 역기전력(Back EMF)을 보호, 회로 손상 방지

4. L298D DC 모터

- L293D 모터드라이버 IC
  - <그림 2>처럼 L293D는 L298N 모터 드라이버와 더불어 대표적인 범용 DC모터 드라이버로 H-브릿지 회로가 내장된 소형 모터 드라이버임.
  - L293D 모터 드라이버 IC는 우노 R3와 DC모터 사이에서 인터페이스 되어 동작되며 동시에 2개의 모터 동작을 제어할 수 있음.
  - 또한 2개의 H 브릿지 회로는 저전력 DC모터를 제어하기에 적합하도록 설계되었음. 특히 L293D 모터 드라이버는 아래와 같은 기능을 제공함.
    - DC 모터 회전제어 정회전, 역회전
    - DC 모터 정지
    - DC 모터 속도조절(PWM 신호)



<그림 2. L293D 모터드라이버 IC>

4. L298D DC 모터

- DC모터 동작을 위한 입력 진리표
  - <표 1>에서는 DC모터 동작을 위한 입력 진리표를 보여줌. 입력 진리표에 따라 모터 제어 시 입력해야 할 신호를 아래에서 설명하고 있음.
  - IN1, IN2핀은 DC모터의 회전방향을 선택할 수 있는 핀으로 각각 하이 혹은 로우 신호를 입력 가능. 또한 이 핀들에 동시에 같은 신호를 입력할 경우에는 DC 모터에 동작을 정지시킬 수도 있음.
  - DC모터 속도 조절시에는 ENA 혹은 ENB에서 PWM 펄스를 입력해야 하며, 만일 2개의 DC모터 속도조절을 원할 경우에는 각각의 ENA, ENB핀에 PWM 펄스를 입력하여 조절함.

ENA, B 입력	IN1, 2 입력	IN1, 2 입력	모터 1,2 동작
하이 or PWM 펄스	하이(HIGH)	하이	정지
	하이	로우(LOW)	정방향 회전
	로우	하이	역방향 회전
	로우	로우	정지

<표 1> DC모터 동작을 위한 입력 진리표

4. L298D DC 모터

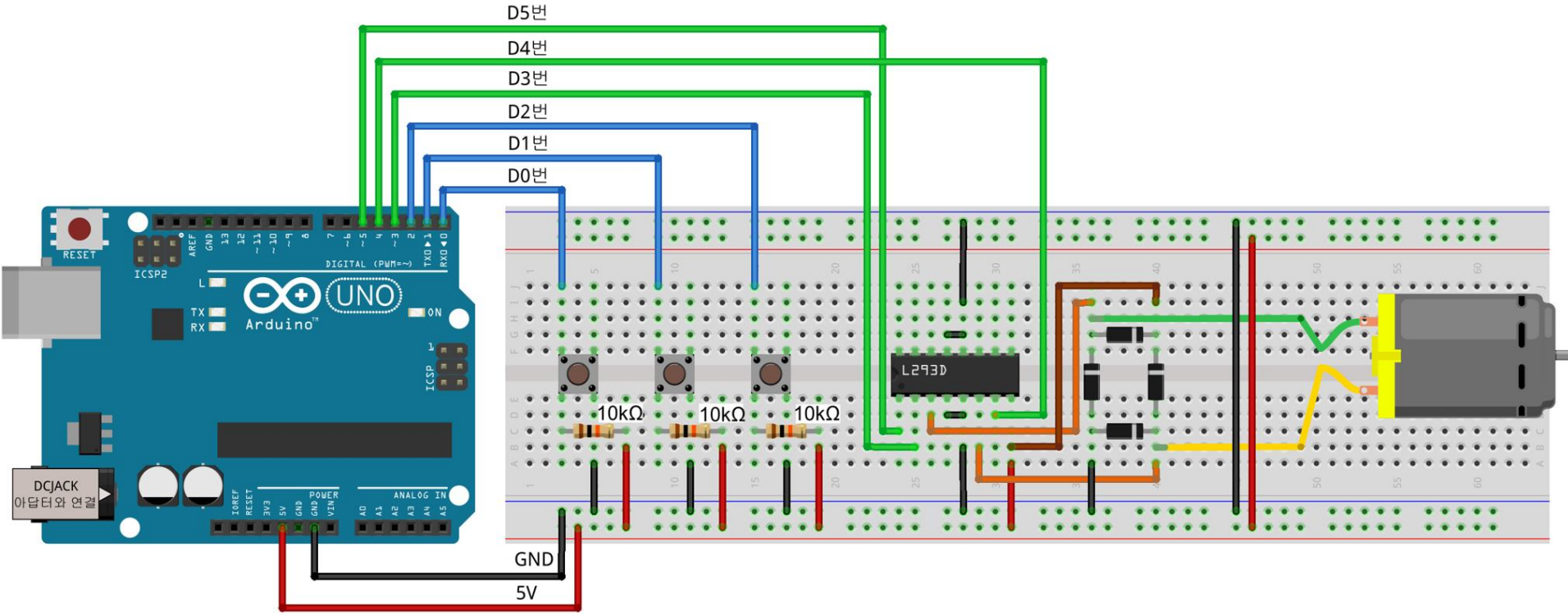
- DC 모터 드라이버가 왜 필요한가?
  - 우노 R3 출력 전압은 3.3~5V, 전류는 최대 25mA 수준임.
  - 이 전력으로는 DC 모터를 충분히 구동할 수 없어, 힘 없이 돌거나 정상 동작 불가능함.
  - 우노 R3로 직접 DC 모터를 구동하면 보드가 고장 나거나 반복적으로 리셋될 위험이 있음. 심하면 우노 R3와 연결된 PC에도 손상이 발생할 수 있음.
  - 따라서 우노 R3와 DC 모터 사이에는 L293D 같은 전용 모터 드라이버가 반드시 필요하고 모터 제어 시에는 항상 모터 드라이버를 사용해야 안전함.
- DC 모터 는 무엇인가?
  - DC 모터는 구조가 단순하고 전원을 연결하면 바로 회전하기 때문에 여러 모터 중 가장 다루기 쉬운 모터임.
  - 전원의 극성을 바꾸면 회전 방향이 변해 제어가 매우 직관적임. 예를 들어 전원을 + / – 방향으로 연결하면 시계 방향으로 회전하고, 극성을 반대로 하여 – / + 방향으로 연결하면 반시계 방향으로 회전함.

4. L298D DC 모터

- 모터 구현 회로도

우노 R3	스위치
디지털 0번	스위치1
디지털 1번	스위치2
디지털 2번	스위치3

우노 R3	L293D
디지털 3번	DIR1
디지털 4번	DIR2
디지털 5번	EN1
5V	+, VSS
GND	-, GND



4. L298D DC 모터

주요 프로그래밍 시나리오는 우노 R3에서 모터 L293D로 신호를 보내 DC모터의 반시계 방향 회전, 시계 방향 회전,정지를 시키는 동작을 구현한 예제이다. EN1은 DC모터 동작을 활성화 시키는 핀으로 하이 신호가 입력되어야 모터가 작동한다. 주로PWM을 넣어 모터의 속도 제어를 위해 사용되며 DIR1과 DIR2는 모터의 방향을 설정하는 핀이다.

```
int Switch1_Pin = 0; // 스위치1
int Switch2_Pin = 1; // 스위치2
int Switch3_Pin = 2; // 스위치3


int DIR1 = 3; // L293D DIR1
int DIR2 = 4; // L293D DIR2
int EN1 = 5; // L293D EN1


void Motor_ccw() { // 반시계 방향 회전
  digitalWrite(DIR1, HIGH); // DIR1 On
  digitalWrite(DIR2, LOW);  // DIR2 Off
  digitalWrite(EN1, HIGH);  // Motor Enable
}
void Motor_cw() // 시계 방향 회전
{
  digitalWrite(DIR1, LOW); // DIR1 Off
  digitalWrite(DIR2, HIGH);// DIR2 On
  digitalWrite(EN1, HIGH); // Motor Enable
}
void Motor_stop() // 회전 정지
{
  digitalWrite(DIR1, HIGH); // DIR1 On
  digitalWrite(DIR2, HIGH); // DIR2 On
  digitalWrite(EN1, LOW); // Motor Disable
}
```

```
void setup() // 초기화
{
  Serial.begin(9600);
  pinMode(Switch1_Pin, INPUT);  // 스위치 1번핀 입력포트 설정
  pinMode(Switch2_Pin, INPUT);  // 스위치 2번핀 입력포트 설정
  pinMode(Switch3_Pin, INPUT);  // 스위치 3번핀 입력포트 설정


  pinMode(DIR1, OUTPUT);    // DIR1 핀 출력포트 설정
  pinMode(DIR2, OUTPUT);    // DIR2 핀 출력포트 설정
  pinMode(EN1, OUTPUT);    // EN1 핀 출력포트 설정
}
/*
   스위치를 이용하여 DC모터에 정회전, 역회전, 정지하도록 한다.
*/
void loop() // 무한루프
{

  if (digitalRead(Switch1_Pin) == LOW)//1번째 스위치 클릭
  {
    Motor_ccw();
  }
  else if (digitalRead(Switch2_Pin) == LOW)//2번째 스위치 클릭
  {
    Motor_cw();
  }
  else if (digitalRead(Switch3_Pin) == LOW)//3번째 스위치 클릭
  {
    Motor_stop();
  }

}
```

Thank You